B EE 425 Study Guide

Microprocessor System Design, UW Bothell, Fall 2026. The full version: every lecture, linked reading, lab and Homework 1-4 topic from the course study site, with worked examples and a key-fact box for each concept. [68000] and [Z80] mark processor-specific topics.

Contents

1. Welcome to BEE 425
2. The design process
3. Designing a PCB
4. Clocks and pulses
5. Microprocessors and microcontrollers
6. The programmer's model
7. Introduction to assembly language
8. First assembly programs
9. Addressing modes
10. Program control, loops and subroutines
11. Logic and shift instructions
12. Buses and memory cells
13. Memory decoding
14. Memory systems and DRAM
15. Interrupts and I/O ports
16. State machines
17. Diodes and transistors as switches
18. Data conversion
19. Noise, isolation and watchdogs
20. Power, compliance and traces
21. Specifications and partitioning
22. Tools for real-time design and debugging
23. Lab 1: using the logic analyzer
24. Lab 2: real-time debugging of a Z80
25. Lab 3: debugging a 68008 system
26. Homework 1: your first PCB
27. Homework 2: your first 68000 program
28. Homework 3: three 68000 programs
29. Homework 4: memory decoding

Welcome to BEE 425

How the course is organised and graded, analog computers, and Moore's law.

How BEE 425 is organised and graded

Learn

BEE 425 is an introductory, **hardware-centric** course on designing, operating, programming and debugging electronic systems that use microprocessors as the control element. The course material runs in these parts:

Part	Topics
0 Basics	Course introduction, designing a microprocessor system, PCB design, project overview, pulses and clocks
1 Architecture	Components of a microprocessor, CISC and RISC, hardware view and programmer's model of the 68K and Z80
2 Assembly language	Software view of the architecture, program structure, instruction set simulators, executable code, addressing modes, loops, subroutines
3 Memory (intro to I/O)	Memory design, buses and tri-state, addressing and expansion, paging
4 I/O design and control	Transistors as switches, open collector, A/D and D/A, parallel and serial ports, analog issues
5 System design issues	Power management, processor selection, compliance engineering, partitioning
6 Design tools	Time permitting

Grading (Lesson 1 rubric): homework **not graded**; three lab experiments **25%**; midterm **25%**; final **25%**; design project **25%**; class participation an additional **5%**. All labs must be completed to receive credit.

The exams test what you did (or did not do) in the labs and homework: "expect to see lab-based questions on your exams", and exam problems "will not be very different than homework problems". Studying solutions is not a substitute for solving problems yourself. The lectures, reading, labs and homework are complementary, and **everything is fair game**.

(The syllabus on Canvas is the authority for dates and policies; this is a summary of what the first lecture says.)

Worked example

A friend plans to skip the labs because 'they barely count'. What is wrong with the plan?

1. Labs are 25% of the grade, and all of them must be completed for credit in the course.
2. The exams, which are half of the grade, ask lab questions.
3. So skipping a lab costs both the lab points and exam points.

> **Key fact:** Part 0 basics (PCB design, clocks, project), 1 architecture, 2 assembly language, 3 memory design, 4 I/O design and control, 5 system design issues, 6 design tools. Grading: homework not graded; 3 labs 25%, midterm 25%, final 25%, project 25%, participation +5%. Exams cover homework, labs and lectures; everything is fair game.

Analog computers and op-amp arithmetic

Learn

Not all computers are digital. During WWII, **analog computers** used vacuum-tube **op-amp** circuits to do mathematics. Such circuits can **add, subtract, multiply by a constant (gain), multiply two variables, divide, exponentiate, take a log or antilog, and generate sine waves**. They can also **differentiate and integrate**. A **logarithmic amplifier** exploits the logarithmic current-voltage relationship of a transistor (with V_T about 28 mV at room temperature); once you have a log amp you can multiply, divide and raise to a power.

The limits: **typical accuracy about 5%** and **limited numeric range**.

Two famous WWII examples:

- The **Torpedo Data Computer (TDC)** on submarines took the angle between the ships, the speeds of sub and target, and the range, and output when to fire.
- The **Norden bombsight** computed the bomb release point from the plane's height, distance to target, speed, wind speed and air resistance; the discovery of the jet stream caused massive errors.

For contrast, **ENIAC** (Eckert and Mauchly, Penn's Moore School, funded by the Army for artillery firing tables) was the first electronic **digital** general-purpose computer: 18,000 vacuum tubes, about 1,900 additions per second, programmed by plugging cables and setting switches.

Worked example

An analog computer with 5% accuracy computes a quantity whose true value is 200. What range could the answer be in?

1. 5% of 200 = 10.
2. The answer could be anywhere from about **190 to 210**.
3. Digital computers avoid this because a binary number can have as many bits of precision as you provide.

> **Key fact:** Op-amp circuits could add, subtract, multiply (by a constant or another variable via log amps), divide, exponentiate, take log/antilog, differentiate, integrate and generate sine waves, with about 5% accuracy and limited range. Examples: the Torpedo Data Computer and the Norden bombsight.

Why analog computers vanished and why they are being revisited

Learn

An **analog computer** is a physical system set up so that **the same equations govern it as govern the problem**. You impose the initial conditions, let its variables evolve in time, and read the answer. A hose filling a bucket is an integrator.

Early machines were mechanical (wheels and gears, tide predictors, the differential analyzer); from the 1940s there were **electronic** ones, used for missile and aircraft simulation and NASA's Mercury, Gemini and Apollo programs. They faded for two reasons:

- they were **complex, quirky and needed specially trained personnel**;
- digital computers offered **programmability, algorithmic operation, ease of storage, high precision and any size of problem**, and rode the rapid improvement of MOS integrated circuits.

Today the digital approach has a cost: **every switching transistor consumes energy**, and solving hard differential equations needs extremely **fine time steps**, hence enormous numbers of calculations. A modern single-chip analog computer works with **no clock**, continuously, and gives answers **within a few**

percent. Passing that answer to a digital computer as an initial guess gave **speed-ups of about 10 times** with similar energy savings. The hybrid keeps the best of both: analog for fast, approximate work; digital for programming, storage and precision.

Worked example

An analog chip solves a differential equation to within 3% in 1 ms using 1 mJ. A digital solver from scratch needs 10 ms and 10 mJ for the precise answer. If the analog answer lets the digital solver finish in 1 ms and 1 mJ, what are the hybrid's time and energy? (Illustrative numbers, not from the article.)

1. Time: 1 ms (analog) + 1 ms (digital refinement) = **2 ms**, five times faster than 10 ms.
2. Energy: 1 mJ + 1 mJ = **2 mJ**, five times less than 10 mJ.
3. The article's reported speed-up for the real chips was about 10 times.

> **Key fact:** Analog computer = a physical system governed by the same equations as the problem. They vanished because they were hard to build, operate and maintain and digital offered programmability, storage and precision. They may return as hybrids: analog gives a fast, low-energy approximation (within a few percent, no clock), digital refines it (about 10x speed-up and energy saving).

Moore's law and process geometry

Learn

Moore's law (first predicted by Intel's founder Gordon Moore) roughly states that the number of transistors on an integrated circuit **doubles approximately every 18 months**. DRAM capacity followed the same curve through DRAM, SDRAM, DDR, DDR2, DDR3, DDR4 and DDR5.

Today's mainstream IC processes use **2 to 130 nanometres** (0.002 to 0.13 micron). The lecture's reference points:

- **AMD Ryzen 9**: 7 nm CMOS FinFET.
- **Atmel AVR microcontrollers (Arduino)**: 180 nm.
- **Motorola 68000** (Lesson 3): 3.6 µm = 3,600 nm.

Finer processes mean higher speeds, lower power and more functionality on a single chip. Multiple cores are the new trend (clock frequencies above 20 GHz are doubtful). Device physicists once predicted current technology would break down at 9 nm geometries.

Worked example

A chip has 10 million transistors. How many 18-month doublings take place in 6 years, and what is the count?

1. 6 years = 72 months = 72 / 18 = **4 doublings**.
2. Count = 10 million x 2^4 = **160 million** transistors.

> **Key fact:** Transistors per IC double roughly every 18 months (Gordon Moore). Mainstream processes now run from 2 to 130 nm; a Ryzen 9 uses 7 nm CMOS FinFET, Arduino's AVR uses 180 nm, and the 68000 used 3.6 μm. Finer processes give higher speed, lower power and more function.

Why shrinking transistors got hard (leakage, Dennard scaling, 3-D devices)

Learn

The transistor was invented at Bell Labs in 1947 by Bardeen and Brattain (Shockley followed with the more manufacturable junction type), and it has shrunk ever since. Moore's law (the doubling of transistor density) is quoted as every two years or about 18 months.

Why it got hard. The smaller a CMOS transistor, the more current it **leaks when switched off**: the gate's control over the channel weakens as source and drain come close, and current sneaks through the channel far from the gate and through the substrate.

Dennard scaling (IBM, 1974) said shrinking gate length, insulator thickness and other dimensions together would raise speed, cut power and raise density at once. In practice voltage could not be lowered as hoped, because leakage forced a high drain voltage for fast switching. By **2001 leakage power was nearly the power needed to switch**, which would make a chip use the same energy idle or busy.

Fixes (the industry's "ways to buy Moore's law more years"): strained silicon (2003), a hafnium oxide gate insulator (2007), and then a new structure that gives the gate better control: a **very thin silicon channel on insulator** (UTB SOI) or a channel **turned on its side into a 3-D device** (Intel's 2011 announcement).

Worked example

A chip's leakage power is 10% of its switching power at one node. If leakage doubles with every shrink while switching power stays constant, after how many shrinks is leakage equal to switching power? (Illustrative numbers, not from the reading.)

1. 10% x 2 = 20% after one shrink, 40% after two, 80% after three, **160% after four**.
2. After three shrinks it is still 80% of switching power; it passes switching power at the **fourth** shrink.
3. This is the pattern the reading describes by 2001: leakage close to switching power, motivating new structures.

> **Key fact:** Smaller CMOS transistors leak more when off because the gate loses control of the channel. Dennard scaling (shrink everything for better speed, power, density) stopped working; by 2001 leakage was nearly the switching power. Fixes: thin channel on insulator (UTB SOI) or a 3-D channel (Intel 2011). Moore's law quoted as every 18 months or two years.

The design process

From research to bring-up: design phases, software planning, design reviews, test plans and choosing parts.

The four phases of a design

Learn

Whether the project is a function generator, a Capstone project or anything else, the process is very similar:

1. **Research phase.** Assess the problem and decide what to create; write down a loose set of specifications (formalised later); list what you know and what you need to research (including how to design a PCB); read articles and application notes; gather manufacturers' data; discuss it with knowledgeable people.
2. **Design phase.** Create a **project schedule**; use a **spreadsheet to bound the problem space**; sketch the design at **block diagram** level; do calculations and simulations to home in on the final design.
3. **Implementation phase.** Before sending off for parts, check: Do I fully understand how my design works? Do I have complete documentation and a full set of schematics? Have I done a **design review** and addressed all the issues, with separate reviews of the schematic, the PCB and the software architecture? Are the software requirements understood, and can I write the code?
4. **Assembly and test phase.** The process that starts with loading parts onto the PCB and ends with a product that meets all specifications. **Without a plan you will waste time and destroy PCBs and parts.**

Worked example

You have a finished schematic and PCB layout for a signal generator. What must happen before you order the boards?

1. Confirm you understand how every block works and that the documentation is complete.
2. Hold separate design reviews of the schematic, of the PCB design and of the software architecture, and fix what they find.
3. Make sure the software requirements are understood and you can write the code. Only then order parts.

> **Key fact:** 1 Research: assess the problem, loose specs, learn what you need, read notes, gather data, talk to experts. 2 Design: schedule, spreadsheet to bound the problem, block diagram, calculations and simulation. 3 Implementation: checklist (understand it, full schematics, design reviews, software requirements) before ordering. 4 Assembly and test: a plan from loading parts to a product that meets specs.

Software is not an afterthought

Learn

Designing an embedded system needs **software and hardware that work together**. Do not treat the software as an afterthought:

- Software is a **notorious time-suck**; without planning, the hardware will not work because of buggy software.

- Even code you found online must be **understood**.
- "I can't start the software until the hardware is done" is false. If the software is modular you can write **test software that looks like hardware** to your program. Example: reading an A/D converter: write a function that returns a representative value, and let the rest of the code process it; swap in the real read later.
- Arduinos use a dialect of **C++**, not Java.

Worked example

Your temperature controller's firmware must read an ADC that does not exist yet. How do you start?

1. Define `read_adc()` returning a 10-bit count.
2. For now make it return a fixed, or scripted, value such as 512.
3. Write and test the filtering and control code against it; replace the stub with the real driver when the board arrives.

> **Key fact:** Plan it from the start (it is a notorious time-sink). Write it modularly with stub functions returning representative hardware values so you can test before the hardware exists. Understand any driver code you borrow. Arduino code is a C++ dialect.

Formal design reviews

Learn

The **hardware design review** is arguably more important than a software design review. A formal review finds errors when they are relatively easy to fix (fixing a hardware defect in a released product is a nightmare), and studies show a **return on investment of more than 20 times**.

The process in the lecture:

1. About a week before, choose **one to three reviewers and a moderator/recording secretary**.
2. Make sure engineers have time in their schedules for lab-wide reviews; everyone takes turns.
3. **Circulate the documents** (schematics, data sheets, ABEL or Verilog code, timing calculations, simulations) at least **3 or 4 days ahead**.
4. In the review the moderator leads; reviewers raise issues. **The purpose is to identify issues, not to solve them.** The designer explains but **does not defend** the design. The moderator notes each issue and whether it needs follow-up.
5. At the end the moderator restates each issue needing attention.
6. The moderator writes the summary and sends it to the lab manager and participants.
7. The designer addresses each issue and writes up how.
8. A **follow-up review** may be needed if there were many issues or substantial changes.
9. After sign-off, the design can move to **PC board layout**.

The lecture's HP story makes the point from another side: a retrained hardware engineer who never had a software bug logged against him explained, "I didn't know that I was allowed to have bugs."

Worked example

A reviewer says 'your regulator's footprint looks wrong'. How should the designer respond?

1. Note it; ask clarifying questions if needed; do not argue the design is right.
2. The moderator records it as an issue to follow up.

3. After the meeting the designer checks the data sheet and reports how it was resolved.

> **Key fact:** Pick 1-3 reviewers plus a moderator a week ahead; circulate documents 3-4 days before; in the meeting the moderator lists issues (the goal is to identify, not solve; the designer explains but does not defend); moderator writes the summary; designer fixes and reports; follow-up review if needed; sign-off before PCB layout. ROI above 20x.

Designing in reliability (worst-case timing, reviews, thermal checks, ESD)

Learn

A 1984 Hewlett-Packard example (one page, scanned) shows reliability being **designed in**, because, in its words, reliability, like quality, "must be designed in. It cannot be added on later." The practices:

1. **Design margin by worst-case analysis** of each critical timing path, accounting for power supply variation, output loading, temperature variation and the stripline characteristics of the circuit board.
2. **Peer design reviews** of each module before the board is made; each engineer learns a part well enough to explain it to the group.
3. **Layout check:** the board layout is read back by software into a list of all wires and connections and compared with the schematic before fabrication.
4. **Thermal check:** thermocouples find hot spots, and the data gives each IC's junction temperature so no device is overstressed.
5. **Environmental tests** over temperature, line voltage and frequency, humidity, altitude and vibration.
6. **ESD control** in manufacturing, since weakened parts can pass factory tests and fail soon at the customer.
7. **MTBF** predicted (RADC II methods) and compared with field data; here the field results were 2 to 2.5 times better than predicted.

Worked example

A board has a predicted MTBF of 7,400 hours and field data 2 to 2.5 times better. What field range does that imply?

1. 2 x 7,400 = **14,800 hours**.
2. 2.5 x 7,400 = **18,500 hours**.

> **Key fact:** Reliability must be designed in, not added later. Worst-case analysis of each critical timing path (supply, loading, temperature, board stripline); peer design reviews; layout read back as a connection list and checked against the schematic; thermocouples for hot spots and junction temperature; environmental tests; ESD control; MTBF predictions compared with field data.

A test plan for a new board

Learn

For a student or new engineer, **turn-on and debug** is the most difficult phase. Best practice: **make a test plan and keep a lab notebook**. The lecture's outline:

1. **Raw board:** inspect it against the design; with an ohmmeter measure the resistance between **Vcc and ground** (expected **infinite**).

2. **Loaded board:** parts properly located and aligned; all IC pins in the sockets and ICs correctly oriented; all leads soldered; **no solder bridges** between pins; no residual flux; components seated properly.
3. **Board turn-on (no external inputs):** measure Vcc-to-ground resistance (is it a short?); **apply power with a current limit or a series resistor**; any odors? any heat?; measure the supply voltages (is Vcc correct?).
4. And so forth, step by step.

It may look like a waste of time, but it is worth its weight in time saved, and it impresses your peers.

Worked example

After loading the board, the ohmmeter reads 3 Ω between Vcc and ground. What do you do?

1. That is far from infinity: there is probably a **short** (a solder bridge, a reversed capacitor or a mis-oriented IC).
2. Do **not** apply power. Inspect the board for bridges and orientation, and find the short.
3. Re-measure; only continue to the current-limited power-up when the reading is acceptable.

> **Key fact:** 1 Raw board: inspect against the design; ohmmeter Vcc to ground should be infinite. 2 Loaded board: parts aligned, ICs in sockets and oriented, leads soldered, no bridges, no flux. 3 Turn-on without inputs: recheck Vcc-ground, apply power with a current limit or series resistor, look for odors and heat, measure supply voltages. Keep a lab notebook.

Choosing a microcontroller: the A/D resolution example

Learn

When designing, say, a function generator, the microprocessor handles the user interface, waveform generation and control of digital resistors and switched capacitors. You might pick an Arduino, Raspberry Pi, Teensy 4.1 or ESP32: **any microcontroller board can work, but you must understand the strengths and limits of each.**

The lecture's example is the Arduino's A/D converter:

- **10-bit** over **0 to 5 V**: resolution is **1 part in 1023**, about **5 mV per step** (5 V / 1023 = 4.89 mV).
- Accuracy is worse than resolution: the slide says **+/- 2 LSB, or +/- 15 mV**. (2 LSB is really about 10 mV; 15 mV would be about 3 LSB. The slide's two numbers disagree.)
- Better accuracy: an **external A/D converter**. Too-small signals: an **op-amp** for gain. Some boards have 12-bit A/Ds but less software support than the Arduino. The Arduino uses a 16 MHz clock.

This is the pattern for choosing any part: compute what the specification needs (resolution, range, speed), and compare it with what the board offers.

Worked example

How small a voltage change can a 12-bit converter on 0-3.3 V resolve?

1. Steps = 2^12 - 1 = 4,095.
2. Step = 3.3 V / 4,095 = **0.806 mV**.
3. Compared with the Arduino's 4.89 mV, it resolves a change about six times smaller.

Key fact: Resolution: 1 part in 1023, about 5 mV (4.89 mV) per step. Accuracy: a couple of LSB (the slide says +/-2 LSB or +/-15 mV, which are not quite consistent: 2 LSB is about 10 mV). Better accuracy: external A/D; small signals: an op-amp for gain.

Designing a PCB

From schematic to board: workflow, why layouts fail, trace resistance, planes, bypass capacitors, grounds, clock skew and testability.

The PCB design workflow

Learn

The lecture gives a "cookbook" for going from a schematic to an error-free board. The key rules:

1. **Understand the circuit** before you lay it out.
2. **Do not skip schematic capture** and go straight to the PCB. The schematic must name **every part**: R (resistors), C (capacitors), Q (transistors), U (integrated circuits), J (connectors), plus the power and ground buses.
3. **Learn the CAD tool on a practice design**, not on your real board.
4. **If the schematic is wrong, the PCB will be wrong.** Get data sheets for every complex part, and the footprint dimensions for the layout.
5. **Place parts first and do not start wiring yet.** Move and rotate parts until the traces look easy. A consistent strategy helps, for example north-south traces on one layer and east-west on the other.
6. **Plan trace width early**, especially for power and ground.
7. **Watch hole and via sizes.** A via connects traces on different layers; vias and through-holes are not the same thing.
8. **Allow time to check**, assume there will be errors, and have others review the design.

Worked example

You have the schematic for a board with a microcontroller, a regulator, two connectors and a handful of resistors. List the order of work.

1. Confirm the circuit works (simulation or a known design) and collect data sheets for the microcontroller and the regulator.
2. Capture the schematic with every part labelled: U1 (microcontroller), U2 (regulator), J1 and J2 (connectors), R1 to R6.
3. Pick footprints from the CAD library, checking them against the data sheets.
4. Place parts: connectors at the board edge where the chassis needs them, the microcontroller central.
5. Plan trace widths (wide power and ground), then route.
6. Run the design rule check and get an independent review before ordering.

> **Key fact:** Understand the circuit, capture the schematic, label every part (R resistors, C capacitors, Q transistors, U ICs, J connectors), place parts, then route traces, then review.

Why PCBs fail: mechanical errors and design reviews

Learn

The lecture's rule of thumb: **75% to 90% of the errors on a PCB layout are mechanical issues**: hole alignment, hole size, connectors, interference between parts, footprints that do not match the real part. They are hard for the designer to see, so you need a **design review**.

The author's own case study had four errors in the first boards:

#	Error	Type
1	A typo in the data sheet's footprint dimensions (2 mm, but it mattered)	Mechanical
2	Footprint 1.000 in wide instead of 0.950 in	Mechanical
3	A module too close to the USB socket	Mechanical
4	Assumed the regulator's pinout was standard; it was not	Electrical

Three of four were mechanical. The root-cause fixes are simple: use the **latest data sheet** and double-check it against the real part, check **clearances**, and **never assume a pinout**.

Boards can sometimes be salvaged with kludges (bent pins, cut traces, jumper wires), but the lecture's moral is that it is better not to need to. **Even experienced designers plan for revisions**, and every design should have multiple independent reviews, first of the circuit design and then of the PCB design.

Worked example

A 4-board project found 8 errors in total. Using the lecture's rule of thumb, how many would you expect to be mechanical?

1. The rule of thumb is 75% to 90%.
2. 75% of 8 = 6, and 90% of 8 = 7.2.
3. So expect about **6 or 7** mechanical errors and only 1 or 2 electrical ones.

> **Key fact:** About 75% to 90% (hole alignment and size, connectors, clearances, footprints). The remedy is multiple independent design reviews and checking dimensions against the data sheet.

Trace resistance, voltage drop and trace width

Learn

A trace is a thin copper wire, so it has resistance:

> **R = ρ x L / A**, with ρ the resistivity of copper (1.68 x 10^-6 ohm-cm at 20 C), L the trace length and A its cross-sectional area (**width x copper thickness**).

The geometry that sets a trace's electrical behaviour is its **copper thickness (T)**, **width (W)**, **height above the ground plane (H)** and the **dielectric constant**.

Once you have R:

- **Voltage drop = I x R** (the power rail sags along the trace).
- **Power lost as heat = I^2 x R**, which is also what heats the trace.

Keep the units consistent. Units trap: 1 mil = 0.001 in, and 1 in = 2.54 cm, so 1 mil = 0.00254 cm. If ρ is in ohm-cm, convert width, thickness and length to **cm**.

This is why power and ground traces must be wide: a few amps through a thin trace gives a large drop and a lot of heat.

Worked example

The lecture's question: copper 1 mil thick, trace 125 mil wide and 3 in long, ρ = 1.68 x 10^-6 ohm-cm, current 10 A. Find the voltage drop and power loss.

1. Area: 1 mil x 125 mil = 0.00254 cm x 0.3175 cm = 8.06 x 10^-4 cm^2.
2. Length: 3 in x 2.54 = 7.62 cm.

3. R = 1.68 x 10^-6 x 7.62 / 8.06 x 10^-4 = **0.0159 ohm**.
4. Drop = 10 A x 0.0159 = **0.159 V**. Power = 10^2 x 0.0159 = **1.59 W**.

> **Key fact:** R = ρL/A (A = width x thickness). Drop = I x R. Power loss = I^2 x R. Convert everything to the same units first (1 mil = 0.001 in = 0.00254 cm).

Power planes and bypass capacitors

Learn

Many microprocessor designs use at least **4-layer boards**: the top and bottom layers carry signals and the **inner two layers are power and ground planes**.

- Planes are **low-impedance sources and sinks**, which minimises overshoot and undershoot on logic signals.
- They add capacitance (a low-pass effect) and lower inductance, which lowers voltage spikes.

Planes are not enough: logic switching also needs **filter and bypass capacitors**.

- A fast logic edge needs a **high instantaneous current** to charge the load capacitance.
- **Bypass capacitors store charge** locally to supply that current, so the supply does not sag or spike.
- The slide's rule of thumb: **1 capacitor per 1-2 ICs**. The picture shows 0.1 µF ceramic bypass capacitors (the slide's label reads "mf") and a 10 µF tantalum filter capacitor.

Worked example

A board has 9 logic ICs. How many bypass capacitors does the rule of thumb suggest?

1. One capacitor per 1 to 2 ICs means between 9 / 2 = 4.5 and 9 / 1 = 9.
2. So roughly **5 to 9** bypass capacitors, each placed close to the power pins it serves.

> **Key fact:** Planes give low-impedance power and return paths, reduce overshoot and undershoot, add capacitance and lower inductance. Bypass capacitors (about 1 per 1-2 ICs) store charge for the current spikes of fast logic edges.

Ground management and split ground planes

Learn

A ground plane is good, but it is not the only consideration when a board mixes **analog and digital** circuits.

- **Digital circuitry is noisy**: its switching causes current spikes.
- **Analog circuitry needs a quiet ground** to measure low-amplitude signals. A/D converters often have separate analog and digital ground pins.
- So use **separate planes** (more layers) or **isolated planes on the same layer**. Isolation keeps digital return currents from making noise on analog signals.
- Let the planes **meet at one point only**. If they are connected in several places, tiny voltage differences between those points let current flow around the loop: a **ground loop**, which causes noise, 60-cycle hum or oscillation.

Worked example

A board has a 12-bit A/D converter with AGND and DGND pins, a microcontroller, and a noisy motor driver. How do you handle grounds?

1. Put the A/D converter and its input conditioning on an **analog ground plane**.
2. Put the microcontroller and the motor driver on the **digital ground plane**.
3. Connect AGND to DGND at **one point**, close to the converter, so the two returns meet once.
4. Keep digital traces from crossing the analog plane so digital return currents do not run through it.

> **Key fact:** Digital switching noise would corrupt quiet analog signals, so the planes are isolated; they are joined at one point only, because multiple connections create ground loops (noise, 60 Hz hum, oscillation).

Clock distribution and clock skew

Learn

In most designs the **clock touches almost every component**, so it is the most heavily loaded signal and drives the most capacitance. It must be well controlled, noise-free and laid out carefully.

If the clock is **buffered** to drive all those loads, each buffer adds a **propagation delay**, and paths of different length add delay too. The result is **clock skew**: the same edge reaches different components at different times. Skew **reduces the timing margins**, because the time available between edges is shared with the arrival difference.

Good practice from the lecture: keep the distribution balanced (same buffers, similar path lengths for each branch), and **characterize the clock first** once the board passes the smoke test.

Worked example

A 20 ns clock period feeds two chips. The clock reaches chip A after 2 ns and chip B after 5 ns. What is the skew, and how much of the period is it?

1. Skew = 5 ns - 2 ns = **3 ns**.
2. As a share of the period: 3 / 20 = **15%**.
3. That 3 ns is time the signals launched by A and captured by B cannot use, so the timing margin falls by 3 ns.

> **Key fact:** Time differences in the arrival of the clock edges at different parts, due to different path lengths and propagation delays (for example through buffers). It reduces timing margins.

Design for testability and mechanical limits

Learn

Design for testability. Very few hardware designers treat the first working hardware as the final board, so plan to debug it:

- **Socket** components if you can.
- Bring **critical signals** (such as the clock) to accessible **test points**.
- Provide test points for **power and ground**, especially ground, and room for probes.

Mechanical and thermal limits. The board has to fit something:

- Plan **mounting holes**, the **type and location of connectors** (someone has to assemble and repair it) and **cooling** (fan and air-flow often fix where parts can go).
- Some parts mount on the board and attach to the chassis too: audio jacks, power connectors, serial and USB.

Dos and don'ts. Design in **functional blocks** that sit together; keep the layout flow like the schematic; keep part labels visible after soldering; use the **largest traces** you can; remember that **bigger and more layers cost more**. Through-hole parts are easier to work with, but are becoming harder to find.

Worked example

You have a 2-layer board with a clock, a 5 V rail and a debug header. Which test points do you add?

1. A **clock** test point near the oscillator output.
2. A **ground** test point (a hole or pad tied to the ground plane or trace). The lecture's own 68008 board lacked one and needed a wire tacked to a ground pin.
3. A **5 V** test point.
4. Leave clearance so a probe or scope clip fits.

> **Key fact:** Sockets, test points (especially ground), clock access, room for probes, wide traces and pads, a modular layout following the schematic, labels that stay visible, and connector/cooling/mounting placed for the chassis.

Clocks and pulses

Period and frequency, duty cycle, pulse measurements, ringing and propagation delay.

Period, frequency and engineering prefixes

Learn

Frequency is the **inverse of time**: **f = 1 / T**. The lecture's engineering prefixes:

Prefix	Factor	Example
tera (T)	10^12	
giga (G)	10^9	1 ns period = **1 GHz**
mega (M)	10^6	1 μs period = **1 MHz**
kilo (k)	10^3	1 ms period = **1 kHz**
milli (m)	10^-3	
micro (μ)	10^-6	
nano (n)	10^-9	
pico (p)	10^-12	
femto (f)	10^-15	

(The slide's printed list shows "Micro = 10^6" next to "Giga"; that is a typo for **Mega**. Micro is 10^-6.)

A quick way to convert: **1000 / (period in ns) = frequency in MHz**. A 2 ns period is 500 MHz, **not** 50 GHz.

Distance matters too: the lecture's figure of merit is that a signal travels **about 1 ns per foot in free space** and **about 2 ns per foot on a printed circuit board**.

Worked example

A clock has a period of 8 ns. What is its frequency?

1. 1000 / 8 = **125 MHz**.
2. Check: 125 MHz = 125 x 10^6 Hz, and 1 / (125 x 10^6) = 8 x 10^-9 s = 8 ns. ✓

> **Key fact:** f = 1/T. A 1 ns period is 1 GHz, 1 μs is 1 MHz, 1 ms is 1 kHz, 1 s is 1 Hz.

Duty cycle

Learn

A clock is a continuous stream of pulses, but the pulses need not be symmetrical. Let **tPH** be the time the pulse is high and **tPL** the time it is low in one cycle.

> **Duty cycle = tPH / (tPH + tPL) x 100%**

- High time equals low time: **50%**.
- High for 1/4 of the cycle: **25%**.
- Note that tPH + tPL is the whole **period**, so duty cycle = tPH / T.

Duty cycle says nothing about frequency: a 1 kHz and a 1 MHz clock can both be 50%.

Worked example

A clock is high for 30 ns and low for 90 ns. Find its duty cycle and frequency.

1. Period = 30 + 90 = 120 ns.
2. Duty cycle = 30 / 120 = **25%**.
3. Frequency = 1000 / 120 = **8.33 MHz**.

> **Key fact:** Duty cycle = t_high / (t_high + t_low) x 100%. A symmetrical clock (high time equals low time) has a 50% duty cycle.

Pulse width, rise time and fall time

Learn

Dissecting a **positive pulse**:

- **Pulse width**: the width of the pulse measured at **50% of the maximum voltage**.
- **Rise time**: the time to go from **10% to 90%** of the pulse height.
- **Fall time**: the time to go from **90% to 10%**.

An oscilloscope can calculate these automatically.

Real pulses are not ideal, and that is fine as long as they stay on the right side of the logic thresholds. In the lecture's real-clock slide, a signal above **2.5 V** is a logic 1 and one below **1 V** is a logic 0; in between is **not defined**, so the signal must pass through that region quickly.

Worked example

A pulse swings from 0 V to 3 V. At what voltages do you measure rise time and pulse width?

1. Rise time: from 10% of 3 V = **0.3 V** to 90% = **2.7 V**.
2. Pulse width: at 50% = **1.5 V** (the time between the rising and falling crossings of 1.5 V).
3. Fall time: from 2.7 V down to 0.3 V.

> **Key fact:** Pulse width: width at 50% of maximum voltage. Rise time: from 10% to 90% of pulse height. Fall time: from 90% to 10%.

Overshoot, undershoot and ringing

Learn

Real clocks and pulses deviate from the ideal shape:

- **Overshoot**: voltage or current goes **above the power rail**.
- **Undershoot**: goes **below the ground reference**.
- **Ringing**: **damped oscillations** at each transition.

Causes listed in the lecture: **poor grounding** (including the scope probe's own ground), **poor or insufficient bypassing**, the overall grounding scheme, **transmission-line effects** from impedance mismatch (gate output, PCB conductor, gate input), **inductive loads** such as relays or solenoids, and **fast gate rise and fall times**.

As speeds go up, PCB design becomes more critical. This links straight back to power planes, bypass capacitors and ground management.

Worked example

You see a 5 V logic signal that spikes to 6.2 V and then rings for a few cycles after every rising edge. Name the effects and one fix.

1. The spike above the rail is **overshoot**; the decaying oscillation is **ringing**.
2. Likely causes: impedance mismatch on the trace, a weak ground return or too little bypassing.
3. Fixes to try: improve the ground path or the scope-probe ground, add bypass capacitors close to the IC, or slow the edge.

> **Key fact:** Overshoot: above the power rail. Undershoot: below ground. Ringing: damped oscillation at each transition. Causes: poor grounding or bypassing, impedance mismatch (transmission-line effects), inductive loads, fast edges.

Propagation delay and why clocks cannot go faster

Learn

Propagation delay is the time from an input change to the output change. Gates need a finite time to switch state, so a clock cannot be made arbitrarily fast: everything the computer does in one cycle must fit inside the period.

Numbers from the lecture:

- Today's maximum PC clocks are around **4 GHz**; internal gate delays are about **100 ps**, and everything in one cycle must happen in about **250 ps**.
- A signal on an IC travels roughly **1.4 inches in 270 ps**. On a chip the speed is about **6 inches per ns**, while on a PCB it is about **2 ns per foot**.
- With a period of about **0.3 ns** and a clock crossing time of about **0.16 ns**, **more than half** the available time is wasted just getting the clock signal there.

The inverter example on the slide measures a delay of 12.60 ns on a slower gate.

Worked example

A signal path has three gates in series, each with 8 ns propagation delay, and the wire adds 2 ns. What is the minimum clock period, and the maximum frequency?

1. Total delay = 3 x 8 + 2 = **26 ns**.
2. The period must be at least 26 ns.
3. Maximum frequency = 1000 / 26 = **38.5 MHz**.

> **Key fact:** Gates need a finite propagation delay to switch, and signals need time to cross the chip and board (about 2 ns per foot on a PCB), so everything must fit inside one clock period.

Microprocessors and microcontrollers

What a microprocessor is, its buses, CISC versus RISC, the blocks inside, and where the 68000 and Z80 came from.

Microprocessor versus microcontroller

Learn

A **microprocessor** is an integrated circuit that forms the central processing unit of a computer or embedded controller, but **requires additional support circuitry** to function. Examples in the lecture: the MC68000, Z80, RISC-V, and the ARM cores inside the FPGA on a DE-1 SoC board.

A **microcontroller** is a microprocessor **plus peripheral support devices integrated into one package**. The peripherals can include serial and parallel ports, Ethernet, A/D and D/A converters, interval and watchdog timers, event counters, a real-time clock, and other local processors. Examples: Arduino, Raspberry Pi, TIVA.

The difference is where the blocks live. In a microprocessor system, program memory, data storage, serial and parallel I/O, timers and converters are **on the printed circuit board**. In a microcontroller they are **integrated with the processor core on the silicon chip**.

Worked example

You need a processor with a UART, a timer and 256 KB of flash in one package. Microprocessor or microcontroller?

1. A UART (serial port), timer and on-chip flash are exactly the peripherals the lecture lists for a microcontroller.
2. A bare microprocessor would need each of them as a separate chip or circuit on the PCB.
3. So choose a **microcontroller**, such as the TIVA TM4C123 that the lecture describes (256 KB of flash, 8 UARTs and a number of timers).

> **Key fact:** Microprocessor: the CPU only; memory, I/O and timers are separate chips on the PCB. Microcontroller: a microprocessor core plus peripherals (serial/parallel ports, timers, A/D and D/A, and so on) integrated in one package.

Address, data and control buses

Learn

Every microprocessor has three groups of signals, plus clock, power and ground:

Bus	Carries	Direction
Address bus (A0 to An, with 10 <= n <= 35)	Which memory or I/O location is wanted; addressable memory = 2^n bytes	**Unidirectional**: all signals go from the processor to memory
Data bus (D0 to Dn, with 4 <= n <= 255)	The information written to or read from memory or I/O	**Bidirectional**: data may travel to or from the processor
Status and control bus	Control and "housekeeping" signals such as RESET, INTERRUPT, READ, WRITE, ADDRESS VALID	**Heterogeneous**: signals have different functions, and may be input, output or both

So the address bus is the only strictly one-way bus; the data bus is **completely bidirectional**; the control bus is a **mix**.

Worked example

A processor has a 24-bit address bus and an 8-bit data bus. How many lines are in each bus, and how many bytes can it address?

1. Address bus: 24 lines, A0 to A23.
2. Data bus: 8 lines, D0 to D7.
3. Addressable memory = 2^24 = **16,777,216 bytes (16 MB)**. The data width does not change that.

> **Key fact:** Address bus: unidirectional (processor to memory). Data bus: bidirectional. Status and control bus: heterogeneous, with different signals (RESET, INTERRUPT, READ, WRITE, ADDRESS VALID) that may be input, output or both.

CISC, RISC and DSP architectures

Learn

Three families of architecture:

- **CISC** (Complex Instruction Set Computer): **many instructions and addressing modes**, each able to do involved operations; code is compact; instructions are **variable length** and may take many clock cycles; **von Neumann** (one memory for instructions and data). The 68000 and Z80 are CISC.
- **RISC** (Reduced Instruction Set Computer): came from compiler research at IBM, which found that compilers use a small set of instructions. **Fewer instructions of uniform length**, about **one instruction per clock**, **many general-purpose registers**, **all arithmetic between registers**, memory only by LOAD and STORE, and often a **Harvard** architecture (separate instruction and data memories). ARM is the most popular.
- **DSP** (Digital Signal Processor): special instructions and an architecture designed for signal processing.

	CISC	RISC
Instructions	Many, variable length	Fewer, single length
Addressing modes	Many	Few
Registers	Fewer, specific functions	Many, general purpose
Memory	Memory to memory, absolute addresses	Load/store only

Worked example

Add the long word at mem1 to the long word at mem2, storing the sum in mem2. Compare the 68000 and ARM.

- **68000:** `LEA mem1,A0`, `LEA mem2,A1`, `MOVE.L (A0),D0`, `ADD.L D0,(A1)`: the ADD works directly on memory, and the result lands in memory.
- **ARM:** load both addresses, `LDR` each value into a register, `ADD r3,r3,r2`, then `STR` the result: arithmetic only on registers.
- The CISC version is shorter because one instruction can mix a register and a memory operand.

Key fact: CISC: many instructions and addressing modes, variable length, few special-purpose registers, memory-to-memory operations, von Neumann. RISC: few instructions, fixed length, many general registers, register-only arithmetic with LOAD/STORE, no absolute addressing, can be Harvard. DSP: special instructions for signal processing.

Functional blocks inside a processor

Learn

The blocks inside a processor and what each does:

- **Microcode state machine**: decodes instructions, sequences memory reads and writes, provides decision capability and coordinates all internal and external behavior. It is **why microprocessors are synchronous (clock driven)**.
- **ALU (arithmetic and logic unit)**: all mathematical and logical operations. Generally not synchronous (simple arrays of gates). Operands are in registers; results (zero, positive, negative, overflow) go to the status register.
- **General-purpose registers**: local storage for operands, built from D-type flip-flops close to the CPU. Efficient operation is between registers. They may be fully general-purpose or special-purpose, and can often be ganged to hold larger data types.
- **Status register**: bit **flags** that are set (1) or reset (0) by ALU results. For A - B with A = B, the **Zero flag = 1**.
- **Program counter (PC)**: the address of the next instruction to fetch; branches and jumps change it.
- **Stack pointer (SP)**: the current top of the **LIFO** stack, used for subroutines and interrupt service routines.
- **Temporary register(s)**: hold operands or intermediate results; may be hidden.
- **Instruction decode (instruction) register**: holds the fetched instruction for the state machine.
- **Interrupt controller, bus interface unit, debug core, cache control unit, instruction and data caches**: interrupts, the memory interface, debug hardware and on-chip high-speed memory.

Worked example

Trace one instruction through the blocks: ADD D0,D1 on a 68000.

1. The **PC** gives the address; the instruction word is fetched through the **bus interface unit** into the **instruction register**.
2. The **state machine** decodes it as "add, register to register".
3. The operands come from the **registers**; the **ALU** adds them and the result goes back to a register.
4. The **status register** flags (zero, negative, carry, overflow) are updated, and the **PC** advances.

Key fact: State machine (decode and sequence, clock driven); ALU (math and logic, results set flags); registers (fast local storage); status register (flags); PC (next instruction address); SP (top of stack, LIFO); instruction register (holds the fetched instruction); bus interface unit; interrupt controller; debug core; caches.

Parts of a microprocessor system

Learn

A microprocessor system has these parts:

- **Microprocessor core**: all comparisons, arithmetic and logical operations; registers hold local data.

- **Memory interface**: reads instructions and data and writes new data.
- **Control interface**: clocks, RESET, the interrupt controller and other housekeeping.
- **Memory**: holds instructions, data and variables. It can be **RAM**, **ROM** (read-only), **flash** (read-mostly) or **SDRAM** (synchronous burst-access dynamic RAM).
- **I/O**: specific-function devices such as ASICs, timers, Bluetooth, LAN, I/O ports and A/D and D/A converters.
- **Glue logic**: random logic to interface the blocks: quad NAND or hex NOT gates, or PLDs, CPLDs and FPGAs.

Worked example

Classify each part of a small board: a 68000, an EPROM, a 74138 decoder, a UART and a reset button circuit.

1. 68000: **microprocessor core** (with its memory and control interfaces).
2. EPROM: **memory**.
3. 74138 decoder: **glue logic** (it selects chips from address bits).
4. UART: **I/O**.
5. Reset circuit: part of the **control interface** (RESET).

> **Key fact:** Microprocessor core (arithmetic, logic, registers), memory interface, control interface (clocks, RESET, interrupts), memory (RAM, ROM, flash, SDRAM), I/O (ASICs, timers, ports, converters) and glue logic.

Where the 68000 and Z80 came from

Learn

Z80

- Designed by Zilog, started by Federico Faggin and 11 other Intel employees. A faster, cheaper, **code-compatible** improvement on the Intel 8080 with more instructions.
- Designed for the embedded market but powered the home-computer revolution (for example the TRS-80).
- **4 µm** process, **8,500 transistors**, originally **2.5 MHz** and later up to 10 MHz; licensed to many other companies. Introduced in the summer of 1976 and still in use.
- **8-bit CISC.** Simple bus operation that is easy to watch with a logic analyzer; its assembly language is trickier than the 68K's.

68000

- Development began in 1976 as a 32-bit architecture to replace the 8-bit 6800. Formally introduced in **1979** on a **3.6 µm** process, with a **16 MHz** clock.
- IBM chose the 8088 for the PC because the 68K (and the Z8000) was not ready; the 68K took over UNIX workstations and the Apple Macintosh, and evolved into the 68010 to 68060, ColdFire and the 683XX.
- **8-16-32 bit CISC**; assembly is straightforward, bus operation is more complicated than the Z80's.

The course uses both because they contrast well: the 68000 for a clean, easy instruction set; the Z80 for its simple, visible bus.

Worked example

Compare the process geometry of the 68000 to a modern 7 nm process.

1. 68000: 3.6 μm = 3,600 nm.
2. 3,600 / 7 = about **514**, so modern geometry is roughly 500 times smaller (the slide says about 500).

> **Key fact:** Z80: Zilog (founded by Federico Faggin and other Intel staff), introduced in summer 1976, 8-bit, code-compatible with the 8080. 68000: Motorola, introduced in 1979 on a 3.6 μm process, 8-16-32 bit CISC, 16 MHz.

The programmer's model

Registers, flags and number notation on each processor, and two's complement arithmetic.

The programmer's model and why assembly is not portable

Learn

Before writing assembly we need the processor's **programmer's model**: its **instruction set** and its **addressing modes**.

- The **instruction set** says what the processor can do.
- The **addressing modes** say how the processor finds the data for the instruction, and what to do with the data afterwards.

Unlike a Java, C or C++ program, **assembly language is not portable**: a program for an Intel 8086 will not run on a Motorola 68000. A C++ program written for the 8086 might run on the 68000 once the original source is **recompiled**. Coding principles are common across architectures, but every architecture has its own assembly language.

The course looks at two: the **68K** (8-16-32 bit CISC, easy to learn, bus operation more complicated) and the **Z80** (8-bit CISC, assembly trickier, bus operation very simple).

Worked example

Why can't you copy a 68000 `MOVE.B D1,D0` *into a Z80 program?*

1. `MOVE.B D1,D0` uses 68000 data registers and a 68000 opcode.
2. The Z80 has no D0 or D1 as 32-bit data registers and no MOVE instruction; its equivalent is a load such as `LD A,B`.
3. The two processors have different instruction sets, registers and machine code, so the translation is by hand, not automatic.

> **Key fact:** The programmer's model is the instruction set (what the processor can do) plus the addressing modes (how it finds and stores the data). Assembly is not portable between processors; C or C++ source can be recompiled.

Two's complement numbers

Learn

Processors turn subtraction into addition by storing negative numbers in **two's complement**. To negate a number (byte, word or long word):

1. **Complement** every bit (1s become 0s and 0s become 1s).
2. **Add 1**.

Example: $5E = 0101 1110. Complement: 1010 0001. Add 1: 1010 0010 = **$A2**. So $A2 means -$5E (decimal -94).

Facts to keep:

- The **top bit is the sign**: 1 means negative.

- 8-bit range **-128 to +127**; 16-bit **-32,768 to +32,767**; 32-bit **-2,147,483,648 to +2,147,483,647**. Zero counts as positive.
- The processor always treats an addition as signed; it is the programmer's job to decide the context.
- Branching backwards works by adding a negative number to the program counter.
- A quick decode of a negative byte: subtract 256 (so $A2 = 162 - 256 = -94).

Worked example

What does the byte $F6 mean as a signed number?

1. $F6 = 1111 0110; the top bit is 1, so it is negative.
2. Complement: 0000 1001. Add 1: 0000 1010 = $0A = 10.
3. So $F6 = **-10**. (Check: 246 - 256 = -10.)

> **Key fact:** Complement all the bits, then add 1 (for example $5E gives 1010 0001 + 1 = $A2). An 8-bit signed number runs from -128 to +127; 16-bit from -32,768 to +32,767.

Signed overflow

Learn

Because the range is cut in half by the sign bit, **overranging is always possible**. Two's complement addition goes wrong only when:

- you add **two positive numbers** and the sum exceeds +127 (byte), or
- you add **two negative numbers** and the sum is below -128.

Adding a **positive and a negative** number is always correct. The **overflow flag (V)** in the status register warns when an overflow may have occurred.

Quick check by sign: if the operands have the **same sign** but the result has the **opposite sign**, V is set.

Worked example

Does $64 + $64 overflow as bytes?

1. $64 = 100, so the true sum is 200.
2. A signed byte tops out at 127, so 200 does not fit.
3. The byte holds $C8, which reads as -56. **V = 1** (two positives gave a negative).

> **Key fact:** Only when both operands have the same sign (two positives, or two negatives) and the true sum does not fit the size. The V (overflow) flag in the status register warns of it.

The 68000 registers and modes [68000]

Learn

The 68000's programmer-visible registers:

- **Data registers D0 to D7**: 8 general-purpose registers, each holding a **byte, word or long word** (8, 16 or 32 bits).
- **Address registers A0 to A6 and A7**: hold addresses so that memory addresses can be manipulated mathematically. They can only hold **words or long words**, never bytes. **A7 is the**

stack pointer.

- **A7 and A7'**: the **user** and **supervisor** stack pointers; they are **banked**, so only one is active, depending on the processor mode.
- **Program counter (PC)**: always holds the address of the next instruction; changing it changes program flow (this is how if-then-else works).
- **Status register (SR)**: holds the **condition code register (CCR)** flags (X, N, Z, V, C), processor status, the debug (trace) bit and the interrupt level.

Internally there are also hidden registers: the **effective address register** (holds the memory address being used), the **instruction register** (holds the first word of the current instruction) and a **temporary register**.

Two modes: user mode can execute only a subset of instructions, which keeps applications from interfering with each other under an O/S; **supervisor mode** uses the supervisor stack pointer A7'.

Worked example

A 68000 program needs to hold a byte count, a word index and a long address. Which registers?

1. The byte and the word go in **data registers** (for example D0 for the count and D1 for the index): data registers accept any size.
2. The long address goes in an **address register** (for example A0). It could not be a byte.
3. Stack operations use A7 implicitly.

> **Key fact:** D0-D7 data registers (8/16/32 bits); A0-A6 address registers plus A7 (user stack pointer) and A7' (supervisor stack pointer), banked by mode; PC (program counter); SR (status register with the CCR flags X N Z V C). Address registers hold words or long words only.

The 68000 condition code flags [68000]

Learn

The **condition code register (CCR)** is the low byte of the status register. Its five flags display the result of the instruction just executed:

Flag	Name	Set when
X	Extend	a carry bit for multi-precision arithmetic
N	Negative	the result is negative (MSB = 1)
Z	Zero	the result is zero
V	Overflow	the 2's complement result overflowed the register
C	Carry	the operation generated a carry (or a borrow)

A common use is **compare**. `CMP.W D0,D1` subtracts D0 from D1 but throws away the result and just sets the flags:

- **Z** set if D0 = D1 (because D1 - D0 = 0).
- **N** set if D0 > D1 (the result D1 - D0 is negative).
- **V** set if D1 - D0 overflowed.
- **C** set if D1 - D0 resulted in a borrow.

A conditional branch then tests those flags (for example `BEQ` branches when Z is set).

Worked example

D0 = 5 and D1 = 5 (words). After `CMP.W D0,D1`*, which flags are set, and does* `BEQ` *branch?*

1. D1 - D0 = 0, so **Z = 1**.
2. The result is not negative (N = 0), there is no overflow (V = 0) and no borrow (C = 0).
3. `BEQ` branches when Z = 1, so **it branches**.

> **Key fact:** X extend (carry for multi-precision), N negative (MSB = 1), Z zero, V overflow (2's complement result overflowed), C carry (or borrow). CMP.W D0,D1 does D1 - D0 and sets them: Z if equal, N if D0 > D1.

Number notation in 68000 assembly [68000]

Learn

In the lecture's 68000 assembler, the **base** of a number is shown by a **prefix**:

Base	Prefix	Example
Hexadecimal	**$**	`$AF56`, `$55`
Binary	**%**	`%10101100`
Octal	@	
Decimal	none	255

Notation is **assembler dependent**. In C or C++, hex uses `0x` and binary `0b`; on the Z80 hex ends with `H`. A `#` in front of an operand (as in `MOVE.B #$55,D7`) means an **immediate** value, a separate idea from the base.

Worked example

Write 255 in all three 68000 bases.

1. Decimal: `255`.
2. Hex: 255 = 15 x 16 + 15 = `$FF`.
3. Binary: `%11111111`.

> **Key fact:** $ hex ($AF56, $55), % binary (%10101100), @ octal, no symbol for decimal. (Assembler dependent.)

The Z80 registers [Z80]

Learn

Callback. The 68000 gave you eight wide data registers and eight address registers. The Z80 is a smaller, 8-bit machine:

- **A** (accumulator): **must** be used with any arithmetic or logical operation (`ADD A,D`, `AND B`).
- **F** (flags): the results of arithmetic and logical operations.
- **B, C, D, E, H, L**: 8-bit general-purpose registers (`LD E,55H`, `LD C,L`).
- **Register pairs BC, DE, HL**: 16-bit values; they become specific for some instructions, for example `LD B,(HL)` uses the address in HL as a pointer.
- **Alternate registers** A', F', B', C', D', E', H', L': swapped with the main set by a **single instruction** (`EXX`, `EX AF,AF'`) for fast interrupt handling.
- **IX and IY**: 16-bit index registers for tables and data structures: `LD (IX+d),L` stores L at IX plus an 8-bit displacement.
- **SP**: the 16-bit stack pointer; the stack is LIFO and **grows toward lower memory**. `PUSH BC` stacks a pair; `CALL 2000H` pushes the PC and loads 2000H; `RET` pops it back into the PC.

- **PC**: 16-bit program counter.
- **I** (interrupt vector) and **R** (memory refresh).

Worked example

Add the contents of D to A, then add 1 to the 16-bit pair HL. Which registers are used?

1. `ADD A,D`: the accumulator **A** must be one operand and gets the result.
2. `INC HL`: HL is a 16-bit **register pair** and is incremented as one value.
3. The flags in **F** are updated by the add.

> **Key fact:** A (accumulator) and F (flags); B, C, D, E, H, L (8-bit; pairs BC, DE, HL for 16-bit); alternate set A', F', B'-L' (swapped with EXX and EX AF,AF'); IX and IY (16-bit index registers); SP (stack pointer, grows toward lower memory); PC (program counter); I and R.

The Z80 flag register [Z80]

Learn

Callback. The 68000's flags were X N Z V C. The Z80's **F register** holds:

Z80 flag	Name	Closest 68000 flag	Meaning
Z	Zero	Z	Set if the result is zero
S	Sign	N	Set if the result is negative (MSB = 1)
C	Carry	C	Set if the result generates a carry
P/V	Parity / overflow	V	Set if the 2's complement result overflows; on the Z80 also indicates an **even number of bits set to 1**
H	Half carry	none	Carry bit for BCD arithmetic
N	Subtract	none (different N)	Set if the last operation was a subtraction
none	none	**X** (Extend)	Carry bit for multi-precision arithmetic

Careful: the letter **N** means *negative* on the 68000 but *subtract* on the Z80. The Z80's *sign* flag is **S**.

Worked example

On a Z80, `CP D` *(compare) sets Z when A = D. What are Z and S after A = $10 and D = $20?*

1. A - D = $10 - $20 = -$10, not zero: **Z = 0**.
2. The result is negative: **S = 1**. (A comparison also sets N = 1 because it is a subtraction.)
3. This matches the 68000's `CMP` logic, except that the compare is against the accumulator A.

> **Key fact:** Z = Z (zero). S = N (negative / sign). C = C (carry). P/V = V (overflow; also parity on the Z80). 68000 only: X (extend). Z80 only: H (half carry, BCD) and N (last operation was a subtraction).

Number notation in Z80 assembly [Z80]

Learn

Callback. The 68000 uses `$` in front of hex. The Z80 puts an **H after** it:

- `55H`, `0FFFFH`.

- A number that would begin with a letter A-F **must begin with 0** (`0A4H`, not `A4H`), otherwise the assembler reads it as a name.

Number	68000	Z80
55 hex	`$55`	`55H`
FFFF hex	`$FFFF`	`0FFFFH`
A4 hex	`$A4`	`0A4H`

In C and C++, hex is `0x` and binary is `0b`.

Worked example

Write `$FF35` *for a Z80 assembler.*

1. The hex digits are FF35 and it starts with F (a letter).
2. Add a leading 0 and a trailing H: `0FF35H`.

> **Key fact:** Z80: hex ends with H and needs a leading 0 if it starts with A-F (0FFFFH, 55H). 68000: hex starts with $ ($FFFF, $55).

Introduction to assembly language

Why assembly, machine code versus assembly, the layout of a line, and flow charts.

Why learn assembly language

Learn

Reasons from the lecture:

- **Hardware bring-up.** Electrical engineers must turn on new hardware, and the most direct way to write test drivers is in assembly. When functionality is assured the coders write the applications in C, C++ or Java.
- **It is tied to the processor.** There is a **1:1 match** between an assembly instruction and a machine instruction, so understanding assembly means understanding the architecture from the programmer's point of view.
- **Tightest, fastest code.** No overhead in space or time. Game engines and critical parts of programs are written in assembly.
- **Compilers are not perfect**, especially for interrupt handlers and DSP algorithms; not every architecture (for example 4-bit parts) has a compiler.
- **Determinism.** Code that must be absolutely, positively deterministic cannot be written in C; assembly controls the program at the level of a single clock cycle.
- **Run-time environments** need some assembly, and **debugging real-time systems** requires understanding it.

Worked example

An interrupt handler must respond within exactly 40 clock cycles. Which language, and why?

1. A compiler decides the instructions, so its cycle count is hard to guarantee.
2. In assembly you choose each instruction and know its cycle count, so you can control timing at the single-clock level.
3. Use **assembly** (this is the lecture's example of critical code and interrupt handlers).

> **Key fact:** Bring-up and test drivers for new hardware; 1:1 match to machine code, so you understand the architecture; tightest, fastest code with no overhead; deterministic timing to a single clock cycle; parts of the run-time environment; debugging real-time systems; not every architecture has a compiler.

Machine language, assembly and the assembler

Learn

- **Machine language** is the pattern of bits the processor's finite state machine interprets to start a series of state transitions. That pattern is an **instruction**. Each instruction is **atomic**: it cannot be subdivided and, once started, it completes.
- **Assembly language** is machine language in a human-readable form. You can program in machine language, but it is not recommended.
- The **assembler** is the program that converts assembly language to machine language.

Most instructions have two parts, like a sentence:

Part	Role	Examples
Operation (verb)	What to do; also called the **mnemonic**	add, subtract, multiply, divide, AND, OR, XOR, copy data, test data
Operands (nouns)	The data being manipulated	where it is (registers, memory, a literal number); how to access it (**addressing modes**); what to do with the result

Worked example

Describe `MOVE.B D0,D3` *in these terms.*

1. Operation: **MOVE** (copy), size **.B** (byte).
2. Operands: the data is in **D0** (a register); the result goes to **D3**.
3. The assembler turns it into the machine word `$1600`, which you will see in the next lesson on machine code.

> **Key fact:** Operation (mnemonic: add, subtract, AND, copy, test) plus operands (where the data is, how it is accessed, where the result goes). The assembler converts assembly into machine language: bit patterns the processor's state machine acts on.

The layout of an assembly line

Learn

A 68000 assembly line has up to four fields, separated by spaces or tabs:

```
LABEL OP CODE OPERAND1,OPERAND2 * COMMENT
```

- **Label**: optional, **must start in column 1**, usually limited to about 14 characters (the exact limit depends on the assembler). The first whitespace after it starts the op-code field.
- **Op-code**: column 2 or higher; it includes the **size** (`.B`, `.W`, `.L`) on the 68000.
- **Operands**: the first whitespace after the op-code starts them. Separate them with a **comma, with no spaces**.
- **Comment**: starts with `*` or `;` (a `:` is also accepted after the operands as in the slide).
- **Every instruction is exactly one line**, at most 132 characters.

Labels name an instruction or variable, make the program readable and let the assembler calculate branch and jump addresses. Instruction counts by operands: `NOP` (0 operands), `BRA FOO` (1), `ADD.W D0,D3` (2).

Worked example

Label the fields of: `LOOP MOVE.B (A2),D6 *Get the pattern`.

1. Label: **LOOP** (column 1).
2. Op-code: **MOVE.B**.
3. Operands: **(A2),D6** (no space after the comma).
4. Comment: ***Get the pattern**.

> **Key fact:** LABEL (optional, column 1), OP-CODE (column 2 or later, after whitespace), OPERANDS (comma separated, no spaces), COMMENT (starts with * or ;). One instruction per line, up to 132 characters.

Flow charts for assembly programs

Learn

A flow chart is a useful way to plan an **assembly** language program (it is typically not used for high-level languages). The symbols:

- **Rectangle**: a distinct operation; it may be one instruction, several or many, and generally means "do something".
- **Diamond**: a **decision point** (program flow control). It has two exits, TRUE and FALSE.
- **Arrow**: the direction of program flow.

Students tend to write the program and then draw the chart; the lecture says to draw it first. Flow charts save time and prevent avoidable errors, and one is **required** for the homework.

The lecture's "largest of four bytes" chart reads: get the first and second numbers; if 1 >= 2 save 1, else save 2; get the third; if 3 >= saved keep saved, else save 3; and so on, then quit.

Worked example

Chart a loop that clears memory from START to END.

1. Rectangle: set the pointer to START.
2. Diamond: pointer = END? If yes, go to Done.
3. If no, rectangle: clear the location and advance the pointer, then arrow back to the diamond.
4. This is the WHILE-loop shape the lecture uses.

> **Key fact:** Rectangle: a distinct operation (one or many instructions). Diamond: a decision. Arrow: direction of flow. Draw it before coding an assembly program.

First assembly programs

Machine code, directives, copying data, and two worked programs on each processor.

68000 machine code: the op-code word and extension words [68000]

Learn

Every 68000 instruction is **at least 16 bits** long. The first word is the **op-code word**: it holds the operation, and the addressing modes for the source and destination operands, and it tells the processor how many more words belong to the instruction. Extra words carry immediate data or absolute addresses (high word first).

Assembly	Machine code	Words
`NOP`	`$4E71`	1
`MOVE.B D0,D3`	`$1600`	1
`ADD.L D4,D7`	`$DE84`	1
`ADDI.W #$4000,D5`	`$0645 $4000`	2
`MOVE.W (A6),$4000A000`	`$33D6 $4000 $A000`	3
`ADDI.W #$AA55,$5A33A706`	`$0679 $AA55 $5A33 $A706`	4
`MOVE.B $AA550000,$5A33A706`	`$13F9 $AA55 $0000 $5A33 $A706`	5

Every encoding in this table is checked by assembling the line with this site's 68000 assembler.

Instruction length is why the 68000 is CISC: it varies from 2 bytes up to 10.

Worked example

How many bytes is `MOVE.W $0A550000,$1000BB00`*?*

1. Op-code word `$33F9`: 2 bytes.
2. Source absolute long address: 2 words = 4 bytes.
3. Destination absolute long address: 4 bytes.
4. Total: 2 + 4 + 4 = **10 bytes**, five words.

> **Key fact:** At least one 16-bit op-code word (operation, size, addressing modes), then extension words for immediate data or absolute addresses (high word first). Examples: NOP $4E71; MOVE.B D0,D3 $1600; ADD.L D4,D7 $DE84; ADDI.W #$4000,D5 $0645 $4000.

68000 assembler directives [68000]

Learn

Pseudo-ops (assembler **directives**) tell the assembler how to assemble and load the program. They do **not** generate executable instructions, except `MACRO`.

Directive	Meaning
`ORG <expr>`	Where the code will execute in memory; resolves address references
`label EQU <expr>`	Symbolic name for a constant (like `#define`); replaced at assembly time
`label SET <expr>`	Like EQU, but may be redefined later
`label REG <list>`	A register list such as `A0–A3/A5/D0–D7`, used only with `MOVEM`
`DC.<size> a,b,c`	Define constants; `DC.B` also takes ASCII in quotes: `DC.B 'Error',$0D,$0A,0`
`DCB.<size> n,value`	Define a block of n copies of a value: `DCB.B 100,$AA`
`DS.<size> n`	Define storage: n uninitialised units: `DS.L 200` is 800 bytes
`END <address>`	End of source; the address tells the **loader** where to put the code
`OPT ...`	Assembler options (CRE cross-reference, BRB/BRW 8/16-bit branches, MEX macro expansion)
`MACRO ... ENDM`	A named group of instructions; `\1` is a parameter

ORG and END are complementary: ORG tells the assembler how to resolve addresses; END tells the loader where to put the code. A label equated with EQU does not use any memory.

The oracle check at the top of this concept assembles and runs a program that uses EQU, DCB, DC.B and DS together.

Worked example

Define a 3-byte message "Hi" ending in 0, then a 2-long-word buffer.

1. `MSG DC.B 'Hi',0`: the bytes $48 $69 $00.
2. `BUF DS.L 2`: reserves 2 x 4 = 8 uninitialised bytes right after.
3. Neither line produces an executable instruction; they only place data.

> **Key fact:** ORG: where code is assembled to run. EQU: symbolic constant (like #define). SET: like EQU but redefinable. REG: register list for MOVEM. DC: define constants (DC.B can hold ASCII). DCB: block of the same value. DS: uninitialised storage. END: end of source and start address for the loader. OPT: assembler options. MACRO: groups instructions (generates code).

MOVE: copying data on the 68000 [68000]

Learn

The most frequently used instructions move data between the processor and memory, or between internal registers. **MOVE** (68000) and **LD** (Z80) are misleading names: they **copy**. The source is left unchanged.

68000 forms from the lecture (source first, destination second):

- `MOVE.B D0,D1` copy the byte in D0 to D1.
- `MOVE.W D7,$004AAA00` copy the word from D7 to memory location $004AAA00.
- `MOVE.L (A3),D4` copy the long word at the address pointed to by A3 into D4.

The size is part of the instruction. `#` before a number marks an **immediate** value (`MOVE.B #$55,D7`); `(A3)` means **the memory location whose address is in A3**.

This concept's program is assembled and run on the 68000 core, and the registers and memory it ends with are asserted.

Worked example

D0 = $11. After `MOVE.B D0,D1` *and* `MOVE.W #$2233,D7`*, what are D1 and D7?*

1. `MOVE.B D0,D1` copies the low byte: D1's low byte = $11 (the upper bytes of D1 are untouched).
2. `MOVE.W #$2233,D7` loads $2233 into D7's low word.
3. D0 still holds $11.

> **Key fact:** It copies data from the first operand (source) to the second (destination); the source is unchanged. The size suffix .B/.W/.L says how many bits. Examples: MOVE.B D0,D1; MOVE.W D7,$004AAA00; MOVE.L (A3),D4.

Zeroing a block of memory (68000) [68000]

Learn

The lecture starts simple and improves the code step by step, to clear a block from `$00002000` up to but **not including** `$00002008`:

1. **Eight** `MOVE.B #$00,addr` **instructions**: works, but inefficient and not suitable for large blocks.
2. **Two** `MOVE.L #0,addr`: clear 4 bytes at a time (the end must be on a long-word boundary).
3. **Address register as a pointer**: `MOVEA.L #$2000,A0` then `MOVE.L #0,(A0)`; use `MOVEA` when the destination is an address register.
4. **Auto-increment**: `MOVE.L #0,(A0)+` clears and advances the pointer, nearly halving the code.
5. `LEA` initialises the pointer.
6. **A loop**, once the block is large. Three things to settle: the loop type (FOR, WHILE, DO-WHILE), how to initialise it, and how to stop it.

The final program (checked by running it on this site's 68000 core, with the block preloaded to $FF so it must really clear it):

```
st_addr   EQU   $00002000
end_addr  EQU   $00003000
          ORG   $1000
START     LEA     st_addr,A0     * pointer to the start
          LEA     end_addr,A1    * pointer to the end
clr_loop  CMPA.L  A1,A0          * A0 - A1, result ignored
          BEQ     done           * Z = 1 when A0 = A1
          CLR.L   (A0)+          * clear a long word, advance 4
          BRA     clr_loop       * end of the loop (a GOTO)
done      SIMHALT
          END     START
```

Pseudo-ops: `EQU` makes the program readable and flexible (like `#define`); `ORG` sets where it assembles; `END START` marks the end and the start address. Header and comment lines start with * or `;`.

Worked example

Trace the loop for `st_addr = $2000` *and* `end_addr = $2008`.

1. A0 = $2000, A1 = $2008. Compare: not equal, so Z = 0 and BEQ is not taken.
2. `CLR.L (A0)+` clears $2000-$2003; A0 = $2004. BRA back.
3. Compare again: still not equal; clear $2004-$2007; A0 = $2008.
4. Compare: A0 = A1, Z = 1, BEQ takes the branch to `done`. **$2008 itself is never cleared.**

> **Key fact:** LEA st_addr,A0 / LEA end_addr,A1 / clr_loop CMPA.L A1,A0 / BEQ done / CLR.L (A0)+ / BRA clr_loop / done SIMHALT. CLR.L (A0)+ clears a long word and advances the pointer by 4.

Finding the largest of four bytes (68000) [68000]

Learn

The lecture's sample program compares four bytes stored in successive memory locations and saves the largest. The method: get two numbers, compare, keep the larger in D0; get the next, compare, keep the larger; and so on. The flow chart's diamonds are `CMP.B D1,D0` + `BGE`:

```
START   LEA     DATA,A0         * pointer to data
        LEA     MAX_NUM,A1      * pointer to result
        MOVE.B  (A0)+,D0        * first number
        MOVE.B  (A0)+,D1        * second number
        CMP.B   D1,D0           * D0 - D1, flags only
        BGE     NEXT1           * keep D0 if D0 >= D1
        MOVE.B  D1,D0           * else the second is bigger
NEXT1   ...                     * repeat for the third and fourth
DATA    DC.B    $05,$4C,$42,$7F
MAX_NUM DS.B    1               * result goes here
```

The assembler's **list file** shows each instruction's address and machine code, and a **symbol table** (for example `DATA = 1036`, `START = 1000`). Data and variable regions are placed after the code; `END START` ends the source.

Signed or unsigned? `BGE` treats bytes as signed, so `$90` counts as -112. The program therefore picks `$42` from `$05,$90,$42,$30`. If you want the unsigned maximum you need an unsigned branch such as `BHS`; this is exactly the signed/unsigned ambiguity of Homework 3.

Worked example

Trace the first comparison for data $05 then $4C.

1. D0 = $05, D1 = $4C. `CMP.B D1,D0` computes $05 - $4C = $B9 (negative).
2. N = 1 and V = 0, so BGE (taken when N = V) is **not** taken.
3. `MOVE.B D1,D0` runs: D0 = $4C, the larger.

> **Key fact:** MOVE.B (A0)+,D1 / CMP.B D1,D0 / BGE keep / MOVE.B D1,D0 / keep ... D0 holds the largest so far. BGE is a signed test, so bytes $80-$FF count as negative.

Z80 machine code: opcode bytes and the direction of the operation [Z80]

Learn

Callback. In the 68000 lesson an instruction began with a 16-bit op-code word. On the Z80 the first **byte** is the **opcode**, and it holds all the information needed to decode the rest. The lecture's example: `ADD A,n` adds an 8-bit number n to the accumulator and puts the result in A. With A = 33H and n = 22H, afterwards A = 55H. The machine code is **C6 22**.

Z80 code that you will see on the exam as pairs with the 68000:

68000	Z80	Meaning
`MOVE.B D0,D1`	`LD B,A`	68000 copies D0 to D1; Z80 copies A to B
`CMP.B D0,D1`	`CP D`	68000 compares the byte values in D0 and D1; Z80 compares register D with A
`BEQ Label`	`JR Z,Label`	both branch to Label if the Zero flag is true

Direction. On the 68000 the **source comes first** (source + destination goes to destination). On the Z80 the **destination comes first** (destination = destination + source). Also note that the Z80 can use **A only**

for arithmetic and logic.

Worked example

Which register ends up changed by `LD B,A`*, and by* `MOVE.B D0,D1`*?*

1. `LD B,A`: destination is first, so **B** is overwritten with A.
2. `MOVE.B D0,D1`: source is first, so **D1** is overwritten with D0.
3. Reading order is reversed between the processors.

> **Key fact:** Z80: opcode byte(s), then immediate bytes (ADD A,22H = C6 22). Operand order is destination, source (LD B,A copies A into B). 68000: op-code word then extension words, operand order is source, destination (MOVE.B D0,D1 copies D0 into D1).

Z80 assembler directives [Z80]

Learn

Callback. The 68000 directives you met (ORG, EQU, DC, DS, END) all exist on the Z80, sometimes under other names:

Directive	68000	Z80
Set origin	`ORG`	`ORG`
Equate	`EQU`	`EQU`
End of source	`END`	`END`
Define byte	`DC.B`	`DEFB`
Define word	`DC.W`	`DEFW`
Define storage in bytes	`DS.B nn`	`DEFS nn`
Define storage in words	`DS.W nn`	N/A
ASCII string	`DC.B 'string'`	the slide prints `DEFII 's'`, which looks like a typo; in Oshonsoft style use `.DB "string"`

The **Oshonsoft** Z80 assembler used in the lab requires a **period in front** of the pseudo-op codes: `END` becomes `.END`. (This site's Z80 sandbox follows that style, with `.ORG`, `.EQU`, `.DB`, `.DS` and `.END`.)

Worked example

Reserve a 2-byte buffer named BUF and define the string "Hi" ending in 0, in Oshonsoft style.

1. `MSG: .DB "Hi",0` places the bytes 48H 69H 00H.
2. `BUF: .DS 2` reserves two uninitialised bytes.
3. Both are directives: no executable code is generated.

> **Key fact:** ORG = ORG, EQU = EQU, END = END, DC.B = DEFB, DC.W = DEFW, DS.B nn = DEFS nn, DS.W = N/A, DC.B 'string': the slide prints DEFII 's' (a typo; Oshonsoft style is .DB with the string in quotes). In the Oshonsoft assembler every pseudo-op begins with a period (.ORG, .EQU, .DB, .DS, .END).

LD: copying data on the Z80 [Z80]

Learn

Callback. `MOVE` copied data on the 68000. On the Z80 the instruction is **LD** ("load"), and it also **copies**. The big difference is operand order: **destination first**.

- `LD A,C` copies the byte from C into A.
- `LD A,(1234H)` puts the contents of memory address 1234H into A.
- `LD (HL),A` copies A into the memory location pointed to by HL.

The Oshonsoft assembler accepts brackets as well: `LD A,[HL]`. There is no size suffix: 8-bit loads are `LD r,r'`, and 16-bit loads use register pairs (`LD HL,1240H`).

Compare with the lecture's pairs: `MOVE.B D0,D1` copies D0 to D1; `LD B,A` copies A to B.

Worked example

Copy the byte at address 1234H into B, using only A as the go-between.

1. `LD A,(1234H)`: memory to A.
2. `LD B,A`: A to B.
3. Both copy; the memory byte is not changed. (The program above does the same and is run on the Z80 core.)

> **Key fact:** Both copy and leave the source unchanged. Z80 writes the destination first: LD A,C (C to A), LD A,(1234H) (memory to A), LD (HL),A (A to memory at HL). The 68000 writes the source first: MOVE.B D0,D1.

Zeroing a block of memory (Z80) [Z80]

Learn

Callback. The 68000 loop was `LEA`, `LEA`, `CMPA.L A1,A0`, `BEQ done`, `CLR.L (A0)+`, `BRA`. The same job on the Z80 uses **HL as the pointer** and **DE as the end address**:

```
START:  LD HL,STADDR
        LD DE,ENADDR
LOOP:   LD A,H          ; compare the high bytes
        CP D
        JR NZ,CLR       ; different: not at the end
        LD A,L          ; same: compare the low bytes
        CP E
        JR Z,DONE       ; equal: finished
CLR:    LD [HL],0       ; clear one byte
        INC HL          ; advance the pointer by hand
        JR LOOP
DONE:   HALT
```

Differences to notice: no `CMPA`-style 16-bit compare (so two byte compares through A), no auto-increment (so `INC HL`), one byte per pass instead of a long word, and `HALT` in place of the simulator's `SIMHALT`. The lecture shows only the 68000 version; this Z80 version is our own, and it is run on the Z80 core with the block preloaded to FFH to prove it clears exactly 2000H to 200FH.

Worked example

Which flags drive the two branches in the loop?

1. `CP D` sets Z when H = D. `JR NZ,CLR` is taken when Z = 0 (high bytes differ, so definitely not at the end).
2. If the high bytes match, `CP E` sets Z when L = E, and `JR Z,DONE` leaves the loop.

3. The 68000 gets the same result from a single Z flag after `CMPA.L`.

> **Key fact:** LOOP: LD A,H / CP D / JR NZ,CLR / LD A,L / CP E / JR Z,DONE / CLR: LD (HL),0 / INC HL / JR LOOP. No auto-increment and no 16-bit compare, so the pointer is advanced by hand and compared a byte at a time.

Finding the largest of four bytes (Z80) [Z80]

Learn

Callback. The 68000 program kept the running maximum in D0 and compared with `CMP.B D1,D0` and `BGE`. The Z80 version keeps it in **A** (the accumulator is where compares happen) and walks HL along the data:

```
START:  LD HL,DATA
        LD A,[HL]       ; first number is the maximum so far
        LD B,3          ; three more to examine
LOOP:   INC HL
        CP [HL]         ; A - (HL), flags only
        JR NC,NEXT      ; no carry: A >= (HL), keep A
        LD A,[HL]       ; otherwise the new byte is larger
NEXT:   DJNZ LOOP
        LD [MAXNUM],A
        HALT
```

Note the contrast the programs expose: the 68000 version used **BGE (signed)** and returned `$42` for `$05,$90,$42,$30`; this Z80 version uses the **carry flag (unsigned)** and returns `90H`. Both processors have signed and unsigned tests; the difference is the test chosen. Both results are confirmed by running the programs.

Worked example

Trace data 05H, 4CH, 42H, 7FH.

1. A = 05H, B = 3. Examine 4CH: A - 4C borrows (carry set), so `JR NC` is not taken and A = 4CH.
2. Examine 42H: 4C - 42 does not borrow, so A stays 4CH.
3. Examine 7FH: 4C - 7F borrows, so A = 7FH. DJNZ ends the loop (B = 0). MAXNUM = 7FH.

> **Key fact:** LD A,[HL] (first) ... INC HL / CP [HL] / JR NC,NEXT (A >= byte) / LD A,[HL] / NEXT. The carry test is an unsigned compare; the 68000 program used BGE, a signed compare. A holds the running maximum, DJNZ counts the loop.

Addressing modes

How the 68000 finds its operands: the effective address, register, indirect, auto-increment, displacement, index, absolute, immediate and PC-relative modes.

Effective addresses: modes 0 to 7 [68000]

Learn

An instruction's **op-code word** is always 16 bits and holds: what to do (ADD, AND, XOR), the size, where to find the source, where to put the destination, and extra operations such as post-increment or pre-decrement.

The **effective address (EA)** says where an operand is. It is a 6-bit field made of two 3-bit parts:

- bits **3-5**: the **mode** (0 to 7),
- bits **0-2**: the **register** (0 to 7).

That is why there are **8 modes** and **8 data registers** and **8 address registers**.

Example: `ADD.B D0,(A0)` assembles to `$D110`:

```
15 14 13 12 | 11 10 9 | 8 7 6 | 5 4 3 | 2 1 0
 1  1  0  1 |  0  0 0 | 1 0 0 | 0 1 0 | 0 0 0
   ADD      |   D0    | opmode| mode 2| reg 0 (A0)
```

The bit fields do not line up with hex digits, which is why you decode instructions in binary.

Each instruction has its own set of permitted modes. For example `MOVE.W D0,D1` is legal; `MOVE.W D0,A1` is not (use `MOVEA.W D0,A1`).

Worked example

Decode the low six bits of `$D110`.

1. `$D110` = 1101 0001 0001 0000. The low six bits are 010000.
2. Bits 5-3 = 010 = **mode 2** (address register indirect).
3. Bits 2-0 = 000 = **register 0** = A0. So the destination is `(A0)`.

> **Key fact:** In the low six bits: bits 3-5 = mode (0-7), bits 0-2 = register (0-7). Hence 8 modes and 8 registers. Each instruction allows only some modes (MOVE.W D0,A1 is illegal; use MOVEA).

Modes 0 and 1: register direct [68000]

Learn

- **Mode 0**: the source or destination is a **data register** (D0 to D7).
- **Mode 1**: it is an **address register** (A0 to A7; A7 may be written SP).

Register direct addressing is **efficient** (only 3 bits in the op-code word name one of 8 registers) and **fast** (external memory does not have to be accessed), so programmers keep frequently used data in registers. Compilers use it for performance, and modern processors have 32 or 64 general registers (the AMD AM29000 had 256).

In-class exercises from the lecture, answered here: exchange D0 and D1 with `EXG D0,D1`; exchange D0-D3 with A0-A3 with four `EXG Dn,An` instructions; add the bytes of D3 and D4 with `ADD.B D3,D4`, and a byte sum above $FF wraps and sets the **carry** flag.

Worked example

Add $F0 and $20 as bytes. What happens?

1. $F0 + $20 = $110: nine bits needed.
2. `ADD.B` keeps only the low 8 bits: **$10**.
3. The ninth bit becomes the **carry flag** (C = 1; X is set the same way). The program above is run to confirm.

> **Key fact:** Mode 0: the operand is a data register (D0-D7). Mode 1: an address register (A0-A7; A7 is also called SP). Fast and compact because no memory access is needed.

Mode 2: address register indirect [68000]

Learn

Indirect addressing: the value in a register (or memory location) is the **address of the data**, not the data. A stored address is a **pointer**. On the 68000 only address registers hold pointers, and the parentheses mean "the memory that this register points to":

- `MOVE.L D3,(A3)` writes D3 to the memory address in A3. With `<A3> = $00A34000` and `<D3> = $F34CEF00`, the long word $F34CEF00 is written at $00A34000.
- `MOVEA.W D0,A0` copies D0 into A0; `MOVE.W D0,(A0)` copies D0 into the memory A0 points to.

Indirect addressing lets you do **arithmetic on addresses**. Other architectures differ: some can do indirect addressing through memory locations (not the 68K), ARM lets any register hold addresses, the 80x86 uses indirect addressing almost exclusively, and the Z80 reaches memory through `[HL]` (`LD [HL],B`). C and C++ pointers are the same idea.

The 68000 is **big-endian**: a long word stored at $2000 keeps its most significant byte at $2000. So a byte instruction on `(A5)` changes only the first byte of that long word.

Worked example

`<A5> = $2000`*, long word at $2000 = $3FD90146,* `<D0> = $0056AB27`. *Compute* `ADD.B D0,(A5)`.

1. The operand size is a byte: only the byte at $2000 (the $3F) is involved.
2. The source byte is the low byte of D0, $27.
3. $3F + $27 = $66, so memory becomes **$66D90146**. The other three bytes are untouched. (The program above is run on the core and shows exactly this.)

> **Key fact:** (An): A_n holds a pointer, and the operand is the memory at that address (parentheses = indirect). On a big-endian 68000 a byte access at the long word's address touches its most significant byte; ADDQ.L #3 on the pointer reaches the least significant byte.

Modes 3 and 4: post-increment and pre-decrement [68000]

Learn

- **Mode 3, post-increment** `(An)+`: after the instruction uses the address, the address register is **incremented** by the operand size.
- **Mode 4, pre-decrement** `-(An)`: **before** the instruction uses it, the address register is **decremented** by the operand size.

Both are **size aware**: `MOVE.B D0,(A2)+` adds 1 to A2, `MOVE.W D1,(A6)+` adds 2, `MOVE.L D4,(A3)+` adds 4.

They are made for walking through consecutive memory and for **stacks**. A 68K stack is initialised at the top of memory and grows toward lower addresses. When the stack grows downward:

- **PUSH** = indirect with **pre-decrement**: `MOVE.B D0,-(SP)`.
- **POP** = indirect with **post-increment**: `MOVE.B (SP)+,D0`.
- The stack pointer points at the last item pushed.

(A stack growing the other way would use post-increment for PUSH and pre-decrement for POP, with the pointer at the next empty slot.)

The lecture's loop adds five bytes with `ADD.B (A0)+,D1` and `SUBQ.B #1,D0/BNE` as the counter; the table 1, 4, 2, 6, 5 sums to 18. All three programs here run on the 68000 core.

Worked example

Trace `MOVE.L #$87654321,-(SP)` *with SP = $00100000.*

1. Pre-decrement by 4: SP = $00FFFFFC.
2. Write the long word at $FFFFFC: the high word $8765 at $FFFFFC, the low word $4321 at $FFFFFE.
3. `MOVE.L (SP)+,D5` then reads it back into D5 and returns SP to $00100000.

> **Key fact:** (An)+: use the address, then add the operand size (1/2/4). -(An): subtract the size, then use the address. PUSH = MOVE x,-(SP); POP = MOVE (SP)+,x. The 68000 stack grows from high memory toward low.

Modes 5 and 6: displacement and index [68000]

Learn

- **Mode 5, address register indirect with displacement** `(d16,An)`: a **16-bit two's complement** displacement is added to the address register. The displacement may be a number or a label: `MOVE.W D0,($1300,A3)`. Compilers use it to reach variables as offsets from a pointer (A5 points to the data region).
- **Mode 6, address register indirect with index** `(d8,An,Xn.L)` **or** `(d8,An,Xn.W)`: An + an index register (A0-A7 or D0-D7, used as word or long) + a **two's complement 8-bit displacement**. It suits two-dimensional arrays and linked lists: the address register points to the array start, the index register picks the row, and the displacement picks the element.

The lecture's example: a 50 x 50 array of long words starting at $4000, element (row 23, column 11):

- Index = 23 x 50 x 4 = 4,600 = **$11F8** (in D0).
- Offset = 11 x 4 = 44 = **$2C**.
- `MOVE.L ($2C,D0,A0),D3` (the assembler accepts the register order `($2C,A0,D0.L)`).

The effective address is $4000 + $11F8 + $2C = **$5224**. The program above is run on the core.

Worked example

Compute the address of element (row 23, column 11).

1. Array start = $4000 = 16,384.
2. Row offset = 23 x 50 x 4 = 4,600.
3. Column offset = 11 x 4 = 44.
4. Address = 16,384 + 4,600 + 44 = 21,028 = **$5224**.

> **Key fact:** Mode 5 (d16,An): address = An + sign-extended 16-bit displacement. Mode 6 (d8,An,Xn.W/.L): address = An + Xn + sign-extended 8-bit displacement. For an array: Xn = row x columns x bytes per element; d8 = column x bytes per element.

Mode 7: absolute and immediate addressing [68000]

Learn

Mode 7 is a group of "special" modes; its register field is a *subclass*: 0 = absolute word, 1 = absolute long, 2 = PC with displacement, 3 = PC with index, 4/5 = immediate.

Absolute (direct) addressing gives the exact address in the instruction. It needs extra fetches, so it is slow, takes extra memory and prevents code from being relocated. It is rarely used with high-level languages.

- **Absolute word (subclass 0)**: a 16-bit address, **sign-extended** to 32 bits. `$A3C0` becomes `$FFFFA3C0`. It reaches only **$000000-$007FFF** and **$FF8000-$FFFFFF**, never $008000-$FF7FFF. The lecture calls it **evil**: a source of many coding errors.
- **Absolute long (subclass 1)**: a 32-bit address, `MOVE.B D0,($00001234).L`. `OPT FRL` forces long forms everywhere.
- Rule of thumb: if an address is used repeatedly, load it into an address register.

Immediate addressing puts a number in the instruction (a *literal*), marked by `#`: `MOVE.B #24,D0` (decimal 24) versus `MOVE.B #$24,D0` (hex $24 = 36). It can only be a **source**. Use it once to initialise, not continually. For instructions other than MOVE the immediate form is its own instruction: `ADDI`, `SUBI`, `CMPI`, `ORI`.

A slide-accuracy note: the lecture shows `CLR.B $00001234 -> $4279 $0000 $1234`. By assembling it, `$4279 $0000 $1234` is the encoding of `CLR.W` with a long absolute address; `CLR.B $00001234.L` is `$4239 $0000 $1234`. Without `.L`, the assembler uses the short form `$4238 $1234` because the address fits in 16 bits.

Worked example

Which addresses can `MOVE.B D0,($7000).W` *and* `MOVE.B D0,($9000).W` *reach?*

1. $7000: top bit 0, so it stays $00007000, inside $000000-$007FFF. OK.
2. $9000: top bit 1, so it becomes **$FFFF9000**, inside $FF8000-$FFFFFF. It does **not** reach $00009000.
3. This is the trap: a short address can silently mean the top of memory.

> **Key fact:** Absolute word (short): 16-bit address sign-extended, reaches only $000000-$007FFF and $FF8000-$FFFFFF; 'evil'. Absolute long: full 32-bit address. Both slow and non-relocatable. Immediate: #value, source only, mostly to initialise; instructions other than MOVE have their own immediate forms (ADDI, SUBI, CMPI, ORI).

Mode 7: PC-relative addressing [68000]

Learn

PC-relative modes are like the register modes with the **program counter** instead of an address register:

- **Subclass 2, PC with displacement**: a signed 16-bit displacement is added to the PC. **EA = (PC) + d16**. Very important for **relocatable code**.
- **Subclass 3, PC with index**: the PC plus an index register (A0-A6 or D0-D6) plus an 8-bit displacement. **EA = (PC) + (Xn) + d8**.

Because the address is relative to where the instruction is, there is no need to initialise an address register each time the code runs, and the program works wherever it is loaded. **Branches** are the same idea: the destination is the PC plus a signed displacement.

In source you write a label and let the assembler compute the displacement: `MOVE.B (table,PC),D0` and `MOVE.B (table,PC,D1.W),D2`. PC-relative addressing can be used only to **read** operands (it is not a destination mode on the 68000).

Worked example

A table `$11,$22,$33` *sits after the code. Read the second element with PC-relative indexing.*

1. `MOVE.L #1,D1` sets the index.
2. `MOVE.B (table,PC,D1.W),D2` computes PC + index + displacement to land on table[1].
3. D2 = $22. (The program above is run and the register is asserted.)

> **Key fact:** Mode 7 subclass 2: EA = (PC) + d16. Subclass 3: EA = (PC) + (Xn) + d8. They give relocatable code (no address register needs initialising) and work like branches.

Z80 addressing: register, immediate, indirect and absolute [Z80]

Learn

Callback. You met the 68000's effective-address modes. The Z80 has the same ideas with fewer, more restricted forms. The lecture's comparisons:

Idea	68000	Z80
Register to register	`MOVE.B D0,D1`	`LD A,B` (copies B into A: note direction)
Immediate	`MOVE.B #$55,D1`	`LD C,55H`
Indirect through a pointer	`MOVE.B (A0),D1`	`LD B,[HL]` (HL is the pointer)
Absolute, 8-bit	`MOVE.B D0,$0000FF35`	`LD [0FF20H],A` (**only A**)
16-bit immediate	`MOVE.W #$ABFF,D1`	`LD DE,0A34BH`
Absolute, 16-bit	`MOVE.W D0,$0000FF36`	`LD [0FF20H],HL` (absolute only; **little endian**)

The Z80's bracket (or parenthesis) around an address is **required** for absolute addressing. A hex number starting with A-F needs a leading 0.

Endianness. The 68000 is **big endian** (high byte first). The Z80 stores 16-bit values **little endian**: `LD HL,1234H` then `LD [3020H],HL` puts 34H at 3020H and 12H at 3021H. The program above shows this by running on the Z80 core.

Worked example

Copy the byte at address 3000H into register B using a pointer.

1. `LD HL,3000H`: HL is the pointer.
2. `LD B,[HL]`: B gets the byte at the address in HL (no need to go through A).
3. The 68000 equivalent: `LEA $3000,A0` then `MOVE.B (A0),D1`.

> **Key fact:** Reg-reg: LD A,B (dest first). Immediate: LD C,55H; 16-bit LD DE,0A34BH. Indirect: LD B,[HL] (pair as pointer). Absolute 8-bit: only A: LD [0FF20H],A. Absolute 16-bit: LD [nn],HL, stored little endian (low byte first).

Program control, loops and subroutines

Compare and branch, signed versus unsigned, if/FOR/DO-WHILE, jumps, the stack and subroutines on both processors.

CMP, the Z and N flags and testing bits [68000]

Learn

Programs decide by changing the execution path: most programs execute only 5 to 7 instructions before a non-sequential one (branch or jump). A **conditional branch** looks at the **flags** in the CCR (Z, N, V, X, C), and combinations of flags give more conditions.

CMP versus SUB. Both subtract the source from the destination. `SUB` stores the result; `CMP` **does not**: its sole purpose is to set the flags. Use it to find out whether two values are equal (the result is zero), whether a result is negative or positive, and whether one value is greater or less than another.

- **Z = 1** when all bits of the result are zero. `BEQ` branches if Z = 1; `BNE` branches if Z = 0.
- **N** equals the most significant bit of the result (bit 7, 15 or 31 for byte, word, long). `BMI` branches if N = 1; `BPL` if N = 0.
- Customary: put the test instruction **immediately before** its branch, because most instructions change the flags.
- **Trick question:** `MOVEA` does **not** affect the condition codes, so `MOVEA.W #0,A3` then `BNE` does not see a zero result. Always check the Programmer's Reference Manual for how each instruction affects the flags.

Bit test with a mask. To test bit 9 of D6, put a mask `%0000001000000000` in D0 and `AND.W D0,D6`; then `BNE` is taken if the bit was 1.

Worked example

D0 = 3, D1 = 5. After `CMP.L D0,D1` *does* `BEQ` *branch?*

1. CMP computes D1 - D0 = 5 - 3 = 2, not zero.
2. Z = 0, so `BEQ equal` is **not** taken: execution falls into the `not_equal` code.
3. (If both were 3, Z = 1 and BEQ would branch.)

> **Key fact:** CMP src,dst subtracts without storing and sets the flags; put the branch right after. BEQ: Z=1. BNE: Z=0. BMI: N=1. BPL: N=0. MOVEA changes no flags. To test bit n, AND with a mask and test Z.

Signed and unsigned branches after CMP [68000]

Learn

`CMP D0,D1` computes D1 - D0. Whether D1 is "less than" D0 depends on whether the bytes are **signed or unsigned**, and each case has its own branch instruction:

Relationship	Signed	Unsigned
D1 < D0	**BLT**	**BCS** (branch on carry set)
D1 <= D0	BLE	BLS
D1 = D0	BEQ	BEQ
D1 <> D0	BNE	BNE
D1 > D0	**BGT**	**BHI**
D1 >= D0	BGE	**BCC** (branch on carry clear)

The same bit pattern can be both: `$FF` is 255 unsigned but -1 signed. With D0 = $FF and D1 = $01, unsigned says D1 < D0 and signed says D1 > D0. The assembler does not know which you meant, so you must choose the matching branch. (The course homework asks for both readings.)

Worked example

D0 = $80, D1 = $7F as bytes. After `CMP.B D0,D1`*, which branches are taken for D1 > D0?*

1. Unsigned: $7F = 127, $80 = 128, so D1 < D0: **BHI is not taken**.
2. Signed: $7F = +127, $80 = -128, so D1 > D0: **BGT is taken**.
3. Same bits, opposite answers.

> **Key fact:** D1<D0: BLT / BCS. D1<=D0: BLE / BLS. D1=D0: BEQ / BEQ. D1<>D0: BNE / BNE. D1>D0: BGT / BHI. D1>=D0: BGE / BCC. (Signed / unsigned.)

if, FOR and DO/WHILE in assembly [68000]

Learn

C++ and Java have built-in if/else, while, do/while, switch, for and function calls. In assembly you build them from **branch, jump and jump-to-subroutine** instructions. The rule: **pair an instruction that tests a condition or changes a variable with the proper branch**.

- **if**: `CMP.L D0,D1` then `BEQ equal`. The lecture's example loads 3 and 5; since they differ, `not_equal` executes.
- **FOR**: initialise the counter and the terminal value; `CMP` at the top; `BEQ` out when done; body; increment; `BRA` back.
- **DO/WHILE**: initialise; do the body (even a `JSR`) first; then `CMPI.B #test_value,(A2)+` and `BNE test_loop`. The body always runs at least once.

Both loop programs below are run on the core.

The array-maximum program: the array is null-terminated; `MOVE.B (A0)+,D1` reads a byte and sets the flags; a zero ends the loop; `CMP.B D0,D1` then `BLE` skips the replacement when the new byte is not larger. D0 ends holding 13 for `12,13,5,6,8,4,10,0`. (The lecture ends it with `STOP #$2700`; `SIMHALT` is used in the simulator.)

Worked example

Trace the array program for data 12, 13, 5, 0.

1. D0 = 0. Read 12: not zero; 12 > 0 so D0 = 12.
2. Read 13: 13 > 12 so D0 = 13.
3. Read 5: 5 <= 13 so `BLE` skips. Read 0: Z = 1, so `BEQ exit` ends with **D0 = 13**.

> **Key fact:** Pair a flag-setting instruction (CMP, or one that changes the variable) with a conditional branch. FOR/WHILE: test at the top, BRA back at the bottom. DO/WHILE: body first, test and branch back at the bottom (runs at least once). if: CMP, then branch around or to the matching code.

The stack, subroutines and parameter passing [68000]

Learn

Why a stack? It manages subroutine calls (functions in C, methods in Java) and variable scope. The 68000 manages it as a LIFO structure using **A7 (or A7')**, written **SP** in the EASy68K assembler. The stack grows from the top of RAM toward lower addresses, so **PUSH = pre-decrement** and **POP = post-increment**. Make sure SP is initialised, normally to the highest RAM address, or the program will crash.

Subroutines.

- `JSR` pushes the **long word** address of the next instruction and jumps; `RTS` pops it back into the PC.
- The programmer must make sure all pushes and pops line up, save and restore every register or memory location the subroutine uses, and decide how to **pass parameters**.
- `MOVEM <register list>,-(SP)` on entry and `MOVEM (SP)+,<register list>` on exit save and restore registers.
- Subroutines can **nest** (FOO calls BAR). They return to the instruction after the JSR (jump tables are an exception).

Guidelines from the lecture: set up the stack first; put subroutines **after** the main program; give each a **comment header** (name, what it does, registers used and saved, parameters in and out); put a **label** on the first line; save registers on entry, restore on exit.

The lecture's memory-test program calls `BSR do_test` (not JSR); the program above shows the same pattern running on the core.

Worked example

Trace `JSR double` *with SP = $00100000 in the program above.*

1. JSR pushes the 4-byte return address: SP = $00FFFFFC.
2. `MOVEM.L D1,-(SP)` pushes D1: SP = $00FFFFF8.
3. Do doubles to 10; `MOVEM.L (SP)+,D1` restores D1 (99) and SP = $00FFFFFC.
4. `RTS` pops the return address: SP = $00100000, and execution continues at `MOVE.L D0,D3`.

> **Key fact:** JSR pushes the 32-bit return address on the stack (SP goes down by 4) and jumps; RTS pops it into the PC. Initialise SP at the top of RAM. Save used registers with MOVEM.L list,-(SP) and restore with MOVEM.L (SP)+,list. Document name, purpose, registers and parameters in a header comment. Subroutines nest.

Z80 jumps, relative jumps, calls and returns [Z80]

Learn

Callback. The 68000 had `JMP`, `BRA/Bcc` and `JSR/RTS`. The Z80's versions (from the lecture):

Job	68000	Z80
Absolute jump	`JMP $0000ABCD` (16 or 32-bit address)	`JP 0ABCDH` (16-bit)
Conditional absolute jump	**does not exist**	`JP cc,nn` with cc = NZ, Z, NC, C, P, M
Relative unconditional	`BRA nnnn` (16 bits, +/-32K)	`JR nn` (8 bits, +/-128)
Relative conditional	`Bcc nnnn`	`JR cc,nn`
Call a subroutine	`JSR nnnn`	`CALL nn`
Conditional call	does not exist	`CALL cc,nn`
Return	`RTS`	`RET`
Conditional return	does not exist	`RET cc`
Return from interrupt	`RTE`	`RETI`

In both cases `JSR`/`CALL` places the address of the next instruction on the stack.

A **relative** jump or branch adds a **signed** displacement to the PC, so it can go backward or forward and is independent of where the code is loaded.

The program above runs `CALL DOUBLE` (A = 5 becomes 10) and `RET Z` (a conditional return: with A = 0 it returns immediately, so A stays 0).

Worked example

What does `ZERO_OR_SEVEN` return when A = 3?

1. `OR A` sets flags from A; A = 3 is non-zero, so Z = 0.
2. `RET Z` is **not** taken, so execution continues.
3. `LD A,7` then `RET`: A = 7. With A = 0, Z = 1 and the first `RET Z` returns with A = 0.

> **Key fact:** Absolute jump: JMP / JP (Z80 also has JP cc,nn; the 68000 does not). Relative: BRA nnnn (+/-32K) / JR nn (+/-128), conditional Bcc / JR cc. Call: JSR / CALL (Z80 also CALL cc). Return: RTS / RET (Z80 also RET cc). Interrupt return: RTE / RETI.

Z80 compares, conditions and loops [Z80]

Learn

Callback. The 68000 built its loops from `CMP` and `Bcc`. The Z80 has the same pairing with its own instructions:

- **Compare:** `CP D` computes A - D and sets the flags only. The accumulator is always the other operand.
- **Branch on flags:** `JR NZ,label`, `JR Z,label`, `JR NC,label`, `JR C,label` (and `JP cc` for absolute).
- **Counted loops:** `DJNZ label` decrements **B** and jumps while B is not zero, so a loop that starts with `LD B,9` runs 9 times.
- **Bit test:** `BIT n,r` sets Z when bit n of r is 0, and leaves the register unchanged (the 68000 version needs a mask, an AND and a register to hold the mask).

The FOR example (sum 1 to 9) and the bit test above run on the Z80 core. This Z80 material is our own translation of the lecture's 68000 examples.

Worked example

Trace the sum loop for the first two passes.

1. B = 9, C (sum) = 0, D = 1. Pass 1: A = C + D = 1; C = 1; D = 2; DJNZ: B = 8, not zero, jump back.
2. Pass 2: A = 1 + 2 = 3; C = 3; D = 3; B = 7.
3. After 9 passes B = 0, the loop ends and C = 45.

> **Key fact:** CP r (A - r, flags only); JR cc,nn / JP cc,nn on Z, NZ, C, NC, ...; DJNZ nn (decrement B, jump if not zero); BIT n,r (Z = complement of bit n). Arithmetic and compares involve A.

The Z80 stack, PUSH, POP, CALL and RET [Z80]

Learn

Callback. The 68000 stack used `-(SP)` and `(SP)+`. The Z80 gives the stack its **own instructions**:

- **SP** is a 16-bit register holding the top of the stack; the stack is LIFO and **grows toward lower memory**. You set it with `LD SP,0F000H`.
- `PUSH BC` stores the register pair on the stack (SP decreases by 2); `POP DE` takes it back.
- `CALL 2000H` pushes the current PC and loads 2000H; `RET` pops the return address into the PC.

The program above pushes 1234H, clears BC, pops it into DE, then makes a CALL and RET; afterwards DE = 1234H and SP = F000H again. The memory just below F000H still holds the last thing pushed: the CALL's return address 010EH (the address after the CALL), stored low byte first (0EH at EFFEH, 01H at EFFFH). The core shows exactly those bytes.

Worked example

Trace `PUSH BC` *with BC = 1234H and SP = F000H.*

1. SP is reduced by 2 to EFFEH.
2. The register pair BC (B = 12H, C = 34H) is written to the stack.
3. `POP DE` reads it back, DE = 1234H, and SP returns to F000H.

> **Key fact:** Both grow toward lower memory. Z80: SP is 16 bits; PUSH/POP move register pairs (2 bytes); CALL pushes the 16-bit PC; RET pops it. 68000: -(SP) and (SP)+ with any size; JSR pushes a 32-bit address; MOVEM saves a register list.

Logic and shift instructions

AND, OR, EOR and NOT with masks, ASL and the shift rules, and packing ASCII digits into a hex number.

AND, OR, EOR and NOT [68000]

Learn

The logical instructions apply their operation **bit by bit** to the size you give. **Only that size is touched**: `AND.W D0,D1` changes the low 16 bits of D1 and leaves its upper word alone.

- **AND** keeps a bit only where both are 1. Hex shortcuts: AND with **F** returns the digit, AND with **0** returns 0. That makes it the instruction for **masks** (`ANDI.B #$0F,D2` keeps the low nibble).
- **OR** sets bits; **EOR** (exclusive OR) flips bits where the mask is 1.
- Each has an immediate form: **ANDI, ORI, EORI**.
- **NOT** complements its single operand (`NOT.B D3` on $FF4567FF gives $FF456700). It has **no immediate form**.

Worked example

D0 = $3795AC5F, D1 = $B6D34B9D. Find D1 after AND.W D0,D1.

1. Only the low words matter: $AC5F AND $4B9D.
2. In binary: 1010 1100 0101 1111 AND 0100 1011 1001 1101 = 0000 1000 0001 1101 = **$081D**.
3. The upper word $B6D3 is unchanged, so D1 = **$B6D3081D**.

> **Key fact:** Bit-wise on the given size (.B/.W/.L); other bits of the register are untouched. AND with F keeps a digit, with 0 clears it. ANDI/ORI/EORI take immediates; NOT takes one operand and has no immediate form. Example: AND.W D0,D1 with D0=$3795AC5F, D1=$B6D34B9D gives D1=$B6D3081D.

Shift instructions and converting ASCII digits to a number [68000]

Learn

ASL moves every bit of a byte, word or long word to the left. Each shift inserts a **0 at bit 0** and moves the **top bit into C and X** (the old C is discarded). `ASL.B #3,D0` with D0 = $AB005501 gives $AB005508 because only the low byte shifts.

Rules for the shift and rotate group:

- byte, word or long operations;
- the operand is a **data register** or a **memory location**;
- for a register, the instruction supplies the **count**: an immediate for small counts, otherwise **another data register**;
- for a memory operand, **one bit at a time and word size only**.

Application: ASCII to a number. Four ASCII characters such as $31 $41 $30 $30 ("1A00") become $1A00 in three steps per digit: isolate the byte (`ANDI.W #$00FF`), **strip the ASCII code** (subtract $30 for 0-9, $37 for A-F, $57 for a-f), then **shift into place and OR into the result** (shifts of 12, 8, 4 and 0 bits).

Worked example

Convert the ASCII characters '7' ($37) and 'B' ($42) into the two-digit value $7B.

1. '7': $37 - $30 = **$07**. 'B': $42 - $37 = **$0B**.
2. Shift the first digit left 4 places: $07 becomes **$70** (ASL.B #4).
3. OR in the second digit: $70 OR $0B = **$7B**.

Key fact: ASL.B #3 on $01 gives $08 (a 0 enters bit 0; the top bit goes to C and X). Counts above 8 must be in a data register; memory operands shift one bit, word size only. ASCII to hex: subtract $30 for 0-9, $37 for A-F, $57 for a-f; then ASL by 12, 8, 4, 0 and OR the digits together.

Buses and memory cells

Sharing a bus, chip enable and output enable, the SRAM cell, address space and alignment.

Sharing a bus: contention, open collector and tri-state

Learn

Busses simplify how data moves: many devices connect to the same data path. The problem: **how do you connect outputs together without a short circuit?** Two ordinary outputs, one at 5 V and one at 0 V, create **bus contention**, a short circuit with an indeterminate level.

Open collector (OC). The output is the uncommitted collector of an NPN transistor: either an open circuit or a short to ground.

- OC outputs can only **pull the wire low**; a **pull-up resistor** makes it high. Multiple LOW outputs cannot conflict.
- HIGH-to-LOW is fast (a low-impedance path to ground); **LOW-to-HIGH is slow** because of the RC time constant of the pull-up and the wire's capacitance, which limits system speed.
- The lecture's rise-time exercise: 1 kΩ pull-up, one OC output (10 pF), gate inputs (10 pF each) and a 10-inch trace (1 pF per inch). The 10%-90% rise time of an RC circuit is about **2.2 RC**.

Tri-state. Every bus device is a logic block plus a **bus interface unit** that can supply drive current for both transitions. The interface is a switch (a MOSFET in series with the output): close it to connect, open it to disconnect.

Output Enable	Input	Output
0	1	1
0	0	0
1	1	Hi-Z
1	0	Hi-Z

Only one device's output is enabled at a time, which is how memories and registers share a bus.

Worked example

A 1 kΩ pull-up and 40 pF of total load. What is the rise time?

1. RC = 1,000 ohm x 40 pF = 40 ns.
2. 10%-90% rise time = 2.2 x 40 ns = **88 ns**.
3. That is why OC buses are slow: a few tens of pF with a kilohm pull-up costs tens of ns per rising edge.

> **Key fact:** Not with plain outputs (contention = short circuit). Open collector: outputs only pull low, a pull-up resistor makes high; falling edges fast, rising edges slow (RC). Tri-state: each output has a switch (output enable); only one device is enabled, the others are Hi-Z.

~CE and ~OE: chip enable and output enable

Learn

- **Chip Select / Chip Enable (~CS or ~CE)**: a master control. If it is not LOW (active), the entire chip is off: all outputs tri-stated and clocked inputs inactive.

- **Output Enable (~OE)**: only places the chip's outputs into the tri-state condition.
- If **~CE is LOW and ~OE is HIGH**, the chip will accept new data but its output stays Hi-Z.
- All of these are **active low** (the ~ means low = true).

With tri-state outputs you can wire outputs straight to the bus, but the chip selects must be controlled through proper **addressing**: the address decoder makes exactly one ~CS low. (The lecture suggests deriving the truth table of a 2:4 decoder as an exercise: with two inputs, exactly one of four active-low outputs is low, which gives ~CS0 to ~CS3.)

Worked example

Four 32-bit registers share a 32-bit bus with a 2:4 decoder driving ~CS0 to ~CS3. The address bits are 10. Which register drives the bus (if ~OE is low)?

1. A 2:4 decoder with active-low outputs makes the output numbered by the input value low: input 10 binary = 2, so ~CS2 is low.
2. Only register 2 is enabled; the others stay Hi-Z.
3. No contention.

> **Key fact:** ~~CE/~~CS is the master control: if not low, the whole chip is off (outputs Hi-Z, clocked inputs inactive). ~OE only puts the outputs into or out of tri-state. A chip drives the bus only when ~CE is low and ~OE is low.

Active-low signals and negative logic

Learn

Many memory and processor signals are **active low**: the name carries a tilde (~CS, ~OE, ~WE, ~RD, ~AS) and the signal is true when the pin is **low**. All of the memory control signals in this course are active low.

Negative logic means treating low as true. Swapping the meaning of the two voltage levels swaps the function of a gate:

Physical gate	Positive logic	Negative logic
Output high only if any input is high	**OR**	**AND**
Output high only if all inputs are high	AND	OR

This is why the lecture notes that "OR gates become AND gates in negative logic": to make ~CS low when **both** an address-valid and a page-select condition are true, a single OR gate (of the active-low signals) does the job.

NOR is not the negative-logic OR. NOR is an OR **followed by an inversion**. Changing polarity changes AND into OR (and back); it does not add an inversion. A physical NOR becomes a NAND in negative logic, and a physical NAND becomes a NOR.

The lecture's BEE 271 reminder: a **2-to-4 decoder with active-low outputs** has exactly one output low, the one numbered by the input.

Worked example

A chip's ~CS must be low when ~ADDRVAL is low and the page-select line ~PAGE is low. Which gate?

1. In positive logic we would want ~CS asserted when both are asserted (AND of the assertions).
2. With low = true for all three signals, the assertion of ~CS needs both inputs low.
3. An **OR gate** outputs low only when both inputs are low, so one OR gate does it: OR in positive-logic terms is AND in negative logic.

> **Key fact:** ~NAME means true when low. In negative logic (low = true) the same gate swaps function: an OR gate acts as AND (De Morgan). NOR is OR plus an inverted output, not the negative-logic OR.

Memory cells: the flip-flop and the 6-transistor SRAM cell

Learn

A single-bit memory cell is a **D flip-flop core** (without set/reset) with a **tri-state buffer** on its output: data in/out (D, Q), a write clock (W) and an output enable (OE). A memory chip arranges many of these cells in rows; the memory decoding logic selects a row from address bits (A0, A1) together with CS, RD and WR.

Real **static RAM (SRAM)** is built around a **6-transistor (6T) cell**: two cross-coupled inverters, which hold a stable 0/1 state through feedback, plus two switch transistors. Each inverter is two transistors, each switch one: **6 transistors per cell**. Briefly grounding the input of one inverter flips the state; grounding the other flips it back, and the BIT and ~BIT lines do exactly this when written.

SRAM works well for fast, local storage such as cache, and has a simple interface to the processor.

Worked example

How many transistors does a 1-Kbit SRAM array need for its cells (ignoring decoders)?

1. 1 Kbit = 1,024 cells.
2. 6 transistors per cell = 1,024 x 6 = **6,144 transistors**.
3. A DRAM of the same size would use 1,024 (one per cell) plus capacitors.

> **Key fact:** Two cross-coupled inverters (2 transistors each) hold a stable state by feedback; two switch transistors let a BIT/~BIT line temporarily force the state: 6 transistors per cell (6T).

Address space, alignment and data path width

Learn

The **address space** is the amount of accessible memory: it can vary from 1 KB for a simple microcontroller to over 60 GB in high-performance processors. **The size of the address space does not mean you have that much memory.** Examples from the lecture:

Processor	Address lines	Locations
Simple microcontroller	10 (A0-A9)	1,024 (2^10)
Generic 8-bit	16 (A0-A15)	65,536 (2^16)
68000	24 (A0-A23)	16,777,216 (2^24)
i486	32 (A0-A31)	4,294,967,296 (2^32)
Ryzen	36 (A0-A35)	68,719,476,736

Memory is always counted in 8-bit bytes, so the address range is 0 to 2^n - 1. Wider values occupy several bytes and usually sit on aligned addresses:

- 16-bit **word**: even addresses ($0000, $0002, ... $FFFE).
- 32-bit **long**: multiples of 4 ($0000, $0004, ... $FFFC).
- 64-bit **double**: multiples of 8.

Data path width predicts performance and functionality: 4 or 8 bits for appliances and modems, 16 for industrial controllers and automotive, 32 for telecomm and laser printers, 64 for PCs and workstations. Internal and external paths can differ: the **MC68000 is 32-bit internal and 16-bit external**, the 80C188 is 16 internal and 8 external. An 8-bit processor can do everything a 64-bit one can, only slower; doubling the bus width increases performance by up to 4X as a rule of thumb.

Worked example

Is a word access at address $1003 aligned?

1. A word must be on an even address.
2. $1003 is odd, so it is **not aligned**.
3. Word accesses are expected on even boundaries, so the address is not valid for a word access in the lecture's convention (use a byte access, or align the data).

> **Key fact:** Address space = 2^n bytes for n address lines (capability, not installed memory). Memory is addressed in bytes. Words (16 bit) usually on even addresses, long words on multiples of 4, doubles on multiples of 8. Internal and external data paths can differ (68000: 32 internal, 16 external).

Memory decoding

From address lines to pages: how many chips, which address bits go where, and the logic that picks one page.

Address lines and memory size

Learn

Memory is a long row of numbered locations. The **address** is the number of the location the processor wants. Each **address line** carries one bit of that number, so **n lines can name 2^n different locations**.

Address lines	Locations	Size	Highest address
10	1,024	1 KB	$3FF
16	65,536	64 KB	$FFFF
20	1,048,576	1 MB	$FFFFF
24	16,777,216	16 MB	$FFFFFF

Each added line **doubles** the memory the processor can reach. Hex is handy here because one hex digit is exactly 4 bits, so the number of hex digits in an address is the number of lines divided by 4.

Worked example

How many bytes can a 20-line address bus reach, and what is the highest address?

1. Locations = 2^20. Since 2^10 = 1,024 (1K), 2^20 = 1K x 1K = **1,048,576 bytes = 1 MB**.
2. The highest address is all 20 bits set to 1.
3. 20 bits = 5 hex digits, so the highest address is **$FFFFF**, and the lowest is $00000.

> **Key fact:** 2^n. For example 20 lines reach 2^20 = 1,048,576 locations (1 MB of bytes).

Memory chip organisation

Learn

A memory chip is described by two numbers: **how many locations** it has (depth) and **how many bits each location holds** (width). A **256K x 4** chip has 256K locations of 4 bits, which is 1 Mbit.

The processor's data bus is wider than most chips. A chip only supplies its own width, so chips are placed **side by side** and all selected together; each supplies a slice of the data bus. That group of chips, all answering the same address, is one **page**.

> **Chips per page = data bus width / chip width**

Different chips can build the same memory. For 1 MB on an 8-bit bus the lecture compares four chips: 1M x 1, 512K x 2, 256K x 4 and 128K x 8. All of them use the **same total number of chips** (8) because the total number of bits is the same. Only the number of pages changes (1, 2, 4 and 8), and so does how many chips sit in each.

Worked example

An 8-bit processor uses 256K x 4 chips. How many chips are in one page, and how many in 1 MB?

1. Each chip supplies 4 bits, the bus needs 8, so a page needs 8 / 4 = **2 chips**.
2. A 256K x 4 chip holds 256K x 4 bits = 128 KB. 1 MB / 128 KB = 8 chips.
3. 8 chips / 2 per page = 4 pages. This matches the lecture's table: 256K x 4 gives 4 pages.

> **Key fact:** N / w. Each chip supplies w of the N data bits, so a page needs N / w chips side by side.

Pages, offset bits and page bits

Learn

A **page** is the block of addresses that one set of chips covers. The processor's address is split in two:

- **Offset bits** (the low lines, from A0 up): choose a location **inside** the page. They go straight to the address pins of every chip in the page.
- **Page bits** (the remaining high lines): choose **which page**. They go to the decoder logic, along with the processor's address-valid signal, to switch on exactly one page.

The number of offset bits is set by the chip depth: a chip with 2^k locations needs k offset bits. Everything left over is a page bit.

> **Offset bits = log2(chip depth) | Page bits = address lines - offset bits | Pages = 2^(page bits)**

The lecture's example: 16 MB of memory (24 address lines) in 32K pages. A 32K page needs 15 offset bits (2^15 = 32,768), so **A0-A14 are offset bits** and **A15-A23** (9 lines) are page bits: 2^9 = **512 pages**.

Worked example

A processor has a 20-bit address bus and uses 256K x 4 chips on an 8-bit data bus. Find the offset bits, page bits, pages and chips.

1. 256K = 2^18, so **18 offset bits**: A0-A17.
2. 20 - 18 = **2 page bits**: A18-A19.
3. 2^2 = **4 pages**.
4. Each page needs 8 / 4 = 2 chips, so 4 x 2 = **8 chips**.

This is the 256K x 4 row of the lecture's table: 4 pages, page bits A18-A19, offset A0-A17.

> **Key fact:** Offset = A0 to A(k-1), wired to the chips. Page = the remaining high lines, wired to the decoder. Pages = 2^(page bits).

Address range of each page

Learn

Pages sit one after another with no gaps, and each is the same size. So:

> **start of page n = n x page size | end of page n = start + page size - 1**

For the lecture's 20-bit example with four 256K pages, the whole space is $00000-$FFFFF, so each page covers 1/4 of it:

Page	Page bits (A19 A18)	Range
0	00	$00000 - $3FFFF
1	01	$40000 - $7FFFF
2	10	$80000 - $BFFFF
3	11	$C0000 - $FFFFF

Notice the page bits are the **top two bits of every address** in that page, and the offset bits run from all zeros (start) to all ones (end). That is why the end of a page always looks like the start of the next one minus 1.

Worked example

Memory on a 24-bit address bus is built in 64K pages. What is the range of page 5?

1. A page is 64K = 2^16 = $10000 addresses.
2. Start = 5 x $10000 = **$050000**.
3. End = $050000 + $10000 - 1 = **$05FFFF**.

> **Key fact:** Start = n x 2^k. End = start + 2^k - 1 (one before the next page).

Selecting a page with chip-select logic

Learn

A chip with its **chip select (~CS)** high ignores the bus. To select a page, ~CS must go **low** for every chip in that page and stay high for all others. It should go low only when **both** are true:

1. the processor says the **address is valid** (the lecture calls the signal ~ADDRVAL; the chips are disabled until the address is valid), and
2. the **page bits equal this page's number**.

A **NAND** gate does this naturally. Its output is low only when **all** inputs are 1. So wire in every page bit and the address-valid signal, and put an **inverter** in front of each page bit that must be **0** for this page.

The lecture's example is the page at **$1A4**. In binary that is 1 1010 0100, which spread over A23-A15 is:

A23	A22	A21	A20	A19	A18	A17	A16	A15
1	1	0	1	0	0	1	0	0

The zeros (A21, A19, A18, A16, A15) get inverters; the ones (A23, A22, A20, A17) go straight in.

A decoder chip does the same job for **all** pages at once: with 2 page bits, a 2-to-4 decoder with active-low outputs drives one ~CS line per page.

Worked example

Design the NAND select for page 5 when there are 4 page bits.

1. 5 in 4-bit binary is **0101**.
2. Bits that must be 1: the second and last. They go straight in.
3. Bits that must be 0: the first and third. They get **2 inverters**.
4. Add address-valid as one more input. The output goes to ~CS of every chip in page 5.

> **Key fact:** NAND output is low only if every input is 1. Feed it the page bits plus address-valid; invert every page bit that must be 0 for that page.

The 68000's memory map [68000]

Learn

Everything you just learned applies unchanged to the 68000. Its address space is **24 bits**, **16 MB** ($000000-$FFFFFF), which is exactly the lecture's example: 16 MB in **32K pages** gives **A0-A14 as offset bits** and **A15-A23 as page bits**, 2^9 = **512 pages**.

The 68000's data bus is 16 bits wide, so a page is built from chips that together supply 16 bits (for example two 8-bit-wide chips side by side).

Worked example

Where does a page at $FF8000-$FFFFFF sit in a 24-bit space with 32K pages?

1. 32K = $8000. The last page starts at $1000000 - $8000 = **$FF8000**.
2. That is page number $1FF (all nine page bits A15-A23 equal 1).
3. The page's NAND needs no inverters at all: every page bit must be 1.

> **Key fact:** A0-A14 offset (15 bits), A15-A23 page (9 bits), 512 pages.

The Z80's memory map and paging [Z80]

Learn

In the 68000 lesson you split a 24-bit address into offset and page bits. The Z80 has the same idea but only **16 address lines**, so its whole address space is 2^16 = **64 KB** ($0000-$FFFF). A 64 KB memory needs no page bits at all if you use one 64K x 8 chip.

When a Z80 system needs **more** memory than 64 KB, it uses **hardware paging**: the processor writes a **page number** to a special memory or I/O location (a register). That register's output drives the page decoder, which switches one of several 64 KB chips onto the bus.

The lecture's example reaches **1 MB** with **16 pages of 64 KB**. A value from 0000 to 1111 in the register (a 4-to-16 decoder) selects the current page. Hardware paging is **not** virtual memory.

Code that switches pages lives at the **top** of each page: it writes the new page number, then jumps to the start of the next page.

Worked example

How many bits wide must the page register be to select among 16 pages?

1. 16 pages = 2^4, so the register needs **4 bits** (values 0000-1111).
2. A 4-to-16 decoder turns those 4 bits into 16 chip selects, one per 64 KB chip.
3. The Z80 still only drives A0-A15 itself, so every page uses those 16 lines as offset bits.

> **Key fact:** Paging: sixteen 64 KB pages; a value 0-15 written to a special register selects which page is switched in.

Memory systems and DRAM

Designing a memory system, byte lanes and endianness, the memory map, timing, SRAM versus DRAM and DRAM operation.

Designing a memory system step by step

Learn

The lecture's four steps:

1. **Divide the address space into pages**; the page size is set by the memory devices.
2. **Separate the low-order (offset) bits from the high-order (page) bits.**
3. **Decide which pages are present** and where they live.
4. **Design the decoding logic** to address each page uniquely.

Worked through in the lecture's example. The processor has a 32-bit data bus and (30 of) 32 address bits; the chips are 16-Mbit SRAMs organised **1M x 16**; the system needs **64 MB**, split equally between high and low memory, with the middle left for peripherals.

- **Step 1, page size.** A 32-bit bus needs 2 chips (16 bits each) per page. A 1M x 16 chip holds 2 MB, so a page is **4 MB**; 64 MB / 4 MB = **16 pages**.
- **Step 2, page and offset bits.** Each chip has 20 address bits (1M locations). The lecture counts the **offset as 20 bits of the chip address** and the **page as the other 10 bits** of the 30 processor address lines (the low two byte-address bits A0-A1 are not brought out for a 32-bit bus).
- **Step 3, page locations.** 8 pages in low memory: $00000000 to $01FFFFFF. 8 pages in high memory: $FE000000 to $FFFFFFFF. The pages are consecutive 4 MB blocks: page 0 = $00000000-$003FFFFF, page 1 = $00400000-$007FFFFF, page 2 = $00800000-$00BFFFFF, page 3 = $00C00000-$00FFFFFF, and so on.

Worked example

A 16-bit bus uses 512K x 8 chips. How big is a page, and how many pages are in 16 MB?

1. Chips per page = 16 / 8 = 2.
2. Each chip = 512K x 8 bits = 512 KB, so a page = 2 x 512 KB = 1 MB.
3. 16 MB / 1 MB = **16 pages**. (This matches the lecture's 16-bit, 24-bit-address design with 1 MB pages.)

> **Key fact:** 1. Divide the address space into pages sized by the chips (chips per page = bus width / chip width; page size = chips x chip size). 2. Split the address into offset (low) and page (high) bits. 3. Decide which pages are present and where. 4. Design the decoding logic so each page is addressed uniquely.

Bus width, byte write strobes and endianness

Learn

When the data bus is wider than a byte, each page is built from **several byte-wide chips side by side** (**lanes**), one per byte of the bus. Reading always fetches the whole bus width, but writing a single byte needs **separate write strobes**.

16-bit bus (24-bit address). A page holds 128 KB in two 64 KB chips. **A0 is implied** and synthesised from **WR0 and WR1**; the address lines to the chips start at A1. All 16-bit reads and writes appear on even addresses (00000, 00002, ...). **Byte writes** assert WR0 or WR1 alone; **word writes** assert both. This design is **big endian**: WR0 controls D8-D15, WR1 controls D0-D7; even-numbered bytes go to D8-D15 and odd-numbered bytes to D0-D7.

32-bit bus. A0 and A1 are implied, and WR0 to WR3 synthesise the byte address on writes: a byte write uses one strobe, a word write pairs them (WR0,WR1 and WR2,WR3) and a long word write asserts all four. **Byte operations may be at any address; words at addresses divisible by 2; long words at addresses divisible by 4.** All reads are 32 bits wide, and the operand is selected inside the processor. The lecture's organisation is **little endian**: WR0 to D0-D7, WR1 to D8-D15, WR2 to D16-D23, WR3 to D24-D31.

In the lecture's 16-bit design with a 24-bit address, the page/offset boundary does not line up with a hex digit boundary, so write the addresses in binary.

Worked example

A 16-bit big-endian design writes the single byte at address $00005 (odd). Which strobe and which lines?

1. Odd-numbered bytes go to **D0-D7**, controlled by **WR1**.
2. Only WR1 is asserted; WR0 stays inactive so the even byte at $00004 is untouched.
3. A word write to $00004 would assert both.

> **Key fact:** A page of N-bit data uses one chip per byte lane. Separate write strobes (WR0, WR1, ...) select which lanes are written: byte write = one strobe, word = a pair, long = all four. The low address bits (A0, A1) are implied by the strobes. Which byte lane gets which address decides big or little endian. Alignment: words on even addresses, longs on multiples of 4.

The memory map and decoding regions with a 3-to-8 decoder

Learn

The memory map of a typical 68K system has a ROM area containing the **exception vector table** (addresses $000000-$00001F, then program space from $000020), a region for **peripherals and empty space**, and RAM. The processor begins execution from an address stored in the vector table, and the **stack address is assigned there too**.

Interfacing to memory is usually the most involved part of the design, because it depends on the algorithms, performance, feature set and cost, which in turn drive memory density, speed, depth, width, type and the need for ROM reprogramming.

Address space decoding. The lecture's simple scheme predefines **8 regions of 2 MB** each, any of which can be RAM, ROM or I/O, using a **3:8 decoder**:

- The **74XX138** family has been a popular memory decoder for many years.
- Its three enables (EN1, ~~EN2, ~EN3) must all be true for the outputs to become active; the 68000's **address strobe (~~AS)** is true when the address is valid.
- A23, A22, A21 select one of 8 outputs, each a region of 2^21 = 2 MB.

Question from the lecture: with just a second 138 and another address bit, extend this to **16 regions**: let A20 decide which of the two decoders is enabled, so four bits (A23-A20) select one of 16 regions of 1 MB each.

Worked example

Which region does address $5A0000 fall in with the 8 x 2 MB decoder?

1. A23-A21 are the top three bits of the 24-bit address: $5A0000 = 0101 1010 0000 … so A23-A21 = 010.
2. 010 = region **2** (the third 2 MB region, $400000-$5FFFFF).
3. The decoder asserts output 2.

> **Key fact:** A 74x138 (3:8) with its enables (including address strobe ~AS) splits the space into 8 regions (8 x 2 MB for 16 MB); a second 138 plus one more address bit gives 16. The 68000 memory map has the exception vector table at address 0 (initial stack and start address), ROM, peripherals and empty space, then RAM.

Memory timing and wait states

Learn

Interfacing to memory means making the **processor's timing and the memory's timing compatible**. Memory has timing requirements, the processor has timing requirements, and **any decoding reduces timing margins**.

- **Z80 memory timing** is **synchronous** (no acknowledgement from memory is required). The memory system's **~WAIT** output extends the cycle for slow devices.
- Margins must be analysed as minimum, typical and maximum, which vary with temperature, processor speed, manufacturing lot and process, and PCB path lengths. Also add the **glue logic delays**, which depend on the logic family (LS, ALS, F, HC, HCT, FCT) and on loading. Example: a **74ALS138** decoder could reduce the available memory cycle time by as much as **36 ns**.

The lecture's procedure for margins:

1. Collect the data sheets for all parts.
2. Extract the timing information.
3. For a **read cycle**, compute the worst-case time for the address, ~OE and ~CE to reach memory and be stable.
4. Compute when the processor expects to read the data (de-assertion of signals is also delayed, which can work in your favour).
5. Compare with the memory's requirements.
6. If it fails, **add WAIT states** or **use a faster part**.

Worked example

A processor reads data at 120 ns after the address is output. The decoder adds 25 ns to ~CE and the SRAM needs 100 ns from ~CE low to data valid. Does it fit?

1. The memory sees ~CE after 25 ns and then needs 100 ns, so data is valid at 125 ns.
2. The processor samples at 120 ns: **5 ns too early**.
3. Fix: add one wait state, or use a faster memory or a faster decoder.

> **Key fact:** Collect the data sheets; compute worst-case times for address, ~OE and ~CE to reach and settle at the memory for a read; compute when the processor samples the data; compare with the memory's requirements (including decoder delays). If it fails, add wait states or use a faster memory.

SRAM versus DRAM

Learn

Up to now the lecture considered only **static RAM**: derived from the D flip-flop, using feedback to hold 0 or 1, with a simple processor interface. Real SRAM needs **5 or 6 transistors per bit**, which suits fast local storage such as **cache**.

Today's **high-density memory is DRAM**: **one transistor per cell** plus a small **capacitor**. The amount of charge in the capacitor says whether the bit is a 1 or 0. But **charge leaks out** and must be periodically **re-written (refreshed)**.

	SRAM	DRAM
Cell	flip-flop (feedback)	1 transistor + capacitor
Transistors per bit	5-6	1
Refresh	not needed	required (about every 10 ms)
Interface	simple	row/column addressing, RAS/CAS
Typical use	cache, fast local storage	main memory

Worked example

Compare the transistor count for 1 Mbit of each (cells only).

1. SRAM at 6T: 1,048,576 x 6 = 6,291,456 transistors.
2. DRAM at 1T: 1,048,576 transistors (plus capacitors).
3. DRAM is about **6 times** denser in transistors, which is why main memory is DRAM.

> **Key fact:** SRAM: feedback (flip-flop) cell, 5-6 transistors/bit, no refresh, simple interface, fast: used for cache. DRAM: 1 transistor + capacitor/bit, much denser and cheaper per bit, charge leaks so it must be refreshed (~every 10 ms), complex interface (row/column addressing).

DRAM organisation: rows, columns, RAS, CAS and refresh

Learn

Reading or writing a DRAM cell recharges it, but how do you read or write all the cells often enough (about every 10 ms)? The solution: **"sort of read" a group of cells at a time.**

The cells are organised as a **matrix**. The lecture's example is a **64-Mbit DRAM** as an **8192 x 8192** array: a 26-bit address made of a **13-bit row address** and a **13-bit column address**.

- The **row address** is clocked in first with the **Row Address Strobe (~RAS)**, followed by the **column address** and the **Column Address Strobe (~CAS)**.
- A memory cycle is therefore more complex: each may involve both RAS and CAS.
- **Refresh** is done by a **RAS cycle**: **8,192 cells are refreshed at a time**, and they cannot be accessed for data while it happens.
- To refresh all 8,192 rows every **16.384 ms**, a refresh cycle is needed **every 2 µs**: a significant perturbation to memory access.

The solutions are complex: large on-chip caches and SRAM secondary caches reduce how often the external DRAM bus is active; DRAM variants (EDO, SDRAM, DDR, DDR2, DDR3 and so on) improve efficiency but need more complex support logic. **Most PCs use ASIC chip sets** to manage memory interfaces, and many embedded processors have **built-in DRAM controllers**.

Worked example

A 256-Mbit DRAM is a square array. How many row and column bits, and what is the row refresh interval if 16,384 rows must be refreshed in 32 ms?

1. 256 Mbit = 2^28 cells; square array: **14 row bits and 14 column bits** (16,384 x 16,384).
2. Refresh interval = 32 ms / 16,384 = **1.95 µs** per row.
3. A smaller interval means more of the time is spent refreshing.

> **Key fact:** A square matrix of cells (a 64-Mbit DRAM is 8192 x 8192: 13 row + 13 column bits, multiplexed on the same pins). Row address with ~RAS first, then column address with ~CAS. Refresh by doing a RAS cycle (one row at a time): all rows within ~16 ms means a refresh every (total time / rows), e.g. 2 μs.

DRAM read and write sequence

Learn

Simplified **read** (lecture):

1. Apply the **row address** before ~RAS falls (setup tASR) and hold it after (tRAH).
2. ~RAS goes **low** and stays low (tRAS).
3. Apply the **column address** (tASC) and hold it after ~CAS falls (tCAH).
4. ~WE is **high** before (tRCS) and after (tRCH) ~CAS falls.
5. ~CAS goes **low** and stays low (tCAS).
6. ~OE goes low within its window (it may be tied low).
7. **Data appears** at the outputs, at a time set by when ~RAS (tRAC), ~CAS (tCAC) and ~OE (tOEA) fell and when the address was supplied (tAA).
8. ~CAS and ~RAS return to inactive (tCRP, tRP).

Simplified **write**: row address; ~~RAS low; column address; **~~WE low** for a time (tWP), timed from ~CAS falling (tWCS, tWCH); **data in** applied before ~CAS falls (tDS) and held (tDH); ~CAS low; then ~CAS and ~RAS return to inactive. The chip's on-board logic charges or discharges the cell capacitor depending on whether a 1 or 0 is stored.

You do not need to memorise every t-parameter: know the **order** and which signals are high or low.

Worked example

Which signal order distinguishes a DRAM write from a read?

1. Both send the row address with ~RAS and the column address with ~CAS.
2. In a read, ~WE stays **high** and ~OE goes low so the chip drives the data out.
3. In a write, ~WE goes **low** and data is applied to the pins before ~CAS falls.

> **Key fact:** Read: row address, ~RAS low, column address, ~WE high, ~CAS low, ~OE low, data valid, then ~CAS and ~RAS high. Write: row address, ~RAS low, column address, ~WE low, data in before ~CAS falls, ~CAS low, then ~CAS and ~RAS high. All with set-up and hold times.

Direct memory access and the bus handshake

Learn

The processor, memory and peripherals share the same buses, so a disk drive with data for memory competes with the processor for the only path. Two ways to move 512 bytes:

1. **Through the processor:** the processor asks for byte after byte, 512 request-and-reply exchanges.
2. **DMA:** the peripheral asks for the bus once and moves the whole block itself.

The **handshake**:

1. The peripheral asserts **BUS REQUEST (/BUSREQ)**.
2. The processor finishes its present instruction cycle (and any pending higher-level interrupt) and sends **BUS GRANT**, tri-stating its memory interface.
3. The peripheral writes or reads memory directly, in one burst.
4. When the peripheral drops /BUSREQ, the processor takes the bus back.

While it waits, the processor **idles or continues from its internal cache**, so with large on-chip caches it loses almost nothing.

Worked example

Count the messages the processor and a disk exchange to move a 512-byte block in each of the reading's two scenarios (a calculation from the scenarios, not a figure from the reading).

1. Through the processor: every byte needs a request ("gimme the next one") and a reply ("here it is"), so about 2 x 512 = **1,024** messages.
2. With DMA: one **bus request**, one **bus grant** and one **bus acknowledge** set it up, the disk writes all 512 bytes in a burst, and it signals that it is off the bus: **a handful of messages**.
3. The saving grows with the block size, and during the burst the processor can run from its cache.

> **Key fact:** Peripheral asserts BUS REQUEST; the processor finishes its current cycle (no higher interrupt pending), tri-states its bus and sends BUS GRANT; the peripheral moves the block directly to or from memory; when BUSREQ goes away the processor takes the bus back.

Interrupts and I/O ports

Interrupts versus polling, priorities, NMI, RESET and exceptions, the 68000 vector table, I/O ports, the data direction register and a polled UART.

Interrupts versus polling

Learn

The real world does not run at the processor's clock rate. Its events are **asynchronous**: a key press, a byte arriving at a serial port or a sensor trip can come at any moment and may be lost if not serviced in time. The processor has two ways to notice them.

Interrupts. The event raises an interrupt. The processor stops what it is doing, runs an **interrupt service routine (ISR)**, and then **returns to where it left off**. Think of a phone ringing during dinner: you answer, then go back to eating.

Polling. There is no ringer, so you pick up the phone every few seconds to see if anyone is there. In a computer the program **checks each device in a loop** (a *polling loop*). That is fine when the job is naturally a loop of checks, such as an alarm controller visiting each sensor, but the processor then does almost nothing else.

The cost is easy to see with a serial port. At 50K baud, 10 bits per character gives about **5,000 characters per second**. A 1 GHz processor spends **200,000 clock pulses** per character, which is about **100,000 instructions** at one instruction per two clocks, and a polling loop spends all of them asking "is it ready yet?".

Worked example

A UART delivers 1,200 characters per second. A 20 MHz processor needs 4 clock pulses per instruction on average. How many instructions pass between characters if the program only polls?

1. Time per character = 1/1,200 s.
2. Clock pulses per character = 20,000,000 / 1,200 = **16,667**.
3. Instructions = 16,667 / 4 = **about 4,167**.

All of those are spent in the polling loop. With an interrupt, the same time is available for other work.

> **Key fact:** Interrupt: an asynchronous event forces the processor to run an ISR, then resume. Polling: the program checks each device in a loop, so the processor does little else (waiting for a 5,000 characters-per-second UART at 1 GHz wastes about 100,000 instructions per character).

Interrupt priority, NMI, RESET and exceptions

Learn

Interrupts are asynchronous, so they can arrive while another **ISR** is running. To decide who wins they are given **priorities**: a more important interrupt **preempts** a less important one, and a lower-priority interrupt **waits**. What counts as important depends on the system: a fast data stream you must not drop, a battery monitor that must save the machine's state, or protection of human life.

- **NMI** is the interrupt that **cannot be masked**. It is reserved for catastrophic events, such as power loss or a memory failure, and execution can return to the interrupted code after it is served. On the

68K the lowest interrupt priority is **1** and the NMI is **7**.
- **RESET** is "a very dramatic interrupt": it clears registers, sets a known state and starts again from a known location. It does not return. The RESET line has to be held for 50 to several hundred clock cycles so the processor's state machine reaches the right state.
- **Exceptions** are generated by the **program**: a memory access error, an illegal instruction, divide by zero. They are handled like interrupts but **cannot be masked**.
- A real-time operating system always lets a ready **higher-priority task preempt** a lower one. Its hazard is **priority inversion**, where a low-priority task keeps a resource that a blocked high-priority task needs.

Worked example

Interrupt A has priority 3 and B has priority 5. A's ISR is running when B arrives, and later C (priority 2) arrives while B's ISR runs. In what order do the routines finish?

1. B (5) outranks A (3), so B **preempts** A.
2. C (2) is lower than B, so C **waits**.
3. B finishes. Then A (3) resumes ahead of C (2), because the processor takes the highest pending priority.
4. A finishes, then C runs.

Order of completion: **B, A, C**.

> **Key fact:** Higher-priority interrupts preempt lower ones; lower ones wait. NMI cannot be masked (reserved for power loss or memory failure; 68K level 7, lowest is level 1). RESET restarts from a known state and does not return. Exceptions come from the program (divide by zero, illegal instruction) and cannot be masked.

I/O ports, the data direction register and a polled UART

Learn

A D-type flip-flop or latch lets the fast processor bus and the slow outside world meet. When it faces the outside world we call it an **I/O port**. The 68K and ARM treat ports as part of **memory space** (memory-mapped I/O), so one instruction set serves both; the x86 uses a separate I/O space with its own instructions.

An 8-bit port may sit at an **even** address, with a **data direction register (DDR)** at the next odd address. Writing a 1 to a DDR bit makes that line an **output**; a 0 makes it an **input**. Writing $FF gives an 8-bit output port.

A port is **not memory**. What you wrote to the output side is not necessarily what you read from the input side, and the input side can change at any time. A C++ pointer to a port must be declared **volatile** so the compiler does not optimise the accesses away.

Polling a UART. The status bits tell you when to act: **DR = 1** means a received character can be read; **TB = 0** means the transmitter is free. Both loops mask the status bit and branch back until it is right (`ANDI.W #$0001,D0 / BEQ` for DR, `ANDI.W #$0002,D0 / BNE` for TB).

Worked example

A 16-bit UART word has status in DB0-DB7 and data in DB8-DB15. The word read is $4101. Is a character ready, and what is it?

1. DB0 = bit 0 of $01 = **1**, so DR = 1: a character is ready.

2. The data byte is DB8-DB15 = **$41**, the ASCII letter A.
3. Reading the data clears DR, so the next poll will wait for a new character.

> **Key fact:** A port is a latch that interfaces to the outside world. A DDR bit of 1 = output, 0 = input. What you write is not necessarily what you read; the input side can change on its own (volatile in C). UART: test DR before reading, test TB = 0 before writing.

68000 exception vectors and interrupt response [68000]

Learn

The 68000 keeps a **vector table** in the lowest 1,024 bytes of memory: 256 long words, each the **address** of a service routine. A user program therefore starts at $000400 or above.

On an interrupt (for example the NMI, whose vector is at **$00007C**) the processor does this:

1. completes the current instruction;
2. saves the **status register** on the stack;
3. saves the **address of the next instruction** on the stack;
4. switches to **supervisor mode**;
5. fetches the 32-bit vector from its table address;
6. starts executing the ISR at that address.

The last step is **indirect addressing**: the table location holds a pointer. The first two vectors are special. After RESET the 68K loads the **stack pointer from $000000** and the **program counter from $000004**.

Worked example

The long word at $000064 is $00002400. What does it mean, and which address does the vector table entry occupy for the 100th vector (counting the first as vector 0)?

1. The entry at $000064 is vector number $64 / 4 = **25**, and it says the service routine begins at **$00002400**.
2. Vector 100 sits at 100 x 4 = 400 = **$000190**.

> **Key fact:** $000000-$0003FF = 256 long-word vectors, each the address of its ISR. RESET: SP from $000000, PC from $000004. Interrupt: finish instruction, push SR and next PC, enter supervisor mode, fetch the vector (NMI at $00007C) and jump to the ISR.

State machines

The S-R latch, characteristic tables and state diagrams, finite state acceptors, and Moore and Mealy machines.

The S-R latch, characteristic tables and state diagrams

Learn

A truth table models a **combinational** circuit: change an input and the output follows. A **sequential** circuit has memory, so its output depends on the inputs *and* on its **state**. The simplest example is the **S-R latch**: a pulse on S sets Q to 1, a pulse on R resets Q to 0, and with both inputs 0 it holds. **S = R = 1 is not allowed**; the behaviour is undefined.

Two ways to describe it:

- A **characteristic table** is a truth table in which the present state Q is one of the inputs and the next state **Q+** is the output. It grows large when there are many states or inputs.
- A **state transition diagram** is a directed graph: **nodes are states, directed edges are transitions** labelled by the inputs (S, R) that cause them. Inputs that cause no change are edges that loop back to the same state.

Q	S	R	Q+
0	0	0	0
0	0	1	0
0	1	0	1
1	0	0	1
1	0	1	0
1	1	0	1

(The two rows with S = R = 1 are undefined.)

Worked example

Start at Q = 0 and apply the input sequence (S,R) = 10, 00, 01, 00. What is Q after each input?

1. 10 sets the latch: **Q = 1**.
2. 00 holds: **Q = 1**.
3. 01 resets: **Q = 0**.
4. 00 holds: **Q = 0**.

The same input 00 gave different outputs (1, then 0) because the state differed, which is why a plain truth table cannot describe the latch.

> **Key fact:** By its characteristic table (present state Q and inputs S, R give next state Q+; S=R=1 is undefined) or a state transition diagram (nodes = states 0 and 1; edges labelled with the S,R inputs, including edges back to the same state).

Finite state machines and the vending-machine acceptor

Learn

A **state machine** takes a sequence of inputs; each input produces a new state (from the input and the previous state) and an output. It is a **finite state machine (FSM)** when the states, inputs and outputs can all be listed. Traffic signals and vending machines are typical.

A formal definition is M = (Q, V, t, q0): a finite set of states Q, an input alphabet V, a transition function t and a start state q0. Adding **final states** gives a **finite state acceptor**: a final state that outputs true is an **accepting state**.

Vending machine example (10-cent item). Start state S0 (no money). A dime moves straight to the *unlocked* accepting state. A nickel moves to S1 (5 cents remembered); a second nickel then reaches *unlocked*. The model leaves out details (pennies, the unlocking hardware, returning to the start), which a real design would need.

In hardware, a **state register** stores the state and combinational logic computes the next state. A diagram with **2^n nodes needs an n-bit state register**.

Worked example

Change the price to 15 cents, still accepting only nickels and dimes. How many states are needed, and how many state-register bits?

1. Amounts that can be held before unlocking: 0c, 5c, 10c. Together with the *unlocked* state that is **4 states**.
2. 4 = 2^2, so a **2-bit** state register is enough.

> **Key fact:** A finite state machine with final (accepting) states that output true. Vending machine for a 10c item: S0 --nickel--> S1 --nickel--> unlocked; S0 --dime--> unlocked. A machine with 2^n states needs an n-bit state register.

Moore and Mealy machines

Learn

Both models store the state in a **state register** that changes on the clock, and both compute the next state with combinational logic from the inputs and the present state. They differ in how the **output** is made.

- **Moore:** output = f(state). The output logic sees only the state register, so outputs change **only on clock signals**. In the diagram the outputs sit on the **nodes**.
- **Mealy:** output = f(state, input). The outputs sit on the **edges** and can change **independently of the clock** whenever an input changes.

Because a Mealy output can depend on the transition, **some functions need fewer states**; the price is that outputs can change at any time, which can cause glitches.

Worked example

A detector must output 1 while its input has just been 1 two cycles in a row. Which model needs fewer states?

1. **Mealy:** two states ("last input 0" and "last input 1"). The output is 1 on the edge taken when the state is "last input 1" and the input is 1.
2. **Moore:** the output must come from the state alone, so you also need a state meaning "saw 1, 1": three states.

The Mealy version needs one fewer state, but its output can change in the middle of a clock period if the input does.

> **Key fact:** Moore: output = function of the state only (outputs on nodes, change only with the clock). Mealy: output = function of state and input (outputs on edges, can change without the clock); can need fewer states but outputs can change at any time.

Diodes and transistors as switches

A practical review: diodes, the BJT as a switch, base resistors, saturation, open-collector drivers, Darlington pairs, MOSFETs and PNP circuits.

Diodes: rectifier, Zener and LED

Learn

The lecture gives a **no-theory, just-circuits** review of diodes and transistors, because technical interviews test practical circuit skills.

- **Rectifier (standard) diode**: controls the direction of current flow; conducts only anode to cathode, once the anode is about **600 mV** above the cathode. Consider the forward current rating (power dissipated in the diode) and the reverse breakdown voltage. A resistor limits the current (1 kΩ in the figure).
- **Zener diode**: uses **reverse breakdown (avalanche mode)**: when the reverse voltage reaches a set value it suddenly conducts. Zener-based references can be stable to parts per million; real reference parts contain more than the Zener (such as temperature compensation). It needs a series resistor.
- **LED**: emits light when forward current flows; different semiconductor properties give different colours. **Forward voltage is typically 1-4 V**; light output depends on forward current; power dissipated depends on forward voltage times current. LEDs are long-lived and efficient.

LED resistor: R = (V_supply - V_f) / I_f.

Worked example

A red LED (Vf = 2 V) should run at 10 mA from a 5 V pin. Resistor?

1. Drop across the resistor = 5 - 2 = 3 V.
2. R = 3 V / 10 mA = **300 Ω**.
3. Use the nearest standard value (330 Ω gives about 9 mA).

> **Key fact:** Rectifier: passes current anode to cathode when the anode is about 0.6 V higher; check forward current rating and reverse breakdown voltage. Zener: conducts in reverse at a set voltage, a stable reference (needs a series resistor). LED: emits light with forward current; forward voltage 1-4 V; brightness set by current; R = (Vsupply - Vf) / I.

The BJT as a current-controlled switch

Learn

A microcontroller pin cannot drive big loads, so we use a **transistor**. There are two families, each with two polarities: **bipolar junction transistors (BJT)**, NPN or PNP, and **field-effect transistors (FET)**, N-type or P-type (the lecture ignores the junction-FET versus MOSFET distinction; N- and P-type MOSFETs together make CMOS logic).

Two modes: *linear* (output proportional to input: amplifiers) and *switching* (cutoff or saturation). Digital systems almost always use the switching mode.

The NPN BJT is a current-controlled current source. Current injected into the base flows through the emitter to ground; the base-emitter junction looks like a forward-biased diode with **Vbe about 0.6 V**.

The collector current is **IC = β x IB**, where β (also the forward current transfer ratio **hFE**) can range from 10 to 10,000 or more, and **IE = IC + IB**.

The control source is almost always a **voltage output from an I/O port** (3.3-5 V at 1-10 mA, sometimes more), which must exceed Vbe. **Never apply a voltage to the base of a BJT without a current-limiting resistor.**

Worked example

A transistor has β = 100 and a base current of 440 μA. What is the collector current?

1. IC = β x IB = 100 x 440 μA = **44 mA**.
2. IE = IC + IB = 44.44 mA.
3. (This is the lecture's 5 V, 10 kΩ example: (5 - 0.6) / 10 kΩ = 440 μA.)

> **Key fact:** A current-controlled current source: IC = β x IB, IE = IC + IB, with Vbe about 0.6 V. β (hFE) ranges widely (10 to 10,000). Used in switching mode (cutoff or saturation) in digital systems so a 1-20 mA pin can control large loads.

Choosing the base resistor

Learn

The lecture's recipe: choose the resistance that supplies the correct base current in the **worst case**. With β specified as a range, **use the minimum**.

1. Required base current: **IB = IC / βmin**.
2. The resistor drops the rest of the I/O voltage: **R = (V_IO - 0.6 V) / IB**.
3. Choose a standard value (rounding down gives more margin).

Worked in the lecture: a 5 V pin, a 100 mA collector current and 75 <= β <= 200. IB = 100 mA / 75 = 1.3 mA, R = 4.4 V / 1.3 mA = 3.38 kΩ, so use **3.3 kΩ**.

Why the minimum? **Beta is not a constant.** For the 2N2222A it changes by a factor of 2 as collector current varies from 0.1 mA to 10 mA, and the min/max values at 150 mA differ by 3 times; it also varies with temperature and between devices of the same type. The safest design rule is to be conservative and assume the worst case.

Worked example

A 3.3 V pin must drive 50 mA; βmin = 100. Find R.

1. IB = 50 mA / 100 = 0.5 mA.
2. R = (3.3 - 0.6) / 0.5 mA = 2.7 V / 0.5 mA = **5.4 kΩ**.
3. Standard value: 4.7 kΩ (more margin) or 5.1 kΩ.

> **Key fact:** Ib = Ic / βmin. R = (V_IO - 0.6) / Ib. Use the minimum beta (worst case) and round the resistor down to a standard value so there is a little more base current. Never connect a base without a resistor.

Saturation: what the collector voltage does

Learn

Biasing for switching, with a 25 V supply and a 330 Ω collector resistor:

- **10 kΩ base resistor:** IB = (5 - 0.6) / 10 kΩ = **440 µA**; with β = 100, IC = 44 mA; V across 330 Ω = 14.5 V; **collector = 25 - 14.5 = 10.5 V** (not saturated).
- **1 kΩ base resistor:** IB = 4.4 mA; the formula says IC = 440 mA and 145 V across the resistor: impossible. Don't worry: we have merely driven the transistor into **saturation**. The collector voltage cannot go lower than slightly above ground (about **0.4 V**, Vce(sat)). The transistor is now a switch: **input 0 V gives collector 25 V; input 5 V gives collector 0.4 V.**

Design exercises from the lecture (25 V supply, 100 Ω load, β = 100):

- To make the collector **1 V**: ΔV = 24 V, i = 240 mA, so IB = 2.4 mA; the transistor dissipates 1 V x 240 mA = **240 mW**.
- With **IB = 3 mA**: the formula gives 30 V across the load and -5 V at the collector: fails the **sanity check**, so the transistor is saturated at about 0.4 V. Any base current above roughly **2.45 mA** saturates it.

Completed circuit: a 5 V, 3 mA pin and a **1.5 kΩ** resistor (4.4 V / 3 mA, rounded) switch the 100 Ω load.

Worked example

A 50 V supply, 500 Ω collector resistor, β = 150, and a 3.3 V I/O pin through a base resistor: what is the base current needed for saturation?

1. IC(sat) = (50 - 0.4) / 500 = 99.2 mA (about 100 mA; the lecture rounds to 50 mA for the stated spec).
2. IB(min) = 99.2 / 150 = **0.66 mA**.
3. Any more than that just saturates harder.

> **Key fact:** Compute IC wanted = β x IB. The load can pass at most (Vcc - 0.4) / RL. If wanted >= max, the transistor is saturated: Vc = 0.4 V (a closed switch). Otherwise Vc = Vcc - IC x RL. Saturation needs IB >= (Vcc - 0.4) / (RL x β).

Choosing the right transistor

Learn

Things to look for in a transistor data sheet: **linear or switching type; value of β** (a range depending on temperature and collector current); **maximum collector voltage** (transistor off); **maximum collector current**; **maximum power dissipation**; and the **package**.

- **Off:** the collector sits at the supply, so choose a high enough cut-off voltage. For a 100 V supply, consider a **200 V** maximum.
- **On:** the transistor dissipates about **0.4 V x I**. A **10 A** load needs a **20 A** transistor and the ability to dissipate **4 W**, which may mean a heat sink, a fan or both. (Contrast: an AMD Ryzen dissipates 105 W and needs a large heat sink.)
- **Rules of thumb:** a safe operating value is usually **half the maximum rating**; assume the **worst-case (minimum) β**.
- Switching speed matters only at higher speeds.

Summary of design: set the base current with a series resistor so that β x IB guarantees IC x RL > Vcc (the transistor saturates), and choose a transistor with the right type, current, power and speed.

Worked example

A 24 V motor draws 3 A. What ratings do you look for?

1. Voltage: at least 2 x 24 = 48 V collector cut-off (use 60 V or more).
2. Current: 2 x 3 = 6 A or more.
3. Power when on: 0.4 V x 3 A = 1.2 W, so check the package and maybe use a heat sink.

> **Key fact:** Type (switching), β range, max collector voltage (cut-off, with margin: 200 V for a 100 V supply), max current (20 A for a 10 A load), power (0.4 V x I when on; heat sink if needed), package and switching speed. Operate at about half of any maximum rating and assume the minimum β.

Open-collector outputs, drivers and sinking current

Learn

Sourcing and sinking. An output can **source** current (current flows from the output through the load to ground) or **sink** it (current flows into the output from the load and goes to ground in the output). The two capabilities differ: usually an output can **sink more than it can source**, and the output impedances are different. The current *source* supplies current to the circuit (the power supply is one); the current *sink* accepts the return current (the ground bus is one).

Open-collector (OC) gates. Take any digital gate and add an NPN transistor on the output (a TI 7406, for instance). The output only pulls low: it **sinks current**. It needs a **pull-up resistor** to a positive supply, which need not be the gate's own Vcc. When the transistor is off the output looks like a Thévenin equivalent with series resistance equal to the pull-up.

Open-collector drivers. High-current devices need wider, thicker PC-board traces on the board. **Solenoids and relays generate large inductive spikes and need snubber diodes**; other voltages may need local DC-DC converters; high voltages need care to protect the low-voltage circuitry. The **ULN2003** has **7 power drivers** in one package (with snubber diodes), **inputs compatible with 5 V logic**, **outputs of 500 mA that hold off 50 V**, and parallel outputs for more current. As an LED driver, the load is connected from the supply through the LED and resistor into the **collector**, so power is applied by *sinking*; brightness is controlled by a pulse stream instead of DC.

Worked example

A 12 V relay coil draws 80 mA. Can one ULN2003 output drive it, and what else is needed?

1. 80 mA is well under 500 mA, and 12 V is under 50 V, so yes.
2. The coil is inductive: the ULN2003's built-in snubber diode (connected to the supply) handles the turn-off spike.
3. The coil connects between the 12 V supply and the output; the input is driven by a 5 V logic pin (it sinks the coil current).

> **Key fact:** A logic gate with an NPN on the output: it only pulls low (sinks current). Needs a pull-up resistor to a positive voltage (can differ from the gate's Vcc). Power drivers such as the ULN2003 (7 drivers, 500 mA, 50 V, 5 V-logic inputs, with snubber diodes) switch relays, solenoids and LEDs; parallel outputs for more current.

Darlington and compound (Sziklai) pairs

Learn

One transistor may not be enough for large currents (a 2N3055 power transistor has β of only 20-70). A **pre-driver** transistor boosts the current into the driver: a **Darlington pair**, whose effective **$\beta = \beta_1 \times \beta_2$**.

With β(Q1) = 200 and β(Q2) = 50, β = 10,000.

Example from the lecture: β1 = 20 (2N3055) and β2 = 100 (2N2222), so β = 2,000, driving a 10 Ω load from 40 V:

- IC = 40 / 10 = **4 A**.
- IB = 4 / 2,000 = **2 mA**.
- The pair turns on at **1.2 V** (two base-emitter drops in series).
- With a 5 V pin: R = (5 - 1.2) / 2 mA = **1.9 kΩ**.

Compound feedback pair (also the **Sziklai pair**): one NPN and one PNP. Current gain is similar, but the **turn-on voltage is only 0.6 V**. Total emitter current = IB (1 + βQ1 + βQ2 x βQ1); since βQ2 x βQ1 is much bigger than 1 + βQ1, it is about IB x βQ2 x βQ1.

Worked example

Power rail 75 V, load 50 Ω, βQ1 = 200, βQ2 = 50, pin 5 V at 5 mA max (the lecture's question). Choose Rx.

1. IC = 75 / 50 = 1.5 A.
2. IB = 1.5 / (200 x 50) = 0.15 mA.
3. Rx = (5 - 0.6) / 0.15 mA = 29 kΩ (compound pair turn-on 0.6 V); 27 kΩ standard gives a little extra current.

> **Key fact:** Pre-driver transistor Q1 feeds the base of power transistor Q2: β = β1 x β2. Darlington (two NPN) turn-on 1.2 V; Sziklai/compound (NPN + PNP) turn-on 0.6 V. Ic = Vcc / RL; Ib = Ic / β; R = (V_IO - turn-on) / Ib.

MOSFETs as switches

Learn

The **MOSFET** (P-type or N-type) is a **voltage-controlled resistor** with almost infinite input resistance but significant input capacitance. VGS sets the channel resistance rDS, which can vary over about 10 orders of magnitude between fully on and fully off.

- **N-type:** drive it from an I/O pin as long as it is fully on at the pin's high voltage (5 V or 3.3 V). The **load is referenced to the positive rail**, not ground. NPN and N-type devices switch loads referenced to the supply rail.
- **P-type:** same behaviour with reversed polarities; it can drive a **ground-referenced load**, but VGS is referenced to the **positive rail**, so a transistor is needed to **level-shift the gate**. The lecture's circuit uses an NPN (rated for the rail voltage) and a **5 V Zener**: when the NPN turns on, the Zener conducts and VGS = -5 V, turning the MOSFET on. (The lecture's own comment: "It may not be optimal, but it should work".)
- **Logic-level power MOSFET data sheet:** operates from logic levels, minimum resistance **6 milliohms**, maximum current **100 A**, **100 W**; the microcontroller pin connects **directly** to the gate with no bias resistors. Downsides: **static sensitive**, possibly more expensive than a comparable NPN, and **gate capacitance** may limit speed.

Worked example

A MOSFET with rDS(on) = 6 mΩ carries 20 A. Power dissipated?

1. P = I^2 x R = 20^2 x 0.006 = **2.4 W**.

2. That is why MOSFETs run cool compared with a BJT at 0.4 V x 20 A = 8 W.
3. (Heat sinking is still worth checking.)

> **Key fact:** A MOSFET is a voltage-controlled resistor (gate-source voltage sets the channel resistance; gate draws almost no current). N-type: load to the positive rail, gate driven directly if fully on at the pin's high level. P-type: load to ground but gate referenced to the rail, so a transistor and Zener level-shift the gate. Logic-level power MOSFETs need no base resistor; they are static sensitive and have gate capacitance.

PNP transistors and ground-referenced loads

Learn

PNP transistors work with opposite polarity: current **enters the emitter** and exits through the base and collector. The voltage across the load is now **referenced to ground**, often an advantage over a pull-down from the positive rail.

To drive one from a microcontroller you must **sink current from the base to ground** (the opposite of an NPN). The pin cannot do that alone when the rail is high, so use an **NPN transistor or an open-collector gate** to deal with the high-voltage issue. The lecture's example: a **7406 OC gate**, PNP with β = 50, rail 30 V, load 60 Ω, asks for a good base resistor Rx.

Our own working: load current about 30 V / 60 Ω = 0.5 A (slightly less with saturation drop); base current = 0.5 A / 50 = **10 mA**; Rx drops about 30 - 0.6 (Veb) - 0.4 (gate low) = 29 V, so Rx = 29 V / 10 mA = **2.9 kΩ**, choose **2.7 kΩ** for margin (the 7406 must hold off 30 V and sink 10 mA).

Summary of transistors as switches. BJT: base-current controlled, current-gain β, saturate by injecting enough current. MOSFET: gate-voltage controlled, raise VGS until rDS is as low as it can go. NPN and N-type switch loads referenced to the rail; PNP and P-type switch ground-referenced loads but need a more complex drive.

Worked example

Which one would you use to switch a load whose other end is grounded: NPN or PNP?

1. A load already tied to ground needs the switch on the supply side (high side).
2. An NPN's emitter would sit above ground by the load voltage, so the drive would have to exceed the rail.
3. A **PNP** (emitter at the rail) is the natural fit, driven by an OC gate that pulls its base down.

> **Key fact:** Emitter to the positive rail, collector to a ground-referenced load. Current flows into the emitter and out of the base and collector; base current is i/β. Turn it on by sinking base current to ground through Rx with an open-collector gate or NPN that can withstand the rail voltage. Size Rx as (Vrail - 0.6 - Vol) / (Iload / β).

Data conversion

Sensors, comparators, flash, ramp, successive-approximation and dual-slope converters, D/A converters, sample and hold, multiplexers and aliasing.

The analog world and sensors

Learn

The real world is **analog**: the electrical signal provides an *analogy* of the physical phenomenon. Usually a physical quantity is converted to an **analog voltage** (sometimes a resistance or a current):

Sensor	Measures
Thermocouple	temperature
Resistance thermometer	temperature
Strain gauge	displacement or pressure
Microphone	sound level
Magnetic pickup	disk data
Photocell	light intensity

There are A/D and D/A converters even in your home PC (for example the sound hardware): they connect these analog signals to the digital system.

Worked example

A strain gauge changes its resistance as a beam bends. How does a number reach a microcontroller?

1. Condition: put the gauge in a bridge or divider so the resistance change becomes a small voltage.
2. Amplify it to the converter's input range.
3. Digitise it with an A/D converter and read the code.

> **Key fact:** Thermocouple and resistance thermometer: temperature. Strain gauge: displacement or pressure. Microphone: sound. Magnetic pickup: disk data. Photocell: light. Most are converted to an analog voltage first.

Comparators

Learn

The **comparator** combines analog and digital circuitry: basically an **op-amp running open-loop**, with conditioning on its output so that it is compatible with logic levels. It compares the voltage at its two inputs:

- Output **1 (TRUE)** if the voltage on the **+ input is more positive** than on the - input.
- Output **0 (FALSE)** if it is less positive.

One comparator is a 1-bit converter. **More comparators give higher resolution**: with thresholds at 0.25, 0.50 and 0.75 of VMAX, an input sweeping up from 0 to VMAX produces codes 0, 1, 2, 3 as it crosses each threshold (the lecture's waveform shows the code stepping 0, 1, 2, 3, 3, 3, 2, 2, 1, 0 as the unknown voltage rises and falls).

Worked example

Thresholds at 1, 2, 3 and 4 V (four comparators). What code for an input of 2.7 V?

1. The 1 V and 2 V comparators output 1 (2.7 > 1, 2.7 > 2).
2. The 3 V and 4 V comparators output 0.
3. Two comparators are 1, so the code is **2**.

> **Key fact:** Output = 1 if V+ > V-, else 0 (an open-loop op-amp with logic-level output). A bank of comparators with evenly spaced thresholds counts how many thresholds the input exceeds: that count is the digital code (more comparators, higher resolution).

Flash A/D converters

Learn

A **flash converter** puts a **resistor ladder** across the reference and a comparator at each tap. The lecture's 2-bit example has taps at 1/5, 2/5, 3/5 and 4/5 of Vr and the allowable input range is 0 to 4/5 Vr (an over-range output flags larger inputs); with Vr = 5 V it resolves 1 V.

An **N-bit** flash converter needs **2^N - 1 comparators**. The comparator outputs form a **thermometer (ladder) code**; a **ladder-to-binary encoder** finds the highest comparator that is 1 (detect Cn = 1 and Cn+1 = 0, so only one row of the truth table is true; without this the truth table over all 16 comparator outputs would need 65,535 terms).

Speed depends only on the comparators' and encoder's propagation delays: 1 ns + 1 ns = 2 ns per conversion, **500 MHz**. Commercial flash converters reach about **1 GSPS** (giga samples per second).

Cons: more power, expensive, limited to **8 to 10 bits**. (A slide-accuracy note: the lecture says an 8-bit flash needs '257 equally matched resistors and 256 comparators', but also gives the general rule 2^N - 1 = 255 comparators; the rule is the one to remember.)

Worked example

A flash converter's comparators take 0.8 ns and its encoder 1.2 ns. What is the conversion rate?

1. One conversion = 0.8 + 1.2 = 2.0 ns.
2. Rate = 1 / 2 ns = **500 MHz**.
3. How many comparators for 8 bits? 2^8 - 1 = 255.

> **Key fact:** 2^N - 1 comparators on a resistor ladder give a thermometer code (all 1s up to the input level); an encoder turns it into binary. Speed = comparator + logic delay (1 ns + 1 ns gives 500 MHz). Fastest (about 1 GSPS), but power-hungry, expensive, 8-10 bits.

D/A converters

Learn

A **D/A converter (DAC)** does the opposite of an A/D converter: a digital word in, an analog voltage out. Conceptually simple:

- Inject **current sources into a summing junction**: the total current is the sum of the currents entering.
- Arrange the currents in a **binary progression** (i, 2i, 4i, 8i, ... 128i), each controlled by one digital bit (an analog switch).
- A current-to-voltage converter (here a resistor, **V = i x R**) turns the sum into the output voltage.

The lecture's 4-bit example: i = 0.1 mA and R = 1 kΩ, so each step is **0.1 V** and the maximum (all bits on, 15 x 0.1 mA) is **1.5 V**.

DACs are also the heart of most A/D converters: single-ramp and successive-approximation converters need a **reference voltage, a comparator and a D/A converter**.

Worked example

Code 1010 (decimal 10) into the lecture's DAC (i = 0.1 mA, R = 1 kΩ). Output?

1. Bits set: 8i and 2i = 10i = 1.0 mA.
2. V = 1.0 mA x 1 kΩ = **1.0 V**.
3. Equivalent: 10 steps x 0.1 V.

> **Key fact:** Current sources i, 2i, 4i, 8i... are switched by the digital bits into a summing junction; a current-to-voltage converter (a resistor, V = I x R) gives the output: Vout = code x (i x R). Full scale = (2^N - 1) x i x R.

A/D conversion methods compared

Learn

Choosing a converter forces trade-offs: **accuracy and resolution** (which dictate the number of bits; *they are not synonymous*), **conversion speed**, **overall cost** and **noise immunity**. The lecture's methods:

Method	Character
Flash	Fastest, lowest resolution, greatest cost per bit
Single ramp	Simple, non-deterministic conversion time
Successive approximation	**Most widely used in the 12-18-bit range**
Multiple slope (dual, triple, quad)	Highest precision, 16-24 bits, **very slow**
Voltage to frequency	Medium accuracy, slowest, highest noise immunity

Delta-sigma is an advanced converter that combines the earlier techniques (think of it as a V/F converter with feedback).

Worked example

A data logger needs 16-bit accuracy on a slow thermocouple signal in a noisy factory. Which method?

1. Speed is not an issue (temperature changes slowly).
2. Accuracy (16 bits) and noise immunity matter.
3. A **dual-slope** (multi-slope) or a **V/F** converter fits; a flash converter would be wrong on cost, resolution and noise.

> **Key fact:** Flash: fastest, lowest resolution, most cost per bit. Single ramp: simple, non-deterministic time. Successive approximation: most widely used, 12-18 bits. Multiple slope (dual/triple/quad): highest precision, 16-24 bits, very slow. Voltage-to-frequency: medium accuracy, slowest, highest noise immunity.

Resolution, step size and using the full range

Learn

Resolution is the converter's ability to measure small differences; **accuracy** is how well the code represents the true input. The lecture's numbers: an **8-bit** converter divides its input range into **255** steps;

over 0 to 10 V that is 10/255 = **0.04 V** per step. A **16-bit** converter over the same range gives 10/65,535 = **0.00015 V** per step.

Typically you want to **amplify the signal** so it uses the converter's range. If the input is only 0.00015 V and you do not amplify it, a 16-bit converter gives you **1 bit** of resolution. (A programmable-gain amplifier does exactly this.)

Worked example

A sensor outputs 0 to 100 mV into a 12-bit converter spanning 0-5 V. How many codes does the signal use, and how can you do better?

1. Step = 5 V / 4,095 = 1.22 mV; 100 mV spans about 82 codes.
2. Amplify by 50 (to 5 V full scale).
3. Now the signal uses all 4,095 steps, and the resolution at the sensor is 100 mV / 4,095 = 0.024 mV.

> **Key fact:** Step = full-scale range / (2^N - 1), e.g. 10 V / 255 = 0.04 V for 8 bits, 10 V / 65,535 = 0.00015 V for 16 bits. Resolution is only useful if the signal fills the range: amplify small signals first.

Single-ramp and successive-approximation converters

Learn

Other than flash, a converter needs three elements: a **reference voltage, a comparator and a D/A converter**.

Single-ramp: a counter feeds a D/A whose output ramps up. The comparator tests whether the D/A output is greater than Vx; initialise COUNT to 0, read the TEST bit, and while it is not true increment COUNT. When TEST goes true, COUNT is the digitised voltage. Problem: the digitising time depends on the input (a large voltage needs a long count).

Successive approximation (SAR): still the **workhorse of data acquisition circuits**. It makes a binary search: set the MSB, compare, keep or clear it, then do the next bit. N bits take N steps. The National Semiconductor **ADC1241**: 12 bits plus sign, **13.8 µs** conversion, self-calibrating, with an internal **sample and hold**.

Worked example

A 10-bit SAR converter finds an input of 3.3 V on a 0-5 V range with a 1023-step scale. Trace the first two bits.

1. Trial 1: MSB set = half scale 2.5 V. 3.3 > 2.5, keep the bit.
2. Trial 2: next bit adds 1.25 V, giving 3.75 V. 3.3 < 3.75, clear the bit.
3. Continue 8 more times: ten comparisons total for ten bits.

> **Key fact:** Single ramp: a counter drives a D/A; count until the D/A output passes Vx; count = result; time depends on Vx (up to 2^N counts). Successive approximation: a binary search, one bit per step, MSB first, N comparisons; deterministic, the workhorse for 12-18 bits.

Voltage-to-frequency, delta-sigma and dual-slope converters

Learn

Voltage-to-frequency (V/F): the highest-resolution method (not necessarily the most accurate). A V/F converter outputs **Fout = k x Vin** (k = 100 kHz per volt: 5 V gives 500 kHz); counting pulses for 1 s gives 500,000, for 10 s 5,000,000, for 100 s 50,000,000. **The longer the digitising period, the greater the noise rejection**: it acts like a digital integrator, ideal for noisy industrial control, but it is **not useful for AC voltages**. The AD537 gives 0-10 or 0-100 kHz for 0-10 V with 0.01% linearity.

Delta-sigma is an advanced technique: think of it as a V/F converter with feedback (an integrator, comparators and a D flip-flop producing a bit stream).

Dual slope: based on V/F conversion. (1) Convert Vx to a frequency (say 1 MHz per volt); (2) count the cycles for a fixed window T (100 ms gives 506,382 counts, say); (3) switch to the accurate reference Vref (say 10.0000 V); (4) count down, one per pulse, recording time; (5) when the counter reaches zero, capture the time tref. The counts are equal, so the two areas are equal: **Vx x T = Vref x tref**, hence **Vx = Vref x tref / T**. Errors in one slope cancel in the other. Triple and quad slope refine it (about 10 measurements per second).

Worked example

Vref = 10 V. The fixed window T is 100 ms and the return time tref is 50 ms. What is Vx?

1. Vx x 100 ms = 10 V x 50 ms.
2. Vx = 500 / 100 = **5 V**.
3. Any error common to both slopes (clock drift, say) cancels in the ratio.

> **Key fact:** V/F: Fout = k x Vin; count pulses for a fixed time: counts = k x Vin x T (longer window, more noise rejection; no AC). Dual slope: count V/F pulses from Vx for time T, then count down using Vref for time tref; equal area: Vx x T = Vref x tref, so Vx = Vref x tref / T.

The data-conversion chain: conditioning, multiplexing, sample and hold

Learn

Most A/D systems need several elements before the actual conversion, either integrated into the microcontroller or assembled from discrete parts:

1. **Signal conditioning:** the unknown input must be inside a fixed range (0-5 V, 0-10 V, +/-10 V...). The sensor may give a different voltage range, a current or a resistance. A **programmable-gain amplifier** extends the sensor's range to maximise the converter's full-scale resolution.
2. **Analog multiplexing:** there may be several input sources (an automobile engine controller). Real-time systems may need thousands of conversions per second, so switching must be rapid and clean.
3. **Sample and hold (S/H):** time-varying signals change during the conversion. The lecture's example: the input changes at 1,000 V/s and a conversion takes 1 ms: the input changes by 1 V during the conversion. The S/H freezes it.

Worked example

A sensor produces 0 to 20 mV and the converter spans 0-2 V. Which block do you add?

1. The signal uses only 1% of the range.
2. Add signal conditioning: a gain of 100 (a programmable-gain amplifier).
3. Now 20 mV becomes 2 V, using the full range.

> **Key fact:** 1 Signal conditioning (to a fixed range, e.g. a programmable-gain amp); 2 analog multiplexing (many inputs, fast and clean switching); 3 sample and hold (freeze a time-varying input during digitisation); then the A/D converter. May be integrated in the microcontroller or built from discrete parts.

Analog switches and multiplexers

Learn

A mechanical switch is normally too slow for a computer system (a burglar alarm is an exception). An **analog switch** is a **FET** whose channel resistance is controlled by the gate voltage: it works like a mechanical switch but switches in under a microsecond, has **no bounce**, and works directly from logic levels.

Parameter	Mechanical	Analog
Switching speed	~10 ms	250 ns
Bounce	yes	no
On resistance	0 Ω	100 Ω
Off resistance	infinite	~1 GΩ
Lifetime	finite	infinite
Bandwidth	> 1 GHz	10-100 MHz
Current	1 A	30 mA

A **4-to-1 analog MUX** is four such switches selected by a **2:4 decoder** (select A0, A1). Commercial parts: the **MUX-08** (8-channel multiplexer) and the **AD684** (quad sample-and-hold).

Worked example

Which switch should connect a thermocouple amplifier (high-impedance input) to an ADC at 1,000 channels per second?

1. 1,000 per second is far too fast for a mechanical switch (10 ms each).
2. The amplifier input is high impedance, so the 100 Ω on-resistance causes negligible error.
3. Use an **analog multiplexer**.

> **Key fact:** Analog (FET): ~250 ns, no bounce, ~100 Ω on, ~1 GΩ off, unlimited life, 10-100 MHz, ~30 mA. Mechanical: ~10 ms, bounce, 0 Ω on, infinite off, finite life, >1 GHz, ~1 A. A MUX is a decoder driving analog switches.

Sample-and-hold and droop

Learn

A **sample-and-hold** stores the input as **charge on a capacitor** until the converter can digitise it. Switch SW1 samples (closed) and SW2 connects the output to the system (open); in hold, SW1 opens and SW2 closes. A special circuit design holds the voltage with minimal loss of charge, but you must still check the **droop rate** against the desired accuracy.

The lecture's worked problem: a **10-bit** A/D over **0 to 10.23 V**:

1. 1,023 counts, so each count is **10 mV**.
2. Half an LSB = **5 mV**.
3. At a droop of **1 V per millisecond**: 5 x 10^-3 V / 10^3 V/s = **5 x 10^-6 s = 5 µs**.

4. So the converter has 5 µs.
5. With Analog Devices' typical droop of **50 mV/s** the allowed time is far longer: 5 mV / 50 mV/s = **0.1 s = 100 ms**. (The slide states it as 100 microseconds, which would be a unit slip: the arithmetic gives 100 milliseconds.)

Worked example

LSB = 20 mV; droop = 2 V/ms. Conversion time limit?

1. Half LSB = 10 mV.
2. Time = 10 mV / (2 V/ms) = 0.01 V / 2,000 V/s = **5 µs**.
3. Halving the droop rate would double the time available.

> **Key fact:** The hold capacitor leaks, so the held voltage droops at some V/s. Conversion time must keep the droop below half an LSB: t < (LSB/2) / droop rate. Example: 10 mV LSB, 1 V/ms droop gives 5 μs; with 50 mV/s it is 100 ms (the slide says microseconds, a slip).

Aliasing, the Nyquist rate and bandwidth limiting

Learn

When digitising a time-varying voltage you must consider the signal's frequency against the converter's **sample rate**. A 10 kHz sine sampled every 100 µs (once per cycle) looks like a constant voltage whose level depends on the phase. To avoid **aliasing**, take **at least two samples per cycle** (the **Nyquist** criterion: sample rate at least **2x** the signal frequency).

Apparent frequency: a tone f sampled at fs appears at |f - fs x round(f/fs)|.

Limiting bandwidth: add a low-pass filter ahead of the converter. The lecture's 2-pole filter (R1 = R2 = 11.2 kΩ, C1 = 1000 pF, C2 = 2000 pF) has a 10 kHz cutoff and passes everything below about 5 kHz, enough for voice. Choose the upper frequency by how much accuracy you need: the Fourier components of a 220 Hz square wave are odd harmonics, and even with 6 terms (about 2.4 kHz; the slide says 2,427 Hz) the square wave is not perfect.

Worked example

A 7 kHz tone is sampled at 10 kHz. What do you see?

1. The Nyquist limit is 5 kHz; 7 kHz is above it.
2. Apparent frequency = |7 - 10 x round(0.7)| = |7 - 10| = **3 kHz**.
3. A 3 kHz tone appears that is not in the signal: aliasing.

> **Key fact:** Sampling a signal at less than twice its frequency gives a false lower-frequency signal (one sample per cycle looks constant). Sample at at least 2x the highest frequency (Nyquist), and put a low-pass filter before the converter to limit the bandwidth. A square wave needs many odd harmonics.

Choosing converter resolution for a sensor (furnace example)

Learn

A converter cannot tell apart analog values closer together than one **step**. For an N-bit converter over a range V there are 2^N codes and 2^N - 1 intervals, so **step = V / (2^N - 1)**. For 10 bits over 0 to 2.00 V that is 2.00 / 1,023 = **1.955 mV**. A value close to a switching point may digitise either way.

To decide whether that is good enough, work backwards from what must be detected:

1. **Amplify** the sensor so its range uses the converter's range (gain 20 turns 85 mV into 1.7 V and 0.02 mV per degree into 0.4 mV per degree).
2. Find the **smallest change** that matters (0.1 degree C = 0.04 mV).
3. Compare it with the step: a 2 mV step only sees changes of about 5 degrees, so the resolution must improve by **2 mV / 0.04 mV = 50 times**.
4. Each extra bit halves the step: 12 bits is 4x better, **16 bits is 64x better**, so 16 bits suffices (0.03 mV per step).

The reading's own figures for the thermocouple are not consistent with each other: 85 mV at 400 degrees would be about 0.21 mV per degree, not 0.02. The amplified numbers (1.7 V and 0.4 mV per degree) are the ones its resolution argument uses, so use those.

Worked example

A pressure sensor gives 0 to 5 V and must resolve changes of 2 mV. How many bits are needed?

1. Codes needed: 5 V / 0.002 V = **2,500** steps.
2. 2^11 = 2,048 is too few; 2^12 = 4,096 is enough.
3. A **12-bit** converter (step about 1.2 mV) will do.

> **Key fact:** Step = range / (2^N - 1). Find the sensor's change for the smallest detail you need, amplify so the signal fills the range, then choose N so one step is no larger than that change. Furnace: 2 V range, 0.4 mV/degree after gain 20, 0.1 degree = 0.04 mV needs a 50x finer step, so 16 bits (64x).

Noise, isolation and watchdogs

Hysteresis and debouncing, optical isolation and watchdog timers.

Hysteresis, Schmitt triggers and debouncing

Learn

When the two voltages at a comparator's input are nearly identical the output can become **unstable**. **Hysteresis** is designed in to reduce or eliminate spurious switching: the changing voltage must pass the reference by a margin before the output switches. The switching range is the **deadband** (V_high - V_low); it gives a noise margin but limits accuracy.

Switch bounce. Any mechanical device interfaced to a computer needs special circuitry: the contacts open and close electrically many times before the circuit actually opens or closes, and the circuit sees each bounce as an input. A **Schmitt trigger** (a gate with hysteresis) debounces mechanical switch inputs. In the lecture's RESET circuit, an **RC low-pass filter** debounces the switch and a **NAND gate with Schmitt-trigger inputs** converts the slow ramp into a proper logic edge; without hysteresis the output would oscillate as the slowly rising input approaches the threshold.

Worked example

A Schmitt trigger has thresholds 2.0 V and 3.0 V. A noisy signal wanders between 2.4 V and 2.7 V. How many times does the output toggle?

1. The output only switches high above 3.0 V and low below 2.0 V.
2. The signal never reaches either, so the output **does not toggle**.
3. A plain comparator at 2.5 V would toggle on every wiggle.

> **Key fact:** Hysteresis: separate rising and falling thresholds (the deadband = V_high - V_low) so noise near the threshold does not toggle the output. Debouncing: mechanical contacts bounce; use an RC low-pass filter followed by a Schmitt-trigger gate (or a latch) so the processor sees one clean edge.

Optical isolation

Learn

Embedded systems often work in **harsh environments**: electrical noise (an arc-welding machine) can overwhelm ordinary computer circuits, and **long cables act like antennas** that funnel electrical energy into the electronics. **Optical isolation** protects the computer circuitry: an **LED** and a **phototransistor** couple the outside world to the computer without a physical connection.

Benefits: communication between systems with **different ground references**, often **1,000 V** of isolation between ground references, and safety in **medical instruments** that touch the patient.

The lecture's problem: gate A (from a furnace thermostat, battery powered, biased +1,000 VDC above ground) must drive gate B (turns on a heater, ground referenced). A current amplifier on A's side sets the LED's on-current, and on B's side the phototransistor with a pull-up resistor makes the input to gate B.

Worked example

Why not connect gate A's output to gate B with a resistor?

1. A's output sits 1,000 V above earth ground.

2. A direct connection would put that potential on B's input and destroy it.
3. The optocoupler passes only the on/off information as light.

> **Key fact:** An LED and phototransistor couple a signal by light with no electrical connection: up to ~1000 V of isolation, different ground references, immunity to noise picked up by long cables (arc welding), patient safety in medical instruments.

Watchdog timers

Learn

Industrial and mission-critical embedded systems cannot be allowed to fail because of software faults. A **watchdog timer** monitors the operation of the computer and issues a **RESET or NMI** when:

- the power-supply voltage goes out of range, or
- the computer has not issued a **retrigger pulse** to the timer within the designated time.

It depends on the code **executing in a loop with a predictable cycle time** that writes the retrigger bit (an output port bit) each time around. It **guarantees that the processor will not lock up** and go offline whatever caused the malfunction.

The lecture asks whether to connect the output to **RESET or NMI**. The slides do not answer; our analysis is that RESET is more reliable because a malfunctioning processor may have corrupt state, so a software NMI handler cannot be trusted.

Worked example

The main loop takes 10 ms and kicks the watchdog every pass. The timeout is 100 ms. What happens if the code hangs in an endless loop?

1. No retrigger arrives.
2. After 100 ms the watchdog times out and asserts RESET.
3. The system restarts in a known state instead of staying dead.

> **Key fact:** A timer that must be retriggered (by an output bit from the main loop) within a set time, otherwise it issues RESET or NMI; it also resets on supply-voltage faults. It depends on code with a predictable loop time and guarantees that a malfunction cannot leave the processor locked up offline. RESET is the more reliable output.

Power, compliance and traces

CMOS power, regulation, RF emissions, trace termination and bus loading.

CMOS power: voltage, frequency and heat

Learn

Modern processors need anywhere from microwatts to tens of watts. **Power scales roughly linearly with clock speed** (the lecture: 0.13 W at 8 MHz and 0.38 W at 20 MHz for the 68K, a bit more than strictly linear) and drives the cooling requirement.

Why CMOS dissipates power: a CMOS gate held high or low passes only leakage; during a transition there is a small window when both transistors conduct (and the load capacitance has to be charged). So power depends on:

1. the number of gates switching per unit time (save power by **disabling the clock** to unneeded parts of the processor),
2. the supply voltage: it falls as **Vcc squared**,
3. rise and fall times: it **rises with slower edges**,
4. clock frequency: it rises **linearly**,
5. over-clocking can exceed the package's ability to move heat from die to heat sink.

This is why a 3.5 GHz processor needs a massive cooler while a data logger on a whale can run for a year on a few AAA batteries.

Worked example

A chip dissipates 2 W at 5 V and 16 MHz. Estimate its power at 3.3 V and 16 MHz.

1. Voltage factor = (3.3 / 5)^2 = 0.4356.
2. Power = 2 W x 0.4356 = **0.87 W**.
3. Halving the clock as well would halve it again, to about 0.44 W.

> **Key fact:** P is proportional to Vcc^2 and to clock frequency (more transitions per second), rises with slower edges and with the number of gates switching; save power by gating clocks to idle blocks and sleep modes. Heat is limited by the package and heat sink.

Power distribution: central versus local regulation

Learn

Power distribution has two approaches:

- **Central regulation:** one well-regulated supply distributes power to each board; each board needs only a **local filter capacitor**. The supply may cost more than a loosely regulated one, and you must worry about the **voltage drop** between the supply and the boards (see the trace-resistance lesson).
- **Local regulation:** each board has its own regulation circuit fed from a loosely regulated supply (for example a 78M05 regulator): **additional cost and heat dissipation** on each board.

When **calculating power requirements**, compare **typical versus worst-case** current specifications.

Worked example

Three boards each draw 1 A at 5 V and are 2 m from the supply through wires with 0.05 Ω resistance each way. Why might you regulate locally?

1. Drop = 1 A x (0.05 + 0.05) = 0.1 V per board (and more with all three on one cable).
2. A 0.1 V loss on a 5 V rail is 2% and varies with load.
3. A local regulator fed from a slightly higher voltage removes that variation.

> **Key fact:** Central: one tightly regulated supply feeds all boards (each needs only a filter capacitor), maybe costlier, watch distribution voltage drop. Local: each board regulates from a loosely regulated supply (extra cost and heat per board). Size the supply from worst-case currents.

RF emissions and compliance engineering

Learn

Compliance is a design-in issue: **RF emission limits are one of the most difficult engineering hurdles**, and suppression should be **designed in rather than fixed later**. Techniques are usually handed down from experienced designers (companies hire consultants or send engineers to classes). The lecture's list:

- spread-spectrum clock oscillators (spread the RF energy over a range of frequencies),
- avoid current loops,
- shield the clock lines on inner layers or run parallel guard traces,
- avoid long clock lines and long bus runs,
- avoid fast logic edges (ALS preferable to FCT if the delays allow),
- use microcontrollers and ASICs where possible,
- ferrite suppression cores on cables,
- shield locally rather than the whole chassis,
- avoid switching power supplies,
- run at the **slowest acceptable clock speed**,
- use processors with **sleep modes**,
- terminate long traces in their characteristic impedance,
- reverse engineer products that have solved the problem.

Worked example

A board's clock harmonics fail an emissions test at 144 MHz. Which two changes from the list are cheapest to try first?

1. Slow the edges or the clock if timing margin allows (slowest acceptable clock, ALS instead of FCT).
2. Shorten, guard or terminate the clock trace.
3. A ferrite on the cable is next.

> **Key fact:** Spread-spectrum clocks; avoid current loops; shield or guard-trace clock lines on inner layers; keep clock and bus lines short; slower logic families; use microcontrollers/ASICs; ferrite cores on cables; shield locally; avoid switching supplies; slowest acceptable clock; sleep modes; terminate long traces.

Trace termination

Learn

PC board traces are **transmission lines**. Without termination, a signal **reflects at the receiving end**, causing RF interference and lowering the noise margin. Impedance control becomes an issue when a signal's rise time approaches the **round-trip propagation time**.

When to terminate (rule of thumb): **length >= tR / (2 x tPR)**, where tR is the rise or fall time (whichever is faster) and tPR is the propagation rate, about **150 ps per inch** on a normal PC board. Example: a 0.5 ns edge gives 500 ps / (2 x 150 ps) = **1.67 inches**.

Methods:

- **Parallel:** a resistor R = ZT at the receiver.
- **Thévenin:** R1 and R2 with R1 || R2 = ZT; set the Thévenin voltage at the switching point; do not use with floating outputs.
- Add a **capacitor** (in series with the resistor) to reduce the steady-state current drain.

Worked example

A 1 ns edge on a board with 150 ps/inch. Terminate traces longer than?

1. Length >= 1,000 ps / (2 x 150 ps/in).
2. = 1,000 / 300 = **3.33 inches**.
3. A faster 0.25 ns edge would cut that to 0.83 inches.

> **Key fact:** Terminate when length >= tR / (2 x tPR), with tR the fastest edge and tPR about 150 ps/inch (0.5 ns gives 1.67 in). Methods: parallel R = ZT; Thévenin R1 II R2 = ZT (set the voltage at the switching point; not for floating outputs); add a capacitor to cut DC current.

Bus loading and buffering

Learn

Bus loading: as more inputs connect to a single line, the **capacitance** loading increases, and a processor's output can drive only a limited number of loads. For standard TTL the average output can reliably drive **10 inputs** (the terms are **fan-out** and **fan-in**).

For large memory systems it is common to **buffer** the address and data busses: the **74xx245** is a **bidirectional** bus transmitter/receiver (for the data bus) and the **74xx244** a **unidirectional** buffer (for the address bus). The cost: buffers **add to the propagation delay** of processor-to-memory transactions.

Worked example

A processor output must drive 32 memory chips. How do you handle it?

1. 32 loads exceeds the roughly 10-input limit.
2. Put a 74xx244 buffer on each address line (and 74xx245 transceivers on data) so each buffer drives its share.
3. Re-check the memory timing, since the buffer adds delay.

> **Key fact:** Every input adds capacitance, and a driver can reliably drive only a limited number (standard TTL about 10: fan-out). For large memory systems buffer the busses: 74xx245 bidirectional transceiver on data, 74xx244 unidirectional buffer on address; it adds propagation delay.

Specifications and partitioning

Design specifications, stakeholders, hardware versus software and the cost trade-offs.

Design specifications and stakeholders

Learn

The **design specification** is the formal roadmap for what you are supposed to design. It generally lists: processor type, speed, quantity and memory mapping of RAM, ROM type/size/location and in-service reprogrammability, upgrade plan, data bus width, user environment (office or automotive), operating and storage temperature range, humidity, mechanical shock and vibration, power and cooling requirements, I/O and connectors, custom ASICs, IP purchases, special CAD tool needs... It may contain hundreds of items and a list of optional **wants** versus **musts**.

Stakeholders to talk to before designing:

Stakeholder	Cares about
Software team	performance goals, tools, time, memory, legacy code
Marketing	cost goals, use models, competition, upgradability
Mechanical engineering	board space, connector location, chassis mounting, cooling
Compliance engineering	shielding, foreign regulations, temperature range, mechanical environment
Component engineering	new part qualification, availability, parts to avoid or already in use, obsolescence, preferred vendors

Worked example

Marketing wants the product to be upgradeable. Which spec entries does it affect?

1. 'Upgradability plan' directly.
2. ROM type and reprogrammability (field upgrades).
3. Memory size margin and connector choices.

> **Key fact:** Spec: processor, speed, RAM and memory map, ROM type/size/reprogrammability, upgrade plan, bus width, environment, temperature/humidity, shock/vibration, power, cooling, I/O connectors, ASICs, IP, CAD tools ('musts' versus 'wants'). Stakeholders: software, marketing, mechanical, compliance, component engineering.

Hardware versus software partitioning

Learn

Partitioning is deciding what goes in hardware and what in software. Embedded processors closely link them, so a significant effort goes into splitting the algorithm; mistakes here can produce an inferior product, and the process is **iterative**. Hardware people lean toward hardware, software people toward software.

Hardware is fast but adds real cost to each product; software is slow but very flexible, and its real cost (development time, performance) is often hidden.

	Hardware	Software
Speed	10x, 100x or more	much lower
Per-unit cost	adds to the BOM, power, board space, RFI	none
Up-front cost	NRE about $100K, tools $50K-$100K per seat	cheap tools
Schedule	about 3 months	less certain
Risk	only 50% of ASICs work first time; royalties	bugs fixable even in the field
Volume	cost spread over volume	insensitive to volume

The lecture's example is a laser printer engine: data formatting, mechanical control, laser control and error handling around the algorithm.

Worked example

A function must run 50 times faster than the CPU can manage. Hardware or software?

1. 50x is beyond what software can reach without a much bigger processor.
2. Dedicated hardware (an ASIC or FPGA block) gives 10-100x.
3. Then check the cost, NRE and risk against volume (next lesson concept).

> **Key fact:** Hardware: fast (10x-100x), simpler processor, but BOM cost, complexity, NRE ~$100K, ~3 months, ASIC 50% first-pass success, royalties, costly tools. Software: no BOM impact, fixable in the field, cheap tools, volume-independent, but slower, needs more processor/memory/power, schedule risk. Decide by speed and cost; iterate.

Hardware versus software cost analysis

Learn

The lecture's example adds an **ASIC** to an embedded system:

	1,000 units/yr	500,000 units/yr
Cost per ASIC	$14.00	$6.00
Part total	$14,000	$3,000,000
Development (2 engineer-years)	$300,000	$300,000
NRE foundry charges	$200,000	$200,000
Amortised tools	$25,000	$25,000
Fixed total	$525,000	$525,000

Hardware totals: **$539,000** at 1,000 units and **$3,525,000** at 500,000. The **software** solution costs $300,000 to develop at either volume (about $300 per device at 1,000 units, $0.60 at 500,000).

Then the business questions: the ink-jet printer with the hardware solution prints **4** pages per minute and sells for **$139**; the software solution prints **1** page per minute and sells for **$89**. Risks: a **30%** chance the ASIC needs a second foundry pass (**$200,000 and 2 months**); software teams averaged a **3-month overrun** on the last 4 projects; the **Consumer Electronics Show is in 4 months**. There is no formula answer: you weigh cost, volume, performance, price and schedule risk.

(Slide note: the slide prints the ASIC's fixed cost spread over volume as $525 and $1.05 per device; those are the fixed costs only: $525,000/1,000 and $525,000/500,000.)

Worked example

At what volume would hardware and software total the same, if software is $300,000 flat and hardware is $525,000 plus $6 per unit?

1. $300,000 = $525,000 + $6 x V has no positive solution: software is already cheaper at every volume in this model.
2. Hardware only wins on performance and price, not on cost.
3. A model where software has a per-unit cost (a bigger processor) would give a break-even volume.

Key fact: Total = fixed development (NRE, tools, engineering) + unit cost x volume; per device = total / volume. Hardware: $525,000 NRE + $14/unit (1,000 units: $539,000) or $6/unit (500,000 units: $3,525,000). Software: $300,000 flat. Then weigh price (4 ppm at $139 versus 1 ppm at $89) and risk (30% ASIC respin $200K + 2 months; software overruns about 3 months).

Tools for real-time design and debugging

What makes embedded systems different, debug monitors, ROM emulators, logic analyzers, in-circuit emulators, prefetch and cache headaches, BDM and JTAG.

What makes an embedded system different, and how it gets built

Learn

Why embedded systems differ from desktops: dedicated to a task or tasks; a rich variety of processors (over 300 types); cost-sensitive designs; **real-time performance constraints**; often used with a **real-time operating system (RTOS)**; **software failure can be life-threatening**; power-consumption limits; a wide environmental range; **fewer system resources**; code may all be in **ROM**; **specialized design tools**; possibly on-chip debug resources.

Embedded software development. C or C++ source goes through a **compiler** (with include files); assembly source through the **assembler**; both give **relocatable object modules** (an optional librarian makes a **user library**). The **linker** combines them with libraries under a linker command file and produces an **absolute object module** and a **link map**, which goes to the **device programmer** or the **target development system**.

When are you done? For many developers it is when the system seems to work and does not crash regularly. But if the performance does not meet requirements, how do you decide how to improve it? Two types of performance failure: **time-critical** (the system won't work at all) and **time-sensitive** (it works, but not to specification). Other failures: inadequate resources under full load (such as `malloc()` errors) and **low-duty-cycle failures** (about one per week or less), which are the hardest to catch.

Hardware/software integration starts when the hardware passes the smoke test and is turned over to the software team: load the image, validate the hardware against the software design, integrate, find the bugs, and test for performance and reliability.

Worked example

A motor controller misses a deadline once every few days. Which failure type is that, and why is it hard?

1. It works most of the time, so it is a **time-sensitive** (or low-duty-cycle) failure rather than time-critical.
2. It is rare (low duty cycle), so a debugger that stops the system or an occasional glance will not see it.
3. You need a tool that records in real time and triggers on the event.

> **Key fact:** Dedicated, real-time, cost- and power-sensitive, fewer resources, often ROM, special tools. Build: compile/assemble to relocatable objects, link (with libraries) to an absolute module and link map. Failures: time-critical (won't work), time-sensitive (works but below spec), resource exhaustion (malloc), rare low-duty-cycle bugs.

Debug monitors and ROM emulators

Learn

Debug monitor: the simplest way to debug a microprocessor system. It assumes the **processor and memory are stable**. It is a block of code that may be linked into the user's code, loaded separately (taking over key interrupt vectors) or resident in ROM; it talks to the host over a serial (RS-232) or Ethernet link, and overrides execution by being a higher-priority interrupt.

- **Run control:** examine and modify memory and registers, single step, run to a breakpoint, load code.
- **Breakpoint:** the debugger replaces the instruction with a trap to its entry point and keeps the original; each single step saves and restores the whole machine context.
- **Pros:** low cost ($0 to under $1K), provides most services software designers need, simple serial link, good once hardware is stable.
- **Cons:** needs stable target memory (not for hardware turn-on); the debug kernel is a serious perturbation: the application stops, interrupts are disabled, code is not at full speed, cannot trace in real time; cannot set breakpoints in ROM; needs a serial resource; can be trashed by runaway code.

ROM emulator: a quick way to download ROM-based code into a target through the **ROM socket**, with a **virtual UART** so a target without a serial port can still talk to the debugger. **Advantages:** cost-effective ($1K-$5K), generic, high-speed download, large overlay memory, **real-time trace of ROM code**, **breakpoints in "ROM"**. **Limits:** stable memory required; only for code in standard ROMs (not ASICs or on-chip-ROM microcontrollers); real-time trace only if the program executes directly from ROM, not from RAM.

Worked example

A new board's RAM is not yet working. Can you use a debug monitor on it?

1. The monitor needs stable memory in the target.
2. The RAM is suspect, so the monitor itself cannot run reliably.
3. Use a **ROM emulator** or logic analyzer to bring up the hardware first.

> **Key fact:** Monitor: code (in ROM or loaded) with serial link; run control (examine memory/registers, step, breakpoint, load) but not real time, needs stable memory, can be trashed by runaway code. Cost $0-<$1K. ROM emulator: replaces the ROM via its socket; fast download, real-time trace of ROM code, breakpoints in ROM, virtual UART; needs stable memory and standard ROM; $1K-$5K.

The logic analyzer as a debugging tool

Learn

The **logic analyzer (LA)** is the premier tool for the digital hardware designer: think of it as a **very low-resolution digital storage oscilloscope with 30 to 400 input channels**, costing from $400 to over $25,000. It can also act as a **pattern generator**, providing digital stimuli.

Operation: inputs from the target (16-256 channels with an adjustable trigger threshold) go to clock and trigger logic and into a memory; a PC or built-in screen is the interface.

- **Timing mode:** the LA's internal clock decides when inputs are recorded. With a 100 MHz processor and a 1 GHz LA, **10 samples per processor cycle** and resolution of about +/- 1 ns; gives a waveform display, most useful to the **hardware designer**. The lab's LogicPort samples at 500 MHz internally, with a 2K-state buffer extended by data compression.
- **State mode:** collection is synchronised to the **processor clock**, one snapshot per cycle of the busses or other signals; most useful to the **software designer**; post-processing reduces bus-level data to **instructions** and even **C functions**; data is **time-stamped** so software timing can be measured.
- **Real-time trace:** the trace memory is a **circular buffer**; the **trigger point** decides when to stop overwriting. With a 16M-deep buffer and the trigger at the middle, 8M states are recorded before and

8M after. Events are recorded in "negative time".
- **Trace specification:** triggers can use levels of conditions (address, data, status), ranges, don't cares, counters and timers (the LogicPort uses counter, range, timer and logical AND/OR/NOT; HP analyzers have an 8-level sequence state machine and link to high-level languages).
- **Benchmarking:** markers in the code (for example writing 5555 on entry to a function and AAAA on exit) let the LA time functions from the time stamps.

Pros: versatile, inexpensive units work quite well, both timing and state mode, complex triggering, and the trace can be abstracted to the level you need. **Cons:** loses relevance when pin-level behaviour does not reflect internal operation (instruction cache on); both branch destinations in cache is the limit; needs many tiny probes on pins as close as 0.005 in.

Worked example

A 16M-state buffer with the trigger set at 25%: states before and after?

1. Before = 25% of 16M = **4M** states.
2. After = 75% = **12M** states.
3. A later trigger point shows more history; an earlier one more of the aftermath.

> **Key fact:** 30-400 channels ($400 to $25,000+). Timing mode (internal clock, waveforms, +/- one sample period); state mode (processor clock, bus states, time-stamped, post-processed to instructions and C functions). Complex triggers (levels, ranges, counters); circular buffer shows events before the trigger; can benchmark with markers. Limits: caches, prefetch, tiny pins.

Timing analyzer sampling, transitional timing and glitches

Learn

A **timing analyzer** samples each input against a single user-set voltage threshold: above it is shown as 1, below as 0. The result is a one-bit picture of the waveform. It is not a parametric tool (use a scope for rise time) but it shows **many signals at once** and their timing relationships.

- **Resolution against record length.** Each sample uses one memory location, so a faster sampling rate means a **shorter acquisition window**. At 4 ns a 4K memory covers only 16.4 microseconds.
- **Placement ambiguity.** An edge is placed at the sample after it happened, so it can be late by up to **one sample period**.
- **Transitional timing** stores only a sample that follows a transition plus the time since the last transition: **two locations per transition and none when idle**. That keeps 4 ns resolution while catching later data bursts.
- **Glitches.** Because the analyzer tracks transitions between samples, it can flag any line that **crosses the threshold more than once between samples**, and can trigger on it to show what led up to a rare failure (for example a glitch that occurs once every 36 hours).

Worked example

A burst of fast pulses is followed 30 microseconds later by a second burst. The analyzer must resolve 4 ns. Does a 4K memory with plain sampling catch the second burst?

1. Window = 4,096 x 4 ns = **16.4 microseconds**.
2. The second burst starts at 30 microseconds, after the memory is full, so it is **missed**.

3. With transitional timing, the idle gap costs no memory, so both bursts fit if they have only a few transitions.

> **Key fact:** Samples a signal against one threshold. Every sample uses one memory location, so faster sampling gives a shorter record (4 ns x 4,096 = 16.4 microseconds). An edge can be misplaced by up to one sample period. Transitional timing stores only transitions (two locations each). A glitch is more than one threshold crossing between samples.

Logic analyzer trigger types and negative time

Learn

An oscilloscope starts the trace right after the trigger. A logic analyzer's **trigger (trace point)** is different: it **captures continuously** and **stops after the trace point is found**, so its memory also holds what happened **before** it, called **negative time**. That is how you see what led up to a fault.

Two ways to set the trace point on a timing analyzer:

- **Pattern trigger:** a pattern of highs and lows across the lines. Channels 0, 2, 4, 6 high and 1, 3, 5, 7 low is 0101 0101, or **55** in hex. Hex (or octal, ASCII, decimal) is much easier on 16 or 24-bit buses.
- **Edge trigger:** the same idea as a scope trigger, but with the level fixed at the logic threshold. Many devices are level-dependent but clocked on an **edge**, so edge triggering starts the capture as the device is clocked; delay the trace point if you want to see the outputs after the propagation delay.

Worked example

You want to start the capture when an address bus A15..A0 equals $41B0. What pattern do you enter, and why use hex?

1. $41B0 = 0100 0001 1011 0000 in binary.
2. Enter **41B0** in hex: four digits instead of sixteen 1s and 0s, and less error-prone.
3. Set the position so some of the memory holds negative time, to see the instructions that led to this address.

> **Key fact:** It captures continuously and stops after the trace point, so it shows data before the trigger (negative time). Pattern trigger: a pattern of highs and lows across lines (e.g. 0101 0101 = 55 hex). Edge trigger: start on a clock edge when a device is clocked.

State analyzer sequence levels and selective storage

Learn

A **state analyzer** records the bus states on each processor clock. A single trigger point is often too blunt, so it has **sequence levels**, a list of steps like:

1. find A (else go to level ...)
2. then find B
3. trigger on C

This lets the analyzer **qualify what it stores more accurately** than one trigger point, for example to enter a **subroutine from one specific place** in the program.

Sequence levels make **selective storage** possible: store only a portion of the whole. To debug a squaring routine, tell the analyzer to **find the start address, store everything in between, and stop storing when it finds the end address**. Nothing outside the routine uses memory.

Worked example

A routine from $1000 to $10A0 is called from many places. You want only the call made from $2400. How do you use sequence levels?

1. Level 1: find the instruction at **$2400** (the call).
2. Level 2: then find **$1000**, the start of the routine; start storing.
3. Level 3: find **$10A0** and stop storing.

Calls made from other places never satisfy level 1, so they are not stored.

> **Key fact:** Sequence levels: 'find X, else go to level..., then find Y, then trigger' to qualify storage more accurately than a single trigger (e.g. reach a subroutine from one specific caller). Selective storage: store only part of the run, e.g. start storing at a routine's start address and stop at its end address.

The in-circuit emulator: run control, overlay memory and real-time trace

Learn

When a debug monitor cannot work (the processor-to-memory interface is dead, code runs wild, rare events must be seen, or the monitor perturbs the system), the **in-circuit emulator (ICE)** provides **absolute control of the processor under all conditions**. It integrates three things:

1. **Run control** (like a debug kernel): peek/poke memory, modify registers, single step, breakpoints, disassemble, download code. A **BREAK** signal (from the user, an illegal access, a trace event or a breakpoint) forces an **NMI**; the target NMI and data buffers are blocked, emulator ROM is mapped in, the monitor saves the context and loops waiting for commands in a **mailbox**. **The processor isn't stopped**; it is running the monitor from shadow memory.
2. **Overlay memory:** substitute memory mapped in blocks anywhere in the address space (a high-speed mapper RAM selects emulator or target memory), with attributes: **ROM** (no writes), **guarded** (break on write), code or data space, shared (simulated I/O), no-memory, coverage.
3. **Real-time trace:** a **dedicated logic analyzer** inside the emulator with symbolic triggering (using file formats and the link map), linked to hardware breakpoints; records processor pins at the right clock phase; post-processing disassembles and shows C/C++ source lines with the assembly.

Vendors include Hitex, Microtek, SofTec, Lauterbach, Nohau and iSystems. The **ideal emulator** would be non-intrusive (no loading or propagation delay), available with first silicon, compatible with all compilers, rugged, easy and inexpensive.

Pros: control is guaranteed independent of target hardware; overlay memory permits tracing code that would run from ROM; traces program flow and filters prefetches. **Cons:** about **$20K**, costly for a team, a new emulator per processor, complex and fragile, availability lags silicon, cannot easily track microcontroller variants, **hard to connect to surface-mount parts** (500-pin BGA, 0.009 in centres), viewing cached activity is difficult or impossible, and it is not the tool for high-level code design.

Worked example

A target's ROM is blank and its RAM faulty. Can an ICE still start debugging?

1. The ICE's run-control monitor and overlay memory are in the emulator, not the target.
2. So the processor can execute from emulator memory even with no working target memory.
3. This is the ICE's advantage over a debug monitor.

> **Key fact:** An in-circuit emulator replaces the processor and gives (1) run control with absolute control even when the target is broken (BREAK -> NMI -> shadow monitor in emulator ROM), (2) overlay memory mapped over target memory with ROM/guarded/code/data attributes, (3) real-time trace with symbolic triggers and break on complex sequences. Cons: ~$20K, per-processor, fragile, packages, caches.

Trace headaches, bond-out chips, BDM and JTAG

Learn

Trace system headaches: (1) **prefetch queues** and (2) **cached processors**: the pins no longer show what the program does. The limit of usefulness is when **both possible branch destinations are within the cache**. This is exactly the effect you saw in Lab 3's trace.

Solutions: dedicate on-chip resources for real-time trace. Intel: **bond-out chips**; Motorola: **instruction-flow status bits**; AMD: **traceable cache**; National: **program counter bond-out**. A *bond-out chip* is a special version of a commercial part that brings extra internal signals to pins. AMD's 29K ran **two processors in lockstep**: the slave outputs the program counter on its address bus, bypassing the cache, so tools can track the real PC. Bond-outs are costly in small volumes, and when the chip vendor is also the tool vendor they can lock out competitors (the "scorched earth" approach behind the 1992-1994 Intel/AMD lawsuit); generally not practised except by Texas Instruments. The significance of the address of a **non-sequential fetch** is that it tells the tool where a branch went, which is what the cache hides.

Chip vendors to the rescue: "without development tools they can't sell microprocessors." They dedicate silicon to **external debug support**: Freescale **BDM**, **JTAG** and **Nexus**; AMD and Intel JTAG; ARM modified JTAG; MIPS **EJTAG**. Run control and limited breakpoints are inside the chip and accessed through a serial port: **N-wire debug interfaces**.

- **BDM** ("the wiggler"): Freescale standard, used on ColdFire, mostly replaced by JTAG.
- **JTAG (IEEE 1149.1):** developed for **board test**, replacing "bed of nails" testers: all outputs on the board form one long serial bit chain (hundreds to thousands of bits), each cell sniffing an output; it shows stuck, shorted or open nodes. Link internal registers into the loop, make them drivable, add debug registers: an on-chip debug core. **Pros:** de facto standard, one low-cost tool (under $100 to $1,000) for any JTAG device, the method of choice for ASIC cores, suits RISC. **Cons:** slow (bus cycles built by shifting bits, loops up to about 10,000 bits), loops hand crafted and rebuilt when the core is re-spun, not good for tool vendors.

Worked example

Why might a trace show an instruction that never executed?

1. A prefetch queue fetches the next instruction early, even before a branch is resolved.
2. If the branch is taken, the prefetched instruction is discarded.
3. A bus trace sees the fetch but cannot know it was discarded: a headache the on-chip trace features were added to cure.

> **Key fact:** Prefetch and cache hide program flow from external traces; bond-out chips and traceable caches expose it but are costly. Vendors instead put debug logic on the chip: Freescale BDM (the 'wiggler', mostly

> replaced), JTAG IEEE 1149.1 (originally board test) used as an N-wire debug interface: one cheap tool for many chips, slower, loops hand-crafted.

Lab 1: using the logic analyzer

The pseudorandom number generator, state-mode set-up and triggers, timing-mode propagation delay, and the report.

The pseudorandom number generator circuit

Learn

A **pseudorandom number generator** is built from an arbitrarily long chain of **D-type flip-flops** arranged as a shift register, mixed with **XOR gates** whose inputs are distributed among the Q outputs and whose outputs feed the stages of the register. It is "pseudo" random because **the sequence of numbers will eventually begin to repeat itself**.

N-input XOR: the output is TRUE when the number of TRUE inputs is **odd**. It can be built as a cascade of two-input XORs: f(a,b,c) = C XOR (A XOR B). The result is a circuit that outputs 1 when the number of 1s at its inputs is odd and 0 when it is even, useful as a **parity generator** or modulo-2 adder.

Simulation step (40% of the lab grade, in Logisim): twelve D-type flip-flops, eight XOR gates (two-input to match the hardware), one XOR output fed back to the first flip-flop, XOR inputs connected to Q or ~Q outputs of the eight flip-flops, XOR outputs connected randomly to the inputs of the other seven. Run it, log the state sequence (Logisim's logging exports to Excel), and count the states before it repeats; over 100 non-repeating states is easy to achieve.

Worked example

Three-input XOR with inputs 1, 1, 1. Output?

1. A xor B = 1 xor 1 = 0.
2. Then 1 xor C(=1) = 0 xor... compute: (1 xor 1) xor 1 = 0 xor 1 = **1**.
3. Three ones is an odd count, so the output is 1, consistent with the parity rule.

> **Key fact:** A shift register of D flip-flops whose inputs are fed by XOR gates connected to randomly chosen Q outputs. An N-input XOR is 1 when an odd number of inputs are 1 (parity), usable as a cascade of two-input XORs. Being a finite state machine, the sequence eventually repeats. The hardware has 12 flip-flops and 8 XORs, and the analyzer reads 8 data bits.

Setting up the logic analyzer for a state measurement

Learn

The instrument. The Intronix **LogicPort** is a measurement head connected by USB to an application on the lab PC. It samples **32 signals plus 2 clock signals at up to 500 MHz** and is powered from the PC's USB port (the yellow LED shows it is powered), so it shares the PC's ground: be careful with grounds, and measure between grounds if in doubt.

Connections for the lab: the board's ground, the clock (CLK1) and eight data bits D0-D7 (D7 is the most significant). The 8-bit output is shown as two hexadecimal digits, giving 256 possible numbers.

Set-up: remove default signals, add CLK1 and Data0-Data7, make a **group** (for example PNG-data) so the eight bits read as hex, save the set-up (not on the lab desktop). Check **Wire Status** shows **T** (toggling) for every signal.

State mode: *Setup > Sample mode > State mode (external sample clock)*. Choose the clock edge carefully: the D-FFs change on the rising edge, so should you sample while the outputs are changing? Set the time base to the crystal frequency (it does not change the states, only the timing display). Set the **pre-trigger buffer to 1%**, use *Trigger Immediate*, and read the *State List*.

Pattern trigger: to trigger on E5h, set Data7..Data0 = 1,1,1,0,0,1,0,1 in the Pattern A column (click each bit to cycle 1, 0, X), then Setup > Trigger > Pattern A is True. Time before the trigger is shown as negative. Export to CSV for Excel, and count how many states occur before the sequence cycles.

Worked example

Pattern for the byte 5Ah?

1. 5Ah = 0101 1010 binary.
2. Data7..Data0 = 0,1,0,1,1,0,1,0.
3. Four of the eight bits are 1.

> **Key fact:** Connect GND, D0-D7 and CLK1 (D7 is the MSB); group the 8 data signals; check the wire status shows T (toggling); set State Mode with the board clock on the chosen edge; pre-trigger 1%; trigger immediate, or pattern A = data value by setting each bit 1/0/X. Pre-trigger states show as negative time.

Timing mode and measuring propagation delay

Learn

For the last part of Lab 1 switch to **timing mode** and set the sample rate to **500 MHz** (2 ns per sample). Set the *Edge A* column for Clock1 to a rising edge (up arrow) and trigger on *Edge A occurs 1 time*; set the pre-trigger buffer to about 10%; take a *single acquisition*; zoom until you can see the data signals changing after the rising clock edge. Place **cursor A** on a rising clock edge and **cursor B** where the data first becomes stable and read the interval: the **propagation delay from the clock edge to stable data**.

Then compare with data sheets: find the propagation-delay tables for the **74LS175** (quad D flip-flop) and the **74LS86** (quad two-input XOR). Sometimes you will find minimum and maximum values, sometimes only maximum, rarely min, typical and maximum. Compare the clock-to-output delay and discuss.

Finally, from the schematic compute the **maximum and minimum clock frequency** that would not violate any timing restriction and name the **critical path**: the path with the longest total delay (flip-flop clock-to-output plus XOR gates plus the next flip-flop's set-up) fixes the minimum clock period, hence the maximum frequency.

Worked example

Flip-flop clock-to-Q is 25 ns max, two XOR levels at 22 ns each, set-up 20 ns. Maximum clock frequency?

1. Critical path = 25 + 2 x 22 + 20 = 89 ns.
2. Maximum frequency = 1 / 89 ns = **11.2 MHz**.
3. The lab's 8 MHz clock (125 ns) leaves margin.

> **Key fact:** Switch to timing mode, 500 MHz sample rate (2 ns resolution), trigger on a rising edge of CLK1; zoom in; place cursor A on the clock edge and B where the data is stable; read the interval; compare with the 74LS175 (D-FF) and 74LS86 (XOR) data sheets; the critical path (flip-flop + XOR delays + set-up) sets the maximum clock frequency.

Lab 1 grading and the report

Learn

The lab handout's scoring: **Step 1, simulation: 40%** of the grade; **Step 2, using the logic analyzer: 50%** (with several **10%** questions); and the **deliverables (20%)**. (The handout's percentages are as printed; the course syllabus is the authority on how the lab counts toward the course grade.)

The report should look like a **research paper**: an introduction; what you measured; what you observed; any conclusions; and a bibliography including classmates who helped ("good form"). For example, if you sampled on the rising or falling clock edge, explain why. Include a paragraph or more on what you thought of the lab and suggestions for improving it.

Worked example

Why does the handout ask you to justify the clock edge you sampled on?

1. D flip-flops change after the rising edge.
2. Sampling on the rising edge risks capturing changing data; sampling on the falling edge gives stable values.
3. Explaining the choice shows you understand setup, hold and propagation delay.

> **Key fact:** Step 1 simulation 40%; Step 2 logic analyzer 50% (including 10% questions on the cycle length, the measured propagation delay, the data-sheet comparison, and the frequency limits); deliverables/report 20% (as the handout states). Report like a research paper with citations and lab feedback.

Lab 2: real-time debugging of a Z80

The Z80 op-code fetch cycle, the lab board, the analyzer procedure and decoding the sample program.

The Z80 op-code fetch (M1) cycle

Learn

The **Z80** is a state machine: every operation is synchronised to the clock. The **op-code fetch** (the first part of an instruction, telling the processor what to do) is the **M1 cycle**: **4 complete clock periods, T1 to T4**, with the **~M1** pin asserted (low) at the start of T1. At **4 MHz** each T state is **250 ns**, so the fetch takes **1 µs**.

- Shortly after the rising edge in T1 the address starts appearing on the 16 address lines; at the **falling edge in T1** the Z80 asserts **~MREQ** (a processor-to-memory operation rather than I/O) and **~RD** (a read). Now the address is stable for this M1 cycle and the memory may respond.
- During T2 the memory places the data (the op-code) on the data lines; on the **rising edge in T3** the Z80 **reads the op-code**. Soon ~M1, ~MREQ and ~RD return to idle.
- During T3 and T4 the processor decodes the instruction. Ordinary memory reads and writes that follow take 3 clock cycles because no decoding is needed.

On the analyzer, find the row where **~M1, ~RD and ~MREQ are all low**: an op-code is being fetched and the address is stable. The ~RFSH refresh address and ~WAIT behaviour can be ignored in this lab (on later boards ~RFSH became ~IOREQ).

Worked example

A Z80 runs at 2 MHz. How long is an M1 cycle?

1. T state = 1 / 2 MHz = 500 ns.
2. M1 = 4 x 500 ns = **2 µs**.
3. An instruction with an op-code and one operand (a 3-cycle read after the M1) takes longer than one fetch.

> **Key fact:** Four T states (T1-T4). T1: address valid; ~M1 low; at the falling edge ~MREQ and ~RD go low (address now stable). T2: memory puts data on the bus. Rising edge of T3: Z80 reads the op-code; signals go inactive. T3-T4 decode (and refresh). At 4 MHz: 250 ns per T state, 1 μs per fetch.

The Z80 experiment board and handling the ROM

Learn

The board (2.5 x 3.8 in, a standard ExpressPCB size, **4 layers** with power and ground as the inner layers, 10 mil trace and space; red traces mostly north-south on the top, green east-west on the bottom, vias joining them):

1. **4 MHz Z80A** in a 40-pin DIP.
2. **32K x 8 EPROM (27C256)** at **0000H-7FFFH**.
3. **32K x 8 RAM (62256)** at **8000H-FFFFH**.
4. **4 MHz crystal oscillator**.
5. **DS1232 supervisor** generating a clean RESET.
6. **74LS00** quad two-input NAND for **glue logic**.

The **40-pin connector** matches the analyzer's (**designing for testability**). EPROM or FLASH parts both fit (two jumpers set the type; FLASH is no longer used because it was more easily damaged).

Handling: keep memory chips in **conductive foam**, ground yourself, check pins are straight, notch toward the edge connector, lever up to insert and rotate to lock; power polarity matters; do not pull the power plug by the wires.

Worked example

Which chip does the Z80 fetch its first instruction from, and why?

1. After RESET the Z80 starts at 0000H.
2. 0000H has A15 = 0, which selects the EPROM.
3. So the first instruction comes from the ROM, which is why a ROM sits at address 0.

> **Key fact:** 4 MHz Z80A (40-pin DIP); 32K x 8 EPROM (27C256) 0000-7FFF; 32K x 8 RAM (62256) 8000-FFFF; 4 MHz oscillator; DS1232 supervisor for a clean RESET; 74LS00 glue logic; 40-pin analyzer connector (designing for testability); 4-layer board (power and ground inner layers, 10 mil trace and space). EPROM in a ZIF socket, notch toward the connector, kept in conductive foam.

Z80 logic analyzer procedure: state mode, triggers and timing

Learn

Procedure (condensed from the handout):

1. Match the analyzer's inputs to the board's signals; use groups so the address and data buses read as hexadecimal.
2. Set **state mode, recording on the falling edge of CLK1**.
3. Insert the sample EPROM (LAB 2-0) with the power off, connect the analyzer first and then the power, and check that all signals show **T**. If a control or data signal is not toggling, press RESET.
4. **Timing mode** (50-100 MHz for a 4 MHz clock) for a first trace; then find the published bus-cycle timing for a **Z80A instruction fetch** and set the analyzer to its **500 MHz** maximum.
5. Answer: the time from the clock edge to ~M1 and ~MREQ **becoming active**, and **becoming inactive**; do they fall within the published min/max; why is the resolution limited to **+/- 2 ns** (screen capture required)?
6. **State mode capture from RESET:** the Z80 starts at 0000H; qualify the address with ~MREQ and use ~RESET going high (and optionally ~M1) as the trigger so the first fetch is captured. Hold RESET, press single acquisition, release when 'prefill complete' appears.
7. Export the state trace as a CSV file to Excel and compare it with the sample program's listing.

Worked example

Why trigger on ~RESET rising rather than on the address 0000H alone?

1. Address 0000H also appears later, for example after a jump to 0000H or if the program wraps.
2. ~RESET going high marks exactly the start of execution after reset.
3. Qualifying with ~MREQ ensures it is a memory address, not an I/O one.

> **Key fact:** Set groups for the address and data buses; state mode on the falling edge of CLK1; insert the sample EPROM; connect cable and power; verify T (toggling) on all signals; timing mode at 50-100 MHz then 500 MHz to

measure ~~M1~~/MREQ timing against the Z80A spec (+/- 2 ns resolution); state-mode trace from RESET with Edge A on ~RESET rising, qualified by ~MREQ; export CSV to Excel.

Decoding the Z80 sample program from the trace

Learn

The lecture's sample program in Lab 2 does nothing useful: it exists to give signals to trace. The program starts at 0000H when ~RESET goes from low to high (the DS1232 holds the Z80 in reset until power is stable or you release the button).

Address	Bytes	Instruction
0000H	`09H`	`ADD HL,BC`: add the 16-bit register pairs, result in HL (1 byte)
0001H-0002H	`3EH 55H`	`LD A,55H`: op-code plus operand (2 bytes)
0003H-0005H	`21H 7CH 9AH`	`LD HL,9A7CH`: **low byte first** (3 bytes)

Note that `LD HL,9A7CH` appears as 21H, **7CH, 9AH**, not 21H, 9AH, 7CH: the Z80 is **little endian**. All three encodings here are checked by assembling them on this site's Z80 assembler.

To decode a trace: the **M1 rows** (with ~M1, ~MREQ and ~RD low) are op-code fetches; the following memory-read rows are operand bytes; the op-code tells you how many operand bytes to expect, so you can find where the next instruction begins. Real-time traces also show data reads and writes from the instructions themselves.

Worked example

After `LD HL,9A7CH` *at 0003H, where does the next op-code fetch occur?*

1. The instruction is 3 bytes: 0003H, 0004H, 0005H.
2. The next instruction starts at **0006H**.
3. The analyzer shows an M1 row with address 0006H.

Key fact: Find the ~M1 rows for op-codes; the following memory-read rows are operand bytes (or data reads/writes). Instruction length is set by the op-code: 09H (ADD HL,BC) 1 byte; 3EH nn (LD A,n) 2 bytes; 21H lo hi (LD HL,nn) 3 bytes with the operand low byte first. Next instruction address = current + length.

Lab 3: debugging a 68008 system

The 68008 board, reset vectors, the prefetch queue, chip selects, wait states and logic-analyzer measurements.

The 68008 experiment board

Learn

Lab 3 studies the **MC68008**, the 8-bit-bus version of the 68000: internally a full **32-bit** processor, externally only an 8-bit data bus. Motorola designed it for low-cost systems that still needed compatibility with the 68000 family. It is a good lab processor because the LogicPort logic analyzer has only **34 channels**, and the 8-bit bus keeps the number of signals to watch manageable.

The board:

- **8 MHz 68008** in a 48-pin DIP (8 more pins than the Z80); **A16-A19** give it **1 MB** of address space.
- **32K x 8 ROM** (27C256) at **0000H-7FFFH**.
- **32K x 8 RAM** (62256) at **8000H-FFFFH**.
- **8 MHz crystal oscillator**.
- **DS1232 supervisor**, which generates a clean RESET (both polarities).
- **GAL22V10** programmable logic device, which makes the memory-interface control signals and a variable wait-state generator.

Differences from the 68000 (Appendix B): the 68008 has ~DS in place of ~UDS and ~LDS; address lines **A0-A19** instead of A1-A23; and data lines **D0-D7** instead of D0-D15. For a given clock frequency the timing relationships are identical.

Worked example

Which chip does the address $9ABC select?

1. $9ABC = 1001 1010 1011 1100: A15 = 1 and A16-A19 = 0 (it is below $10000).
2. A15 = 1 selects the **RAM** (8000H-FFFFH).
3. Address $1234 has A15 = 0, so it is in the **ROM**.

> **Key fact:** 8 MHz 68008 (48-pin DIP, 8-bit data bus, A0-A19 = 1 MB); 32K x 8 ROM (27C256) at 0000-7FFF; 32K x 8 RAM (62256) at 8000-FFFF; 8 MHz oscillator; DS1232 supervisor for RESET; GAL22V10 PLD that makes chip selects, ~~RD~~/WR and a wait-state generator.

What the 68008 does after RESET

Learn

After a **hard RESET** the 68008 does not start executing at address 0. First it reads two long words from the bottom of memory:

1. The long word at byte addresses **0000..0003** goes into the **stack pointer, A7**.
2. The next long word at **0004..0007** goes into the **program counter, PC**.

Then it begins executing at the address in the PC. The lab's program therefore begins:

```
stack   EQU     $0FFFF
pc_st   EQU     $00001000
        ORG     0000
        DC.L    stack           * initial stack pointer
        DC.L    pc_st           * initial program counter
        ORG     pc_st
start   NOP                     * the first instruction executed
```

The two ORGs exist for exactly this reason. This is the same **exception vector table** idea from the memory-map lesson: the processor begins from an address stored in the vector table, and the stack address is assigned there too. With 8 bits of data bus each long word takes four byte fetches, so you see these eight reads at the start of every trace after RESET.

Worked example

A 68008 board's first eight ROM bytes are 00 00 FF FF 00 00 20 00. Where do the stack and the program start?

1. First long word: $0000FFFF becomes **A7**.
2. Second long word: $00002000 becomes the **PC**.
3. The first instruction fetched is at **$2000**, and the stack grows down from $FFFF.

> **Key fact:** It reads the long word at 0000-0003 into A7 (the stack pointer), then the long word at 0004-0007 into the PC, and executes from that address. So code must place two vectors at address 0 (ORG 0 / DC.L stack / DC.L start) and the program at the address in the second.

The prefetch queue and why traces look out of order

Learn

The Z80 was a good first logic-analyzer subject because there is **no ambiguity** between the trace and what the processor does. The 68008, like modern processors, uses **prefetching** to improve performance, which can confuse anyone who reads only the address, data and status buses.

The 68008 has a **two-word prefetch queue**. For our purposes there are two general rules:

1. **On a branch instruction, the instruction after the branch is prefetched before the branch is taken.** If the branch is taken, that instruction is never executed.
2. **When an instruction writes to memory, the write usually happens after the next instruction has been prefetched.**

The Z80, in contrast, fetches the next instruction only when it knows where it will come from, and writes occur in the same instruction cycle.

The lab's list file shows this at $102E: `MOVE.B D0,$00008020` ($13C0 $0000 $8020). In the trace its write of $AA appears only **after** the op-code $45F9 of the next instruction (`LEA $00008AAA,A2`) has been fetched. Similarly `MOVE.L #$11111111,(A2)` writes its data after the following `NOP` ($4E71) has been prefetched.

The pipeline, stepwise for `MOVE.L #$11111111,(A2)` ($24BC $1111 $1111):

1. The op-code word $24BC is fetched into the first stage of the queue (IRC).
2. It moves to the instruction register (IR) and the upper half of the data enters IRC.
3. $24BC moves to the instruction decode register (IRD) and is decoded; IR and IRC now hold its operand.

4. The **next** instruction ($4E71) is prefetched into IRC.
5. Only now does the instruction complete and write $11111111 at $8AAA-$8AAD. The bus is busy with the write, so the following op-code ($D792) cannot be prefetched yet.

Writes appear in **big-endian** order: high-order word high byte first, down to low-order word low byte.

Worked example

Why might you see the instruction after a `BRA` *in a trace even though it never ran?*

1. The queue holds two words, so the word after the BRA was fetched while the BRA was being decoded.
2. The BRA is then taken, so the prefetched word is discarded.
3. The trace shows the fetch, but the instruction never executed.

> **Key fact:** The 68008 has a two-word prefetch queue: it fetches the next instruction before the current one has finished. Rules: (1) on a branch, the next instruction after the branch is prefetched before the branch is taken; (2) a write to memory usually happens after the next instruction has been prefetched. The Z80 does not prefetch, so its traces read in program order.

Function codes and the GAL's chip selects

Learn

The board uses a **GAL22V10B** to synthesise its control signals. The lab's equations (OR is #, AND is &, NOT is !; ASTR is the active-low ~AS) are:

```
OpFetch = !FC0 & FC1        (program fetch, "Pfetch")
DatFetch = FC0 & !FC1       (data read or write)
~RD    = ASTR # !RW
~WR    = ASTR # RW
~CSRAM = ASTR # !A15 # A16 # A17 # A18 # A19
~CSROM = ASTR # A15  # A16 # A17 # A18 # A19
```

Reading these: the outputs are **active low**, and an OR of terms is low only when **all** the terms are low. So:

- ~CSROM is low when ~AS is asserted, **A15 = 0** and **A16-A19 = 0**: addresses **0000-7FFF**.
- ~CSRAM is low when ~AS is asserted, **A15 = 1** and **A16-A19 = 0**: addresses **8000-FFFF**.
- ~~RD is low while ~AS is asserted and R/~~W is high (a read); ~~WR is low while ~AS is asserted and R/~~W is low (a write).

The 68008 outputs a **function code** on every cycle. Using only FC0 and FC1 the board can tell **program fetches** (ROM) from **data reads and writes** (RAM); FC2 would separate supervisor from user mode, which the lab does not need. Because the function codes are routed into the GAL rather than to the analyzer, you cannot measure when they are asserted directly: you see the GAL's Pfetch and Dfetch outputs, which can add up to 15 ns of delay.

Worked example

Which signals are active for a read of the byte at $9000?

1. $9000 has A15 = 1 and A16-A19 = 0, so **~CSRAM** goes low while ~AS is asserted.
2. R/~~W is high for a read, so **~~~~RD**** goes low too.
3. Since it is a data access, DatFetch (not OpFetch) is true.

> **Key fact:** ~~CSROM low when ~AS asserted, A15 = 0 and A16-A19 = 0 (0000-7FFF). ~CSRAM low when ~AS asserted, A15 = 1 and A16-A19 = 0 (8000-FFFF). ~RD = ~AS asserted with R/W = 1; WR = ~AS asserted with R/W = 0.~~ FC0/FC1 give OpFetch (program) and DatFetch (data).

~DTACK and wait states

Learn

Synchronous timing (the Z80's) needs no acknowledgement; the 68000 family is **asynchronous**: the memory system tells the processor when it is ready by asserting **~DTACK** (Data Transfer Acknowledge).

- The state of ~DTACK is **sampled on the falling edge of the clock in state S4**.
- If ~DTACK is **low**, the bus cycle **completes through state S7**.
- If ~DTACK is **high**, **S4 and S5 repeat** until ~DTACK goes low.

On the lab board the GAL22V10B contains a **shift register of three D flip-flops** that generates delayed copies of ~DTACK. The **jumper** selects how many: connecting ~AS directly to ~DTACK (position A, nearest the 68008) gives **0 wait states**; positions B, C and D give **1, 2 and 3 additional S4-S5 cycles**.

Timing arithmetic (our own calculation from the state names): a bus cycle is eight states S0-S7, each half a clock period, so it is **4 clock periods** long; each wait state adds **one more clock period**. At the lab's **8 MHz** the period is **125 ns**, so a normal cycle is 500 ns and each wait state adds 125 ns. You will see this directly as the width of ~AS increasing in the logic-analyzer measurement.

Worked example

At 8 MHz, how long is one bus cycle with the jumper in position C (2 wait states)?

1. Clock period = 1000 / 8 = 125 ns.
2. Cycle = (4 + 2) x 125 = **750 ns**.
3. ~AS is asserted for most of it, so you would measure roughly that long for the width (less the setup before ~AS).

> **Key fact:** A bus cycle is states S0-S7 (4 clock periods). ~DTACK is sampled on the falling edge in S4: low completes the cycle (to S7); high repeats S4 and S5 (one more clock period) until ~DTACK is low. The lab board's GAL shift register delays ~DTACK to give 0-3 wait states, selected by a jumper. At 8 MHz a period is 125 ns.

Logic-analyzer measurements and trigger set-up

Learn

Timing resolution. In timing mode the analyzer samples at its own clock. At its maximum of **500 MHz** each sample is **2 ns** apart, so an edge is located only to within about **+/- 2 ns**: resolution is one sample period (the lab asks you to explain this).

Triggers. Use *Edge A* on RESET (high to low) to capture from reset; use *Pattern A* to catch a specific bus cycle. For the op-code fetch at $1000, set A0..A15 = $1000, **~CSROM, ~AS, ~RD = 0**, **Pfetch = 1**, and everything else don't care. Hold RESET, press single acquisition (wait for "prefill complete"), then release RESET.

Measurement. Place cursor A on the clock edge and cursor B on the edge of interest (turn off snap-to-grid for finer placement) and read the interval in the measurement bar. Compare with the published timing table for a 68000 read/write cycle (the 68008 has the same relationships at the same clock). Example:

clock high-to-low between S0 and S1 to address valid: published maximum **62 ns** at 8 MHz; the lab example measured **28 ns**, which is within spec. Do **not** compute statistical deviations from published values; that has no meaning for a handful of measurements.

GAL warning. Signals from the GAL22V10B (Pfetch, Dfetch, chip selects) can add up to **15 ns**, so report that the actual time might be less.

Procedure safety. Power the analyzer **before** the board and remove board power first.

Worked example

The analyzer samples at 250 MHz. You measure an interval of 12 samples. What is the interval and its uncertainty?

1. Sample period = 1000 / 250 = 4 ns.
2. Interval = 12 x 4 = **48 ns**.
3. Uncertainty about +/- one sample period: **+/- 4 ns**.

> **Key fact:** Timing mode at 500 MHz resolves to one sample period (2 ns). Trigger on a pattern: address plus ~CSROM, ~AS, ~RD low and Pfetch high, or on the RESET edge. Place cursors A and B on edges, read the interval, compare with the published 68000 timing table (e.g. clock-to-address-valid max 62 ns at 8 MHz). GAL outputs can add up to 15 ns. Power the analyzer before the board.

Decoding an unknown ROM from a trace

Learn

The last part of Lab 3 repeats the detective work of Lab 2 with a **mystery ROM** from the instructor, using the analyzer in **state mode** (sampling with the processor's clock). A good first trigger is to capture the processor as it begins to run from **RESET** (a high-to-low transition on this board).

Method:

1. **Review the Prefetch Rev 3 ROM** first, all the way to where it branches back to the start, so you know what prefetch effects look like.
2. **Filter** the trace (for example in Excel) to remove extraneous bus cycles, leaving the instruction stream.
3. **Loops:** do not list every iteration. Count the loop's instructions, then take another trace set to trigger at an address **just past the loop's last address**; set the trigger point **10-20%** into the buffer to see the final iterations as it exits.
4. **Subroutines:** `BSR` or `JSR` is the flag. Note the call address and trigger when it returns (usually the next address).
5. **Verify** in the EASy68K simulator, which simulates program execution but **not bus activity**. If the program looks incomplete, it probably is: take more traces.
6. **Report** a short paragraph on what the program does as a whole (for instance, "fills a region of RAM with a constant", "sets all of memory to zero") and an assembly source file. There are **no points for describing each instruction** by itself.

Deliverables include the answer to "Why is the maximum resolution +/- 2 ns?", the completed timing table, a screen capture of a timing diagram, the filtered spreadsheet, the disassembly, the description and source, and a bibliography. Also say what you thought of the lab and suggest improvements.

Worked example

A filtered trace shows `MOVE.B D1,(A0)+` *followed by* `CMPA.L A1,A0` *and* `BNE` *back, repeating until A0 = A1. What is the algorithm?*

1. The loop writes D1 to successive bytes, advancing the pointer, until the pointer reaches the end address.
2. So the algorithm is **filling a block of memory (from A0 to A1) with the value in D1**.
3. You would state that in one sentence, not list the three instructions.

> **Key fact:** Capture from RESET in state mode; filter the extraneous bus cycles (prefetches, writes out of order) into a clean instruction list in a spreadsheet; for loops, trigger just past the last loop address (trigger point 10-20% in); for BSR/JSR, trigger at the return address; disassemble; verify in EASy68K; report the overall task of the algorithm, not instruction-by-instruction.

Handling lab equipment: ESD, ROMs, cables and power

Learn

Modern digital circuits are small, fast, and **easily damaged**. The handout's rules:

- **Static.** ICs are sensitive to static electricity: always **ground yourself** (wrist strap to a grounded point, or touch the bench ground plate periodically and whenever you return to the bench).
- **ROMs** stay in their **conductive foam**, which holds all pins at the same voltage; hold a chip by its short sides until it is time to insert it into the **ZIF** (zero-insertion-force) socket.
- **Bent pins:** it is easy to bend an IC pin. A bent pin will not seat in the ZIF socket: **do not force it**, tell the instructor. Inspect the analyzer's and board's sockets before plugging in a cable; a bent pin in the analyzer socket can destroy the analyzer.
- **Ribbon cable:** support the plastic header and work it straight out. Do not pull on the cable or one end of the header. The board's ejector ears make removal easier.
- **Power plug:** keyed so it fits only one way; **never pull the wire**; lift the white tab with a fingernail and lift the plug by its housing.
- **Order:** analyzer on first, board second; board off first.

Worked example

You are about to plug in the 40-pin cable and the socket looks odd on one corner. What do you do?

1. Stop and inspect the socket for a bent pin.
2. If you see one, do not force the cable: tell the instructor.
3. A bent pin in the analyzer can destroy it, which is a far larger cost than a short delay.

> **Key fact:** Ground yourself (wrist strap or touch the bench ground plate); keep ROMs in conductive foam; never force a bent pin (tell the instructor); remove cables by supporting the connector and working it straight out; never pull the power cable by the wire; check polarity; power the analyzer before the board.

Homework 1: your first PCB

Schematic before layout, the instrumentation amplifier, CAD tools and Gerber files.

Homework 1: schematic before layout

Learn

Homework Assignment #1 is a jump start on PCB design: design a board for an **instrumentation amplifier** sketched on a napkin. The handout's key rule: **you must never start designing PCBs without first creating the schematic diagram.** The instructor has been asked to review boards for which no schematic existed.

The sketch is not a schematic, just as a sketch of a house is not the plan to build it. The schematic includes:

- **Resistors:** value, size (1/8 W, 1/4 W...), tolerance (1%, 5%, 10%), type (carbon composition, metal film, wire-wound).
- **Op-amps:** type (manufacturer, packaging), electrical characteristics, footprint (surface-mount or through-hole), pinout, power requirements.
- **Connectors:** input, power, output.
- **Part designations:** R1, R2...; U1, U2...; J1, J2...
- **Filter and bypass capacitors:** type (electrolytic, ceramic), value (0.1 µF, 1 µF, 10 µF), polarity.

Only after the schematic is drawn, with all parts labelled and pins numbered, do you go on to the board design. This is the same workflow as the PCB lesson.

Worked example

You designed the op-amp stage but have not chosen the capacitors yet. Can you start the layout?

1. No: the schematic is not complete (no filter or bypass capacitors, no connectors).
2. A missing part on the schematic means a missing part on the board.
3. Finish and review the schematic first.

> **Key fact:** Every part with value, size/rating, tolerance and type (resistors), IC type, package, footprint, pinout and power needs, connectors (input, power, output), designators (R, U, J, C), and filter/bypass capacitors with type, value and polarity; power and ground clearly shown. Draw it first: the sketch is not a schematic.

The instrumentation amplifier: gain, supplies and matched resistors

Learn

The circuit is an **instrumentation amplifier (IAmp)**, commonly used for **low-level differential measurements** such as medical equipment. Design choices from the handout:

- **DC gain:** anything between **100 and 1,000** is usual; you decide.
- **Supplies:** single or dual. The instructor used a single **+5 V** supply and an **ICL7660** converter to make **-5 V** (fine while the current requirement is under **100 mA**). The suggested op-amp, the **OP-07**, operates from **+/-3 V to +/-18 V** (you need at least 6 V in total on a single supply); lower-voltage op-amps exist for very low supplies.
- **Resistor values:** not too big, not too small ("Goldilocks").

- **Matched resistors:** specify matched parts, or use close resistors with a **trim pot in series**. Example: two 1%, 10 kΩ resistors may differ by up to 100 Ω from nominal; put a 100 Ω variable resistor in series with the smaller and tune it using a **four-wire (Kelvin) measurement** on a good multimeter, matching them to better than **0.01%**. (Four-wire measurement is used because it removes the lead and contact resistance that would swamp a 0.01% measurement.)
- Don't forget **filter capacitors and I/O connectors**.

Worked example

Gain resistors R1 = 9.9 kΩ and R2 = 10.1 kΩ (both 1%). What trim do you need to match them?

1. The difference is 200 Ω.
2. A 100 Ω trim pot cannot cancel 200 Ω; the handout's example assumes the pair differ by no more than 100 Ω (nominal 10 kΩ, within 1% each, measured).
3. Measure the actual values, choose the closer pair or a bigger trim range.

> **Key fact:** DC gain 100-1000 (your choice); single +5 V with an ICL7660 for -5 V (under 100 mA) or a dual supply; OP-07 works from +/-3 to +/-18 V; use moderate resistor values; match critical resistors (matched parts or a trim pot in series, measured four-wire); add filter/bypass capacitors and connectors.

PCB CAD tools, Gerber files and layout choices

Learn

Step 0 and 1: pick and learn a tool. The handout's example is ExpressPCB (classic), with Sunstone's PCB123, Autodesk Eagle (free student version), Fritzing, EasyEDA and KiCad as alternatives. **No matter what CAD software you use, you will need to invest time learning it**; there is no way around a learning curve.

Lock-in vs Gerber: ExpressPCB and Sunstone software produce design files that can only be used with their own manufacturing, so you are **locked in**. To use any other fabricator (including inexpensive ones abroad, where shipping can cost more than the boards) you need CAD software that outputs **Gerber files**, the industry standard format for converting a design into a physical PCB. Eagle and Fritzing produce them.

Step 3: the board. Open the board editor (for ExpressPCB a standard 2.75 x 3.75 in outline) and decide: the **number of layers**, **mounting holes** (screw size), **location of connectors**, **ground and power busses**, **signal trace width** and **power bus width**. Place components and move them around until the positions work; it may take several iterations. (This mirrors the PCB-design lesson.)

Worked example

You want to use a cheap overseas fabricator. Which CAD tools can you use?

1. The fabricator needs Gerber files.
2. Use Eagle, Fritzing, KiCad or another tool that exports Gerber.
3. A tool that only exports its own format (ExpressPCB, Sunstone) would tie you to its own manufacturer.

> **Key fact:** Any CAD tool has a learning curve; practice on a simple design. ExpressPCB/Sunstone lock you to their manufacturing; tools that output Gerber files (Eagle, Fritzing, KiCad) let you use any fabricator. Board decisions: layers, mounting holes, connectors, power/ground buses, trace and power widths.

Homework 2: your first 68000 program

The EASy68K workflow and tracing the homework program.

Homework 2: the EASy68K workflow

Learn

Problem 1 of Homework 2 introduces the **EASy68K** editor/assembler/simulator (written by Professor Chuck Kelly; it will run on a Mac under Wine). The assembler turns your source into executable 68000 code, and an **instruction set simulator** (ISS) runs it as if on a real 68000.

Steps in the handout:

1. Install EASy68K; open the template.
2. Copy the program in by hand (**do not cut and paste**: invisible formatting characters cause errors). The eight **EQUATE** statements begin in **column 1**; the ten instructions begin **one tab stop** to the right. Name it **HW2.X68** (source files use the **.X68** extension) and save.
3. Press **F9** (or Project, or the green arrow) to assemble. This creates **HW2.L68** (the **list file**, which has the addresses, machine code and any errors; print it landscape at 9 points because lines are 132 characters) and, if there are no errors, **HW2.S68**, a **Motorola S-record** (a text form of the executable code).
4. Answer **EXECUTE** to open the simulator and **single-step** the program, watching the registers.
5. At the final `BRA start` stop and open **View > Memory** to answer.

Comments from the handout: EQU, ORG and END are **pseudo-ops** (instructions to the assembler); the **default number system is decimal**; binary uses %, hex uses $.

Worked example

Where does the handout say the equates go, and what happens if you put one after its first use?

1. Equates go first, right after the title block.
2. A label used before it is defined is flagged as an error by the assembler.
3. EQU values are substituted at assembly time, so they generate no machine code.

> **Key fact:** Install EASy68K; start a new file named HW2.X68; type (don't paste) the equates in column 1 and the instructions one tab in; save; press F9 to assemble (L68 list file, S68 S-record); if no errors choose EXECUTE; single-step; read memory with View > Memory. Default radix decimal, % binary, $ hex.

Homework 2: tracing the program

Learn

The homework program (with the loop `BRA start` replaced here by `SIMHALT` so it can be run to completion):

```
start   MOVE.W  #data2,D0     * D0 low word = $A7FF
        MOVE.B  #data6,D1     * D1 low byte = %01001111 = $4F
        MOVE.B  #data7,D2     * D2 low byte = %00010111 = $17
        MOVE.W  #data3,D3     * D3 low word = $5555
        MOVEA.W #addr1,A0     * A0 = $4000
        MOVE.B  D1,(A0)+      * [$4000] = $4F, A0 = $4001
        MOVE.B  D2,(A0)+      * [$4001] = $17, A0 = $4002
        MOVEA.W #addr1,A1     * A1 = $4000
        AND.W   D3,(A1)       * word at $4000: $4F17 AND $5555
```

The word at $4000 is $4F17 before the AND (high byte at the lower address, as the 68000 is big endian), and **$4515** after it. This answer was confirmed by running the program on the 68000 core.

Worked example

Compute $4F17 AND $5555 by hand.

1. $4F17 = 0100 1111 0001 0111; $5555 = 0101 0101 0101 0101.
2. AND bit by bit: 0100 0101 0001 0101.
3. = **$4515**.

> **Key fact:** It builds the word $4F17 at $4000 from two bytes (%01001111 and %00010111) using post-increment, then ANDs the word with $5555: result $4515 (hex), i.e. $45 at $4000 and $15 at $4001. Use bytes in memory, big endian.

Homework 3: three 68000 programs

Add longwords, search a block for a word, and find the smallest byte (signed or unsigned).

Homework 3, problem 1: add two longwords

Learn

Problem 1: write a short program in 68K assembly that adds the **long word** at one address to the long word at $0000A304 and stores the result at $0000A308. (The handout prints the first address as `$000A300`, an apparent typo for `$0000A300`.)

The solution uses three instructions, all with the **.L** size:

```
start   MOVE.L  $A300,D0     * first longword
        ADD.L   $A304,D0     * plus the second
        MOVE.L  D0,$A308     * store the sum
```

The program above, with test data $00012345 and $00054321, leaves $00066666 at $A308 (the core confirms the four bytes 00 06 66 66). A wrong size (`ADD.W`) would add only the low words and be rejected by a checker.

Worked example

Which flags are affected by the ADD.L?

1. ADD sets X, N, Z, V and C from the long result.
2. Here the sum is positive and non-zero: N = 0, Z = 0; no overflow or carry: V = 0, C = 0.
3. The following MOVE.L resets N and Z from the stored value.

> **Key fact:** MOVE.L first,D0 / ADD.L second,D0 / MOVE.L D0,result. Use .L throughout; the sum wraps modulo 2^32 and sets the flags (carry if it overflows).

Homework 3, problem 2: search a block for $AA55

Learn

Problem 2: search a block of memory for the **first occurrence of a specific 16-bit word, $AA55**, on an even word boundary. The block can be any size, so use a loop and take the **starting and ending addresses as EQUates**.

The loop (confirmed on the core with test data):

```
start_addr EQU $00002000
end_addr   EQU $00002020     * first address NOT searched
pattern    EQU $AA55
        LEA     start_addr,A0  * pointer
        LEA     end_addr,A1    * limit
        MOVE.W  #pattern,D0
        SUBA.L  A2,A2          * A2 = 0: not found yet
loop    CMPA.L  A1,A0
        BHS     done           * unsigned: reached the end
        CMP.W   (A0)+,D0       * compare, then A0 += 2
        BNE     loop
        LEA     -2(A0),A2      * A0 is one word past the match
done    SIMHALT
```

Three ideas: the **WHILE** loop shape you already know; **post-increment** by 2 for words; and the address compare is **unsigned** (BHS), not signed.

Worked example

The block is three words: $0001, $AA55, $AA55 starting at $3000. What does the loop find?

1. $3000: $0001 is not a match; A0 = $3002.
2. $3002: $AA55 matches; A0 = $3004.
3. A2 = $3004 - 2 = **$3002**, the first match (the second is ignored).

> **Key fact:** LEA start,A0 / LEA end,A1 / MOVE.W #pattern,D0 / SUBA.L A2,A2 / loop: CMPA.L A1,A0 / BHS done / CMP.W (A0)+,D0 / BNE loop / LEA -2(A0),A2 / done. The end address is not searched; words are on even addresses; A2 = 0 if not found.

Homework 3, problem 3: smallest byte, signed versus unsigned

Learn

Problem 3: find the **smallest byte** in a memory block of arbitrary size (reuse the loop structure from the search problem; the handout says 'problem #4', but the handout has only three problems). The handout calls it **tricky**: you are looking for the smallest number and the 68K 'pretty much assumes that you are always working with signed numbers'. If you read the bytes as **signed**, the largest byte is $7F (+127) and **$80 is -128, the smallest**. The instructor's solution used **BHI** (branch if higher), the **unsigned** test.

The handout's table (after `CMP D0,D1`): D1 < D0: BLT / BCS; D1 <= D0: BLE / BLS; D1 = D0: BEQ; D1 <> D0: BNE; D1 > D0: BGT / BHI; D1 >= D0: BGE / BCC (signed / unsigned). The unsigned counterparts of BLT and BGE are **BCS and BCC**, not BLO and BHS in the standard mnemonics.

The handout does not say which reading is intended, so the lessons give both solutions (confirmed on the core): the **signed** one starts D0 at $7F and skips with BGT; the **unsigned** one starts at $FF and skips with BHI.

(The handout header also says 'Spring Quarter 2024' although it is the Fall 2026 problem set.)

Worked example

Data bytes $05, $FF, $80. Signed and unsigned minimum?

1. Signed: $05 = +5, $FF = -1, $80 = -128. Minimum **$80**.
2. Unsigned: 5, 255, 128. Minimum **$05**.
3. Same bytes, different answers: choose the branch that matches the meaning of the data.

> **Key fact:** Start D0 at the largest value of the interpretation and replace D0 with D1 when D1 is smaller or equal. Signed: start $7F, skip with BGT ($80 = -128 smallest). Unsigned: start $FF, skip with BHI. The handout notes the table: BLT/BCS, BLE/BLS, BGT/BHI, BGE/BCC.

Homework 4: memory decoding

Chip enables for ROM and RAM, chips and pages for a 32-bit processor, the zero-wait-state clock limit, and 16 pages from two 74138 decoders.

Homework 4, problem 1: chip enables for two ROMs and two RAMs

Learn

Problem 1 gives four memory parts on a processor with a 20-bit address bus (1 MB space) and asks for the logic that turns address lines into the four ~CE signals.

Part	Size	Low	High	Offset lines	Decode lines	Pattern
ROM 1	128 KB (2^17)	00000H	1FFFFH	A0-A16	A19 A18 A17	0 0 0
ROM 2	128 KB (2^17)	20000H	3FFFFH	A0-A16	A19 A18 A17	0 0 1
RAM 1	64 KB (2^16)	80000H	8FFFFH	A0-A15	A19 A18 A17 A16	1 0 0 0
RAM 2	64 KB (2^16)	90000H	9FFFFH	A0-A15	A19 A18 A17 A16	1 0 0 1

Each chip's low address lines connect directly to its address pins. Each ~CE is low only when **every** decode line matches the pattern **and** the address is valid. With a NAND gate, put an inverter on each line that must be 0:

```
~CE_ROM1 = NAND( /A19, /A18, /A17, valid )
~CE_ROM2 = NAND( /A19, /A18,  A17, valid )
~CE_RAM1 = NAND(  A19, /A18, /A17, /A16, valid )
~CE_RAM2 = NAND(  A19, /A18, /A17,  A16, valid )
```

The two RAMs share A19-A17 and differ only in A16; the two ROMs differ only in A17. Addresses 40000H-7FFFFH and A0000H-FFFFFH select nothing.

Worked example

A 32 KB RAM must occupy 60000H-67FFFH on the same 20-bit bus. Which lines decode it, and what pattern?

1. 32 KB = 2^15, so A0-A14 are offset lines.
2. The decode lines are A19-A15 (5 lines).
3. 60000H = 0110 0000 0000 0000 0000, so A19-A15 = 0 1 1 0 0.
4. ~CE = NAND(/A19, A18, A17, /A16, /A15, valid).

> **Key fact:** ROMs (2^17 bytes): offset A0-A16, decode A19 A18 A17: ROM1 = 000, ROM2 = 001. RAMs (2^16 bytes): offset A0-A15, decode A19-A16: RAM1 = 1000, RAM2 = 1001. Each ~CE = a gate that goes low only when its pattern matches and address-valid is true (NAND of true and inverted address bits).

Homework 4, problem 2: chips, pages and decoder for a 32-bit processor

Learn

Problem 2 shows a 16 Mbit static RAM (2M x 8, address pins A0-A20, ~CE, ~OE, ~WE) and a processor, the AB32X32, whose pins matter for three questions: how many chips fill the space, how many pages there are, and how to decode pages 0, 1, 2 and the last page.

Reading the processor figure:

- Address pins **A2-A31** (30 lines). A0 and A1 are not brought out; the four **byte enables ~~WE0~~-WE3** cover them.
- Data pins **D0-D31**, so every access moves up to 4 bytes.
- Other control pins: ~ADVL (address valid) and ~RD. The pins marked DC are "don't care" and ignored.

The answers:

1. **Space**: 2^30 words x 4 bytes = 2^32 bytes = **4 GB**.
2. **Chips**: a 32-bit word needs 4 byte-wide chips side by side. 4 GB / 2 MB = **2048 chips**.
3. **Pages**: one page is the row of four chips, 2^21 words (8 MB). 2^30 / 2^21 = **512 pages**.
4. **Wiring**: chip A0-A20 to processor **A2-A22**; page bits are **A23-A31** (9 lines).
5. **Decoder**: four ~CE outputs, each low only when ~ADVL is active and A31-A23 match: page 0 = 000000000, page 1 = 000000001, page 2 = 000000010, last page (511) = 111111111. Each ~CE goes to all four chips in its page; ~WE0-3 go to the matching lane's chip and ~RD to every chip's ~OE.

Worked example

A processor has address lines A2-A25 and a 32-bit data bus, and uses 1M x 8 chips (A0-A19). How many chips and pages?

1. Words: 2^24 (24 address lines), so 2^24 x 4 bytes = 64 MB.
2. Page = 2^20 words x 4 lanes = 4 chips (4 MB).
3. Pages = 2^24 / 2^20 = **16**; chips = 16 x 4 = **64**.
4. Page bits: chip A0-A19 connect to A2-A21, so the page lines are A22-A25 (4 lines, 2^4 = 16).

> **Key fact:** Address space 2^30 words x 4 bytes = 4 GB. A page = 4 chips (one per byte lane) = 2^21 words (8 MB). Pages = 2^30 / 2^21 = 512; chips = 512 x 4 = 2048. Chip A0-A20 connect to processor A2-A22; page bits A23-A31 feed the decoder (page 0 = 000000000, last page = 111111111) together with address-valid.

Homework 4, problem 3: the fastest clock with zero wait states

Learn

Problem 3 asks for the maximum clock frequency with **zero wait states** when the memory has a **15 ns access time** (from ~CE low to data valid or accepted), every decoder gate has a **2 ns** propagation delay, and a processor read or write takes **3 clock cycles**.

The chain of events for one access:

1. The address appears; it goes through the decoder gates.
2. ~CE falls after (gates x 2 ns).
3. Data is valid 15 ns after ~CE falls.
4. All of that must finish within the 3 clock periods the processor allows.

> **3 x T >= (decoder gates x 2 ns) + 15 ns and fmax = 1/T = 3 / total delay**

The answer depends on how many gate delays your decoder from Problem 2 puts in the ~CE path. With an inverter followed by one wide gate (2 gates): total 19 ns, T = 6.33 ns, **f = 157.9 MHz**. With 3 levels: 21 ns, T = 7 ns, **f = 142.9 MHz** (calc). If the clock is faster than that, either add wait states or use a faster memory.

Worked example

A 12 ns RAM, a 1-gate decoder (2 ns) and a 2-cycle access. Maximum clock with zero wait states?

1. Total = 2 + 12 = 14 ns.
2. 2 x T >= 14 ns, so T >= 7 ns.
3. fmax = 1/7 ns = **142.9 MHz**.

> **Key fact:** Total delay = (gates in the decoder x 2 ns) + 15 ns. Zero wait states needs 3 x T >= total delay, so fmax = 3 / total. Example: 2 gates -> 19 ns -> T = 6.33 ns -> 158 MHz; 3 gates -> 21 ns -> 143 MHz. Fewer gates or a faster memory raises the clock; otherwise add wait states.

Homework 4, problem 4: 16 pages from two 74XX138 decoders

Learn

Problem 4 starts from a 68000 circuit in which a 74LS138 turns A21, A22, A23 into eight chip selects, ~AS qualifying the decode (EN1 = Vcc, ~EN3 = ground). On a 24-line address bus this is 8 pages of 2 MB. The task: expand to **16 pages** using **only a second 74XX138** and no other gates.

A 74XX138 is enabled only when **EN1 = 1, ~EN2 = 0 and ~EN3 = 0**; then input ABC (A = LSB) pulls exactly one of Y0-Y7 low. Four page bits are needed, so the pages shrink to 1 MB (A0-A19 offset) and the page lines are **A20-A23**:

Pin	Chip 1 (pages 0-7)	Chip 2 (pages 8-15)
A, B, C	A20, A21, A22	A20, A21, A22
~EN2	~AS	~AS
~EN3	**A23** (on when A23 = 0)	ground
EN1	Vcc	**A23** (on when A23 = 1)
Outputs	Y0-Y7 = ~~CS0~~-CS7	Y0-Y7 = ~~CS8~~-CS15

So A23 is both the enable for one chip and, through the active-high versus active-low enable pins, the inverse enable for the other. The enables do the work an inverter and a gate would have done.

Worked example

Which chip select goes low for the address $C80000?

1. $C80000 = 1100 1000 0000 0000 0000 0000, so A23 A22 A21 A20 = 1 1 0 0.
2. A23 = 1 selects chip 2; C B A = A22 A21 A20 = 1 0 0 = 4, so Y4 is low.
3. That is page 8 + 4 = **12** (~CS12), since $C = 12.

> **Key fact:** Use page bits A20-A23. Both chips: A, B, C = A20, A21, A22 and ~EN2 = ~AS. A23 goes to ~EN3 of chip 1 (pages 0-7, A23 = 0) and to EN1 of chip 2 (pages 8-15, A23 = 1), with chip 2's ~EN3 tied low and chip 1's EN1 tied high. Each page is now 1 MB.